Slide 1

Introduction to Networking (NSWI141)

Libor Forst, SISAL MFF UK

- Essential facts concerning communications
- Layered network model (OSI vs. TCP/IP, addressing, multiplexing, ...)
- Application layer (DNS, FTP, email, web, VoIP, ...)
- Transport layer
- Network layer (IPv4, IPv6, routing, firewalls, ...)
- Data link and physical layer (switch vs. repeater, Ethernet, Wi-Fi, cabeling, ...)

Slide 2

Literature

- D. E. Comer, D. L. Stevens: Internetworking With TCP/IP; Prentice Hall 1991
- A. S. Tanenbaum: Computer Networks; Prentice Hall 2003
- C. Hunt: TCP/IP Network Administration; O'Reilly & Associates 1992
- internet resources
- Request For Comment (RFC)
- **http://www.warriorsofthe.net**

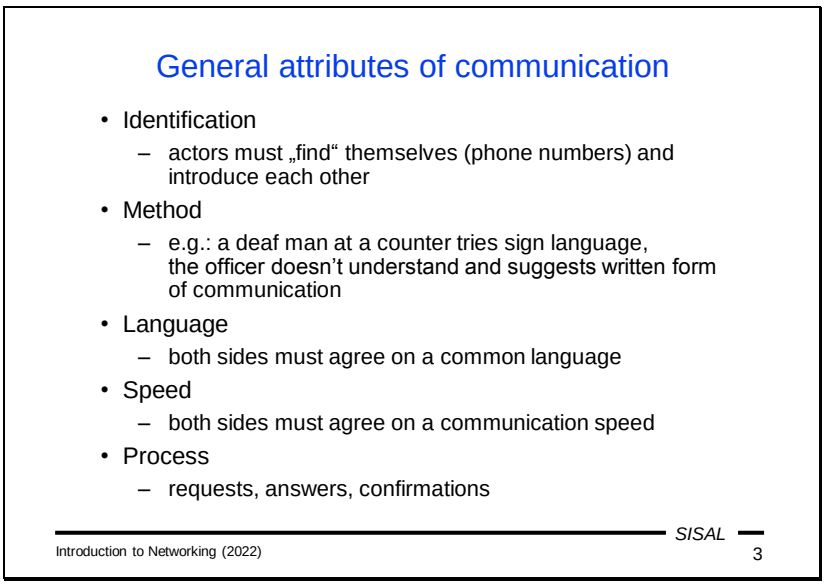
The “classic” books are listed here mostly for historical reasons. Currently, you can find many others in book stores and libraries if you prefer paper over a screen.

You can find some additional information on our topics on the Internet. Generally, information from the Internet is not absolutely correct in all cases. However, in this course we will not go into great depth, so the risk of finding wrong information is very low.

On the contrary, very precise information about most of the protocols we will study can be found in the official standards published (on the Internet) as so-called RFCs. However, it is a bit more detailed than is needed for our course, so usually only an Introduction chapter is enough for providing a sufficient overview of the topic.

For absolute newcomers, the Warriors of the Net video can be used as “a trailer” for this course, although it is a bit older and in some aspects simplifies things too much or formulates slightly inaccurately.

Slide 3



We start by studying common ways of communication between people in real life, trying to find attributes similar to those used in network communication although you don’t feel them as strict rules in real life.

Identification. In face-to-face communication you don’t need special means for identification, your eyes and brain do the job naturally. The only exception is talking with an unknown person when an introduction is needed. In other communication ways you use some identification you are already used to using – a postal address, a phone number etc.

Method. If both sides are able to use all senses, there is typically no need to agree on a method. Of course, there may be exceptions as well. For instance if you use some signals, you must agree what they mean. If you are limited in using senses, an agreement on a communication method is necessary.

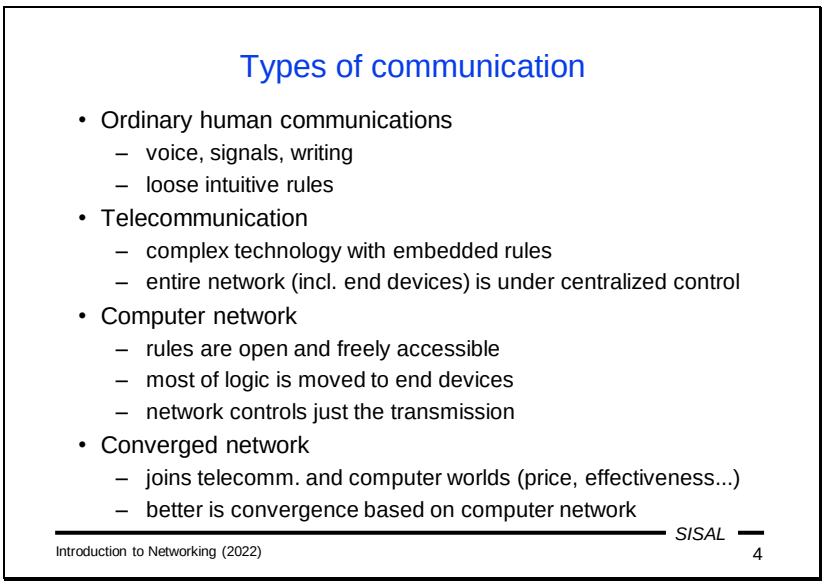
Language. In an international environment you need to agree on a language. In fact, a single common language is not always needed in this case, with natural languages you can also reach a reasonable level of understanding using two different languages, namely in the case of languages from the same family, e.g. Slavic languages.

Speed. Sometimes, speaking speed can cause comprehension problems, especially between native and non-native speakers. You will definitely feel a difference in understanding between a radio sports commentator and a teacher. In this case, again, some informal agreement on speed is needed ("slow down, please").

Process. In a normal situation, you will typically need no special strict rules. Everyone knows that in a conversation they have to address and greet their partner and say goodbye at the end. And if you forget something, it probably does not cause much trouble. However, when you are invited to visit the President, you will need to learn and obey a complicated protocol.

If we move to network communication, all these attributes will become much more formal and important; most differences can cause fatal misunderstandings while some can be accepted. But it is good to know that all of them are not something new; all of them come from real life, they have only been adapted and formalized.

Slide 4



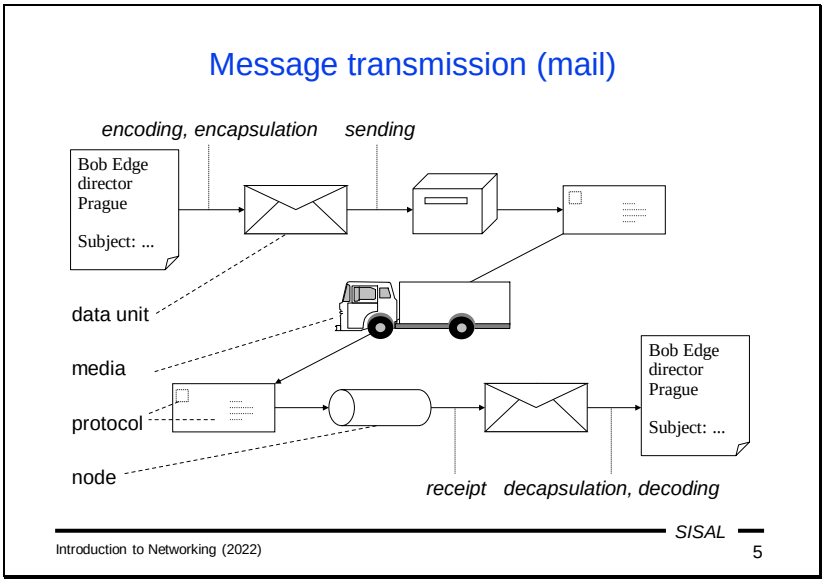
Until now, we have talked about common forms of interpersonal communication and we have come to the conclusion that very loose and intuitive rules are used here.

The first global technology-based means of communication was telephony. From the beginning it was built as a private business so all nodes were under centralized control based on complex proprietary rules embedded into the transport devices. The end device (telephone) was only a simple interface for approaching the infrastructure containing all the application and transport logic.

Computer networks were designed on a different principle: the application logic is concentrated in the end devices and the network infrastructure controls only the transmission logic. That's why it was necessary to design the system as open and publish the rules so that each application implementer can behave according to them.

When studying the communication, we must necessarily ask the question of how to connect both types of networks. Over the history, there have been several modes of such interconnection. Originally, computer networks exploited the telecommunication infrastructure (leased lines). Later, they built their own cabling systems and the situation became inverted: nowadays, telephony is very often transmitted over a computer network (Voice over IP). However, the boom of smartphones with mobile data has once again reversed the situation in many cases – the network communication is again transmitted over telephone channels.

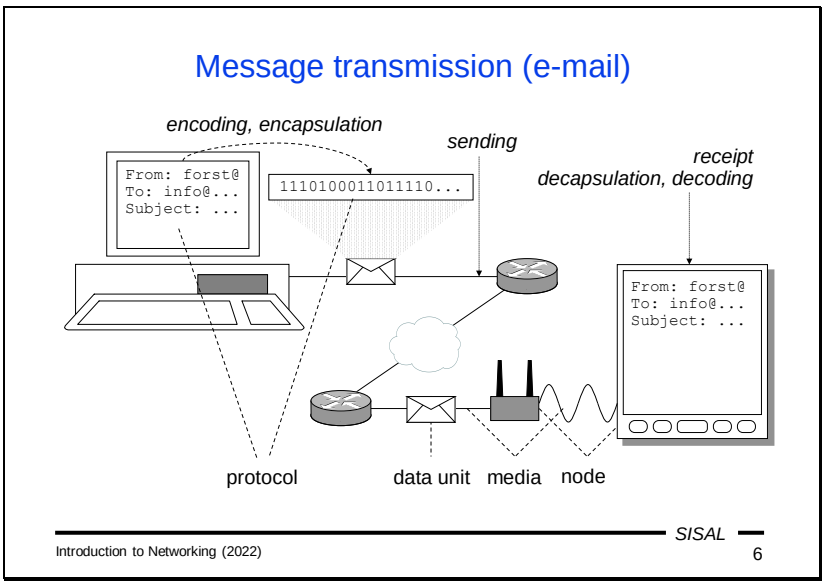
Slide 5



Let's look at the process of transmitting a message by (classic) mail. You take a sheet of paper with the message, put it in an envelope, write an address, attach a stamp and throw it into the mailbox. Then, the post organization handles the message using its infrastructure to deliver it to the recipient's mailbox. Upon delivery, the recipient opens the envelope and takes out the message.

Now, if we want to describe the process by means closer to network communication, we can start to use technical terms that are not commonly used but fully understandable. You take a sheet of paper with the message [data unit], put it in the envelope [encapsulation], write an address [target node address], attach a stamp [follow the mail protocol] and throw it into the mailbox [sending]. Then, the post organization handles the message using its infrastructure [intermediate nodes connected by various media] to deliver it to the recipient's mailbox [target node]. Upon delivery, the recipient opens the envelope [decapsulation] and takes out the message [message delivered].

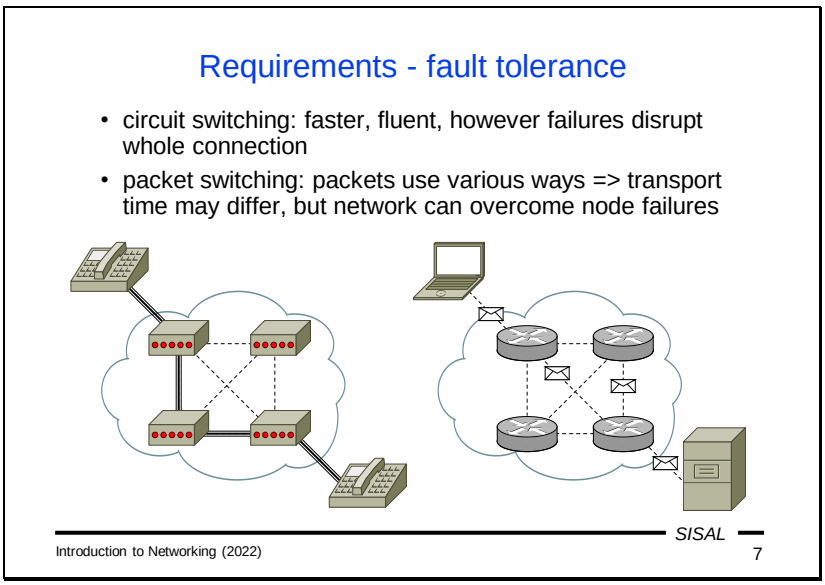
Slide 6



Using very similar words we can describe the message transmission process in the case of electronic mail. We also need to use encoding and encapsulation according a proper protocol to create a data unit that has to be send to the local network. Then the networking infrastructure handles the message over various intermediate nodes and various channels to be delivered to the target local network and through it to the end node (your personal computer, smartphone etc.) where it is decapsulated, decoded and displayed on a screen.

We can see here the same actors, the same roles and the same tasks like in the classic mail, just converted to a technical platform. The basic principle of functioning is in both cases the same.

Slide 7



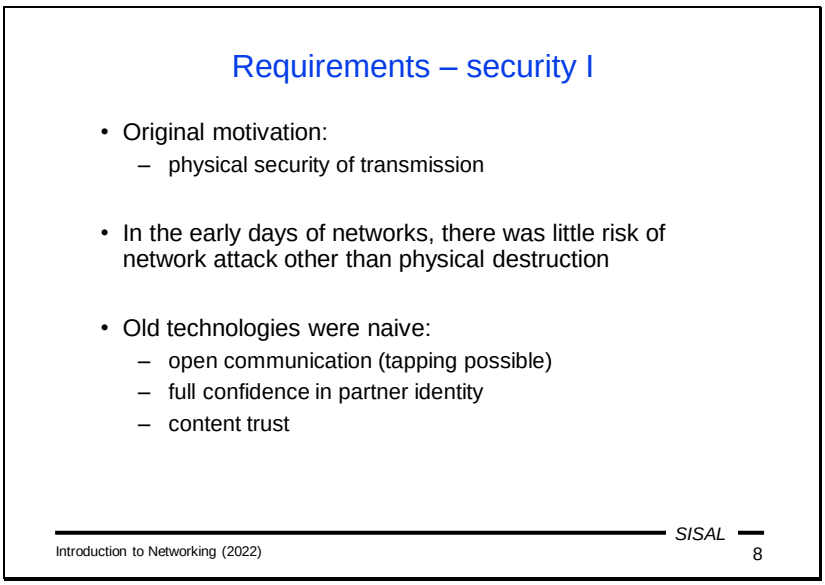
At the inception of the idea to use electronic means for message transmitting, there were several basic requirements. Some of them remained till nowadays, some of them have changed somewhat over time.

A crucial requirement was fault tolerance. In fact, the army (the US Department of Defense) was behind the whole idea. They were solving the major problem of telephone network outages in the event of an enemy attack. The telecommunication network uses so-called *circuit switching*. This means that when you call someone, the network will find a sequence of nodes (a circuit) needed to connect your device and the end device that you are calling. Once this circuit is established, all audio data follows this path. If the enemy breaks a node in the circuit, the circuit is lost.

The solution is pretty easy. We divide the data into smaller blocks so-called *packets* and let each packet to find its own way to the target node. If a node is broken, the packet will find an alternate way. It will be delayed a bit, but it will be delivered. Of course, nowadays the reason for node failure may not only be an enemy (hacker) attack, but also a power failure, misconfiguration etc. So risks still exist and therefore this principle is used by current networks for data delivery.

When comparing these methods, we must say that circuit switching is faster but unreliable, while packet switching is slower but fault-tolerant.

Slide 8

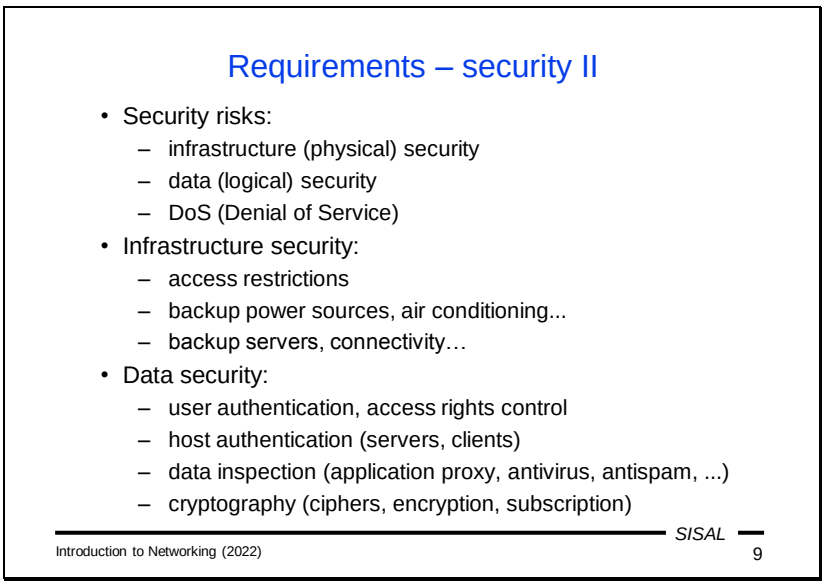


Fault tolerance is closely related to the more general concept of security, specifically physical security, i.e. the security of the physical infrastructure (nodes and cables). Surprisingly, the other side of the coin – logical (data) security was not taken into account when designing the networking model. The reason is probably that the size and price of computers at the time, along with the complexity of programming, was a major obstacle to an attacker being able to get close enough with his device to attack data.

Currently, the situation is the opposite – it is very easy to have a computer capable of connecting to the network and it is quite easy to infect a target computer by attacker software so that you can see or even control the transmitted data.

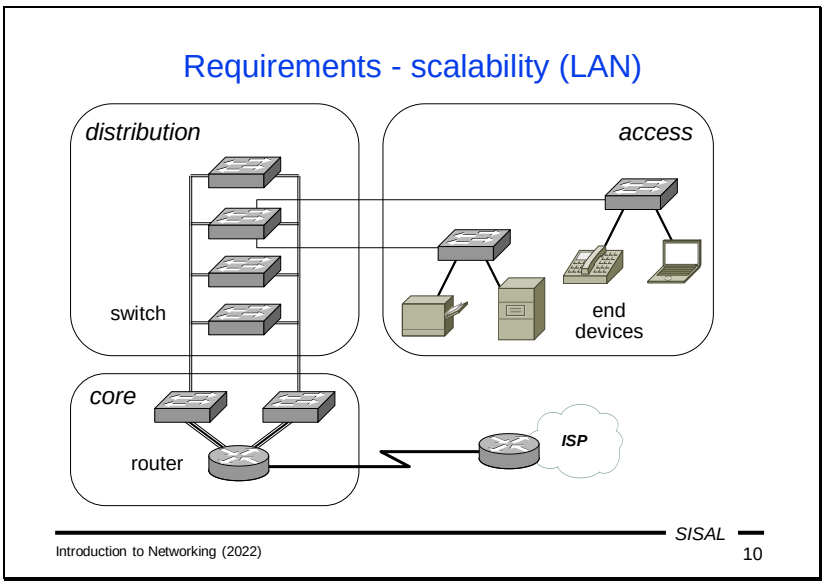
All old protocols have the same problem – they do not use any encryption and trust both the “sender” of the data and the content of the data. Unfortunately, you can see the same behavior now among many people who trust everything they see in the mail or on the web.

Slide 9



Of course, the requirement for physical security has not disappeared even today. In addition to the risks resulting from unauthorized physical access, technical risks such as power outages, connections, etc., were added to at least the same severity. Over time, data security requirements (i.e. prevention of unauthorized access to data or unauthorized data manipulation) have gained much more weight. With the progressive development of connectivity and availability of IT equipment, the risk of DoS (Denial of Service) attacks has also increased significantly. DoS is an attack where an attacker attacks a data source with massive communication, so that it cannot communicate with regular users. In addition, such an attack is usually strengthened in such a way that the attacker takes advantage of some errors/weaknesses in the implementation of a certain service and forces foreign servers to unwittingly generate increased traffic instead of him (the so-called DDoS - Distributed DoS).

Slide 10



One very important requirement is scalability. This means that the system must be designed to make adding a new computer or network easy, without having to make changes in remote locations or parts of the network. Scalability leads to logically similar but technically different approaches when adding a host to a (local) network and adding a network to the Internet.

A recommended approach to design a local network is dividing the infrastructure into three parts:

The **core** layer is the main part of the network, where the backbone of the network is terminated and the network is connected to the infrastructure of an Internet Service Provider, which mediates the Internet connection for our network. Typically, the core layer nodes are located in a special room with air conditioning and an uninterruptible power supply. The key device there is a *router*, a node interconnecting different networks. This router is an entry point of our network, connected to another router on the ISP's side. Another important device is a *switch*, a node interconnecting other switches and end devices in the network. The core layer of the local network usually contains a few main switches connecting the perimeter router to the rest of the local network.

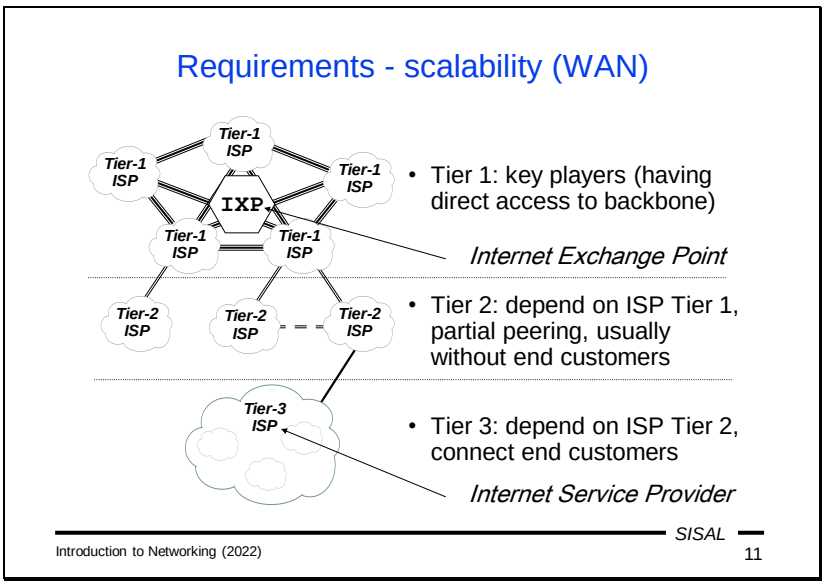
The **distribution** layer is a backbone part of the network, responsible for distributing of the connectivity to all parts of a building or campus. Sometimes, it is also called a *vertical* layer since it is very often located in a vertical well going through all floors of a building.

The **access** layer is the last part of the network, allowing access to network service to end devices, servers, printers, personal computers, phones etc. Sometimes, it is

also called a *horizontal* layer. This layer is terminated as close as possible to where end users need to connect.

This approach to local network design guarantees easy network expansion by adding new hosts. Of course, it can happen that we reach the maximum capacity of some part of the access layer, but in this case the further network expansion again affects only particular parts of the distribution layer.

Slide 11

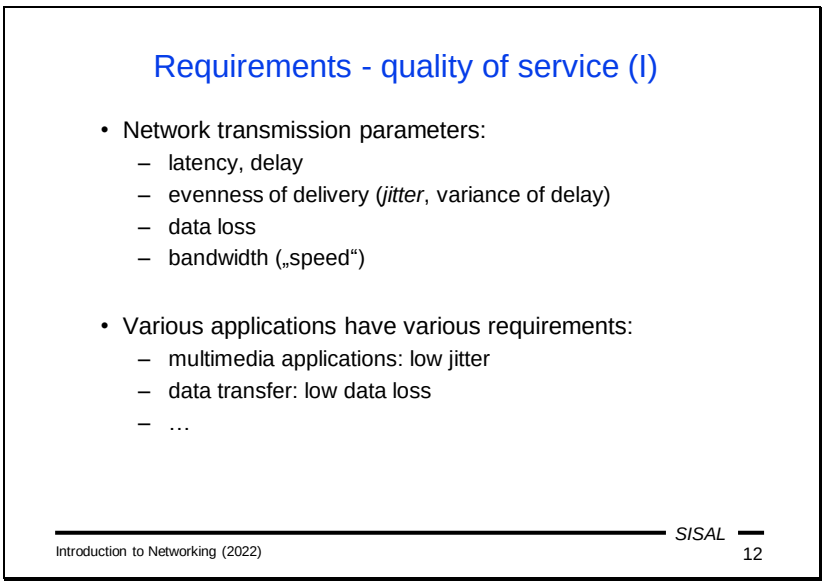


A similar concept of a three-layer model is used also for the world behind a perimeter of a local network.

At the top of the hierarchy, there is a layer called **Tier 1**. In this layer, the key players of the Internet are included, i.e. companies having direct access to the Internet backbone interconnecting all continents. Examples: AT&T, Deutsche Telekom, British Telecom.

The bottom layer of the tree is called **Tier 3**. There are included ISPs who connect end customers – typically companies, organizations, households etc. connecting their local networks to the Internet.

Somewhere in between these two extremes lies the **Tier 2** layer with companies without direct access to the backbone, whose customers are other ISPs. Typical examples are regional or national ISPs.



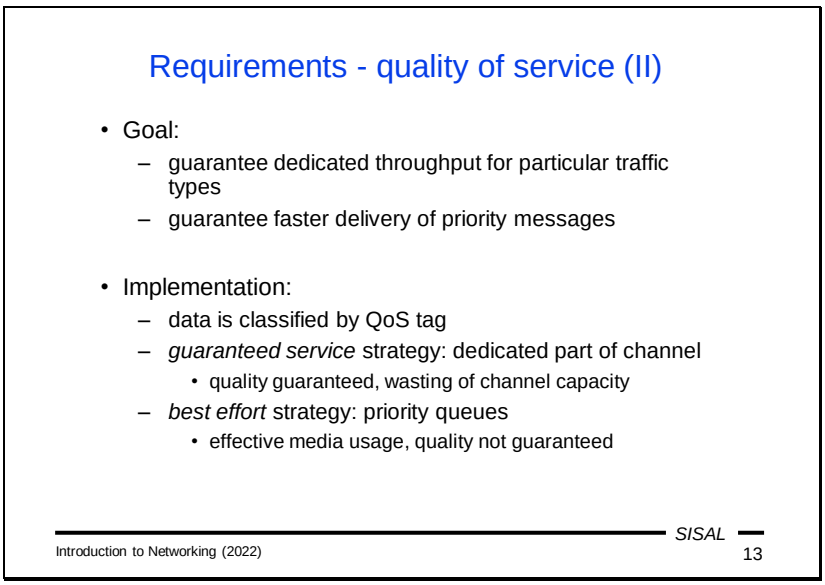
The last requirement is a sufficient quality of services provided by the network to individual applications. However, this requirement does not lead to a single correct approach since for various applications different network transmission parameters are important.

The main transmission parameters we can investigate are:

- **Latency** is in fact the communication delay, i.e. the amount of time from when a piece of data is sent to the network to when it is delivered to the destination.
- **Jitter** is the variance of the delay; this value expresses how regularly the data is delivered, i.e. whether the volume of received data fluctuates.
- **Data loss** means how often a data packet is not delivered; high data loss means either loss of information during the transmission, or the need of repeated sending of lost data packets.
- **Bandwidth** is often called “speed” which is a bit misleading since the speed of transmission of the signals over a given media is always the same (no ISP can use faster light for transmission over an optical fiber); in fact, it means how much data can be encoded and transmitted using particular physical signals, not the actual speed of the signals.

When we look at these parameters, it’s easy to understand that the insufficiency of some of them brings serious problems for various applications. For example, multimedia applications use special encodings which are robust against data loss, so it is not a problem to some extent. In contrast, high latency can be critical when using such applications in real time (e.g. phone calls), but it does not have much impact when watching a movie. But in any case, if the jitter is high, no multimedia application will work well. On the other hand, high data loss is critical for applications based on

entire data transmission, like web, email etc. Here, it causes channel congestion due to massive data resubmission.



In general, we would like to achieve two basic goals:

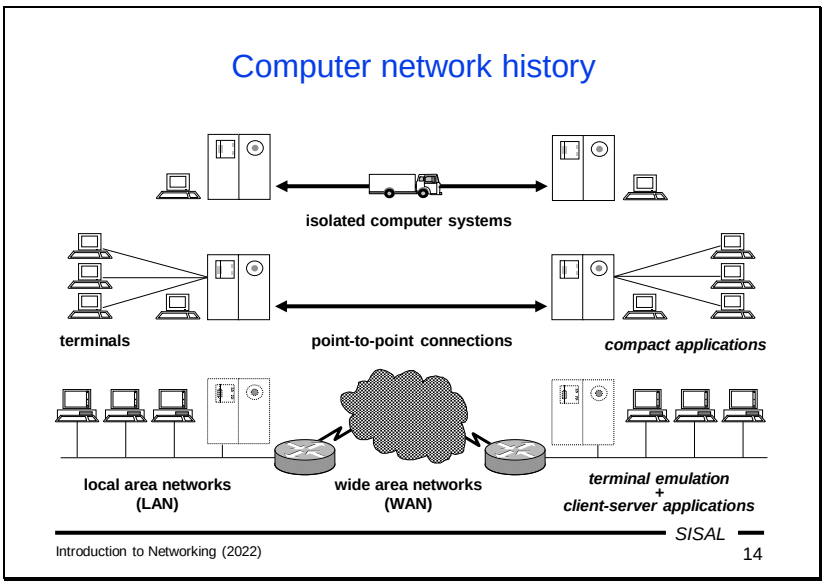
- For some traffic types, we want to dedicate a bandwidth that is sufficient for a specific application to work well.
- For some traffic types (or at least for some types of messages) we need to guarantee fast enough delivery – e.g. during a massive data transfer by one application we still need to deliver urgent messages quickly.

Here are the current methods used to achieve these goals:

- The primary condition is how to recognize different data transfer types. For this purpose, the sender marks packets with a special value called the *QoS tag*. Of course, this approach only works if all applications behave fairly. If an application starts cheating and marks all data with a high-priority tag, this system crashes.
- For applications for which we need to guarantee throughput, we can separate a part of the channel bandwidth and dedicate it to the application. This approach works well if we have enough bandwidth for all the necessary traffic. Otherwise, we have to reduce or cancel the bandwidth reserved for certain traffic. The problem with this approach is that if a particular application does not currently use its dedicated bandwidth, it is simply unused, wasting the channel transmission capacity.
- Ensuring the appropriate speed of messages according to their declared priority can be solved by the so-called *best effort* strategy using priority queues. A network node has different queues for messages with different priorities and selects messages for sending from different queues accordingly. The advantage is that there is no waste of channel capacity, but delivery speed cannot be guaranteed under heavy load. The problem is mainly on the input side of the node - if a packet arrives at the node, the node cannot recognize the packet type or priority classification until it reads a sufficient part of the packet. During this time, the node

must deal with the packet, although it may possibly have a low priority, while other higher/priority packets may wait to be received and processed.

Slide 14



What was the situation at the inception of the idea of a computer network? There were only a few isolated computers with very limited means of interaction both between a computer and users and among computers with each other. A mainframe computer usually had a single alphanumeric operator console from which all jobs were controlled. Input and output devices were limited as well – e.g. punch card readers were used to enter a larger amount of data. If it was necessary to transfer data between computers, you had to punch the data into a set of punch cards, transfer them to the other computer and read them in there.

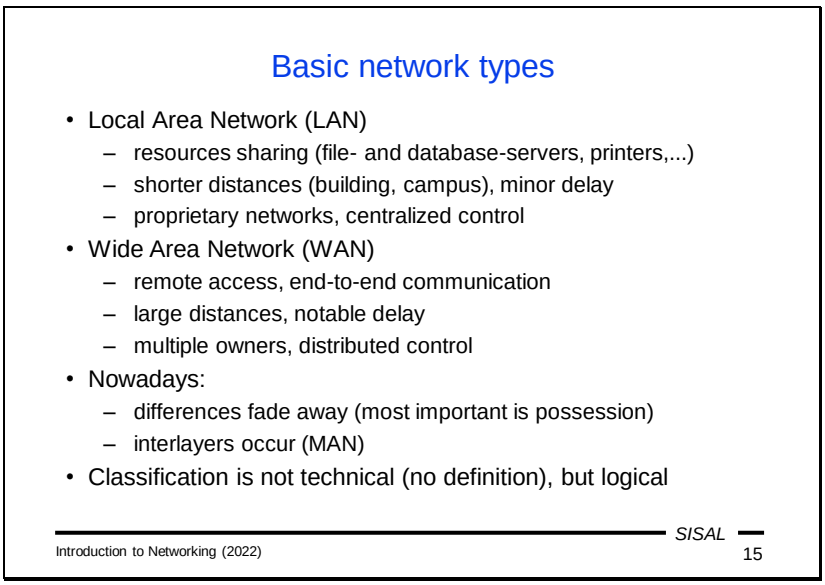
Later, computers could serve many more users, so it was necessary to add some way to allow users to access a shared computer. The solution was terminal networks. A *terminal* is a simple device with only a keyboard and monitor that can transmit keystrokes from a user to a computer and display the response on the user's screen. For communication between two computers, a proprietary point-to-point communication connection was built (usually via telephone leased lines) without any more complex structure. The applications used at that time were mostly still the same compact applications as in previous era.

In the last phase of development, two important changes took place. Random point-to-point connections were converted to a global structured network, a so-called *Wide Area Network*. On the local side, terminals were replaced by personal workstations and were connected each other (and connected to a main site-local computing facility if it remained) by a replacement of the old terminal network – a *Local Area Network*.

One of the consequences of this change was also the introduction of a new model of application, the so-called *client-server* model. It's based on the principle that a part of the computing power is transferred to the front-end computer, so an application has

two parts, a *client* running on your computer and a *server* running somewhere on the network. These two parts communicate with each other, transmit user requests from the client to the server and mediate the responses to the user. Thus, the terms client and server in their original meaning do not refer to computers, but to software.

Slide 15



After this trip into history, we can look at the terminology used for types of networks according to their scope.

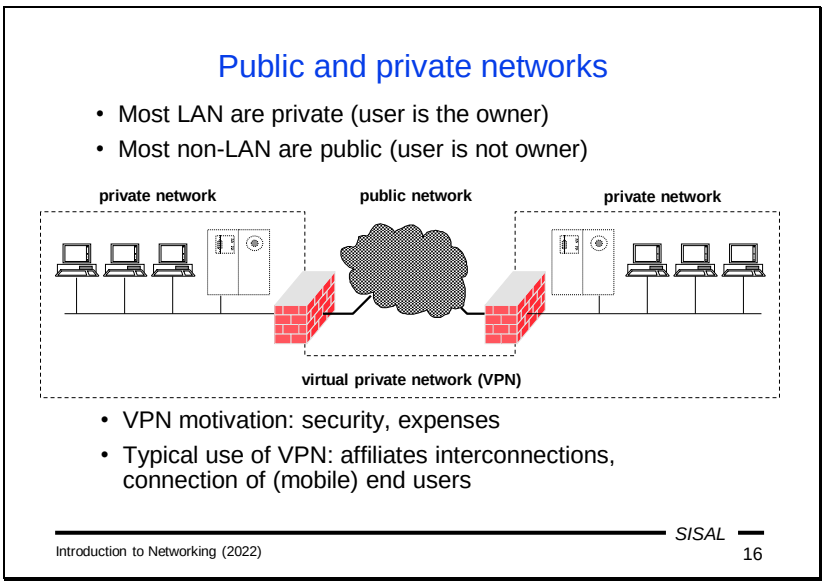
Local Area Networks are descendants of terminal networks, which means that their main purpose is to share resources between nearby computers (we can also consider an Internet connection as one of the shared resources). They are usually geographically limited by the boundaries of a building or campus whose owner builds and manages the network primarily for its employees, students, customers, etc.

A **Wide Area Network** is a global network that has replaced point-to-point connections between computers or computer centers that were built on demand. Its main purpose is interpersonal and inter-computer communication and data transfer. It is built and managed by many companies and used not only by people related to them.

In between these two ends of a wide range of networks, there are a few more categories without an entirely precise meaning, e.g. Metropolitan Area Networks.

In the past, there were also fairly well-defined technical differences between a LAN and a WAN, such as the technology used, bandwidth etc. Over time, these differences have disappeared and currently the main difference is in a relation between equipment owner and users. Unlike WAN, LANs are usually private. **No definition like "A network is a LAN is when…" exists.**

Slide 16



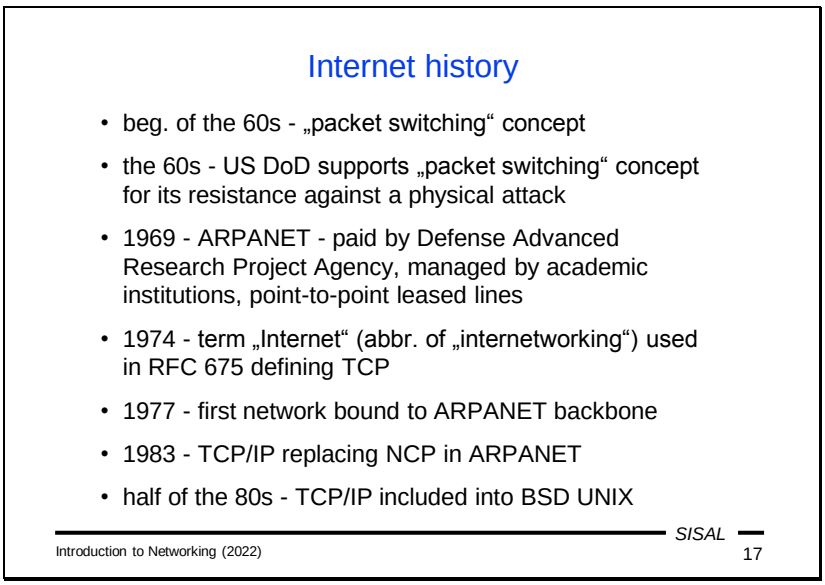
As already mentioned, LANs are in most cases private, while WANs are mostly public. However, this poses a problem for the network designer if he needs to expand a private LAN geographically over a large area, for example, between affiliates in two distant cities. Connecting such sites with private cabling is not realistic and, by using a public network, we lose privacy.

The solution is called a **Virtual Private Network (VPN)**. The basic idea is that on the perimeter of each part of a LAN there is a device that is connected to its counterpart via a public network using a so-called *VPN tunnel*. All the traffic directed to the other affiliate is encrypted at the local end of the tunnel, sent over the public network, then decrypted at the other end of the tunnel and delivered to the second LAN. For computers in both LANs, the whole mechanism is transparent and they do not need to take care of it.

Another modification of this principle is when the whole side of a tunnel is replaced by a piece of software running on a personal computer (notebook). Then this computer seems to be connected (through the tunnel) as a normal node in the private network to which it belongs.

Note: At the end of the semester we will learn another term that sounds very similar, Virtual Local Area Network (VLAN), but its meaning is totally different, so do not confuse them.

Slide 17



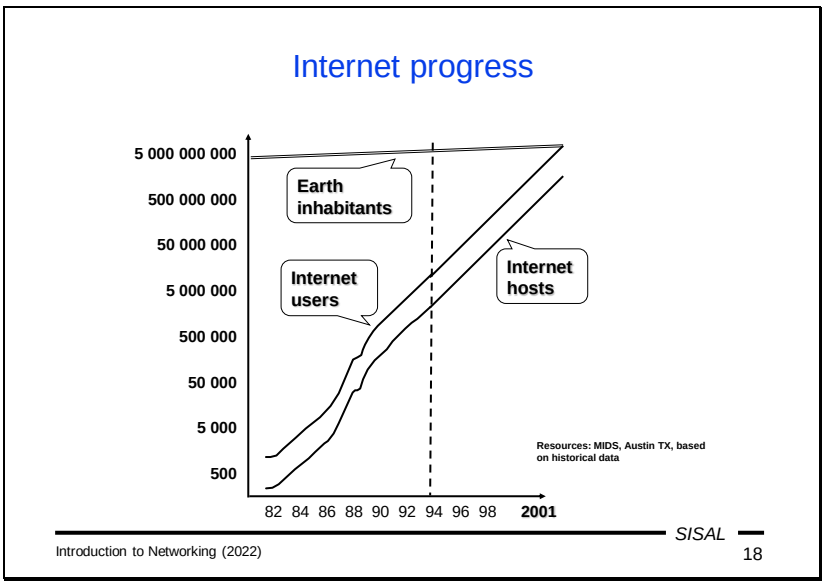
In the early 60s the concept of packet switching originated, supported by the US Department of Defense. It created a specialized agency Advanced Research Project Agency (ARPA) where several academic institutions tried to implement this idea and create a network based on a structure of point-to-point connections over leased telephone lines. This network was created in 1969 and called the ARPANET.

Originally it interconnected only separate computers and in 1977, the first network was connected to the ARPANET backbone. In fact, this moment actually gave another meaning to the term “Internet” which appeared in 1974 – an interconnection of networks.

Currently, the leading networking technology is TCP/IP; it replaced the previous one, Network Control Program (NCP), in the ARPANET in 1983.

The last key development milestone came in the mid-80s when developers of the Berkeley System Distribution (BSD) clone of the UNIX operating system understood the importance of TCP/IP and added support for it directly to the core of their UNIX. At that time, all users of the system were given the means to connect to the Internet.

Slide 18

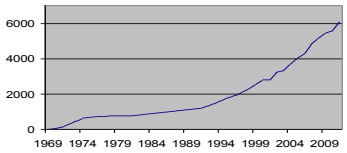


Just as an interesting fact that proves how dramatically this branch was developing, we can look at a chart published in 1993 by the Internet Society. They used an extrapolation of the network development in the preceding four years and the predicted result was that in 2001, the number of people using the Internet (people, not user accounts) would exceed the population of the Earth...

Slide 19

Request for Comments (RFC)

- Means of Internet „standardization"
- RFC 1 published Apr 7,1969



- Freely accessible (`http://www.ietf.org/rfc.html`)
- Various nature: standards, information, best-practice
- Drafts are sent to and judged by IAB ⇒ IETF,IRTF ⇒ WG
- Document texts are fixed, upgrades obtain new number (SMTP: *772*, *780*, *788*, 821, 2821, 5321)
- Current status can be found in the index file
- Recommendations are widely violated

Currently, Requests for Comments represent a means of standardization on the Internet, although their name does not suggest this.

Originally they were used for presenting ideas and discussing them together with publishing various data. That's the reason for the name. As an example of what was presented by them in the prehistory of the Internet, we can mention the fact that e.g. changes of computer addresses were published via RFCs. This practice of relatively free contents ended in the 70s and as you can see in the chart, the increase in the number of RFCs decreased rapidly.

Nowadays, their publishing is controlled by the Internet Advisory Board through two commissions, Internet Research Task Force and Internet Engineering Task Force, and several working groups. When an author submits a draft of a new protocol, the relevant working group judges it and, if it considers it useful, the document receives a number and is **published** "for commenting" (as the original meaning of "RFC" says). The important fact is that the text of the document **never changes** (except for typos and errata). You don't need to search the internet for the newest edition of an RFC. All servers have the same version. When there are a sufficient number of changes, the document is reissued with **a new number**. RFC number changes can be tracked in an index file called `rfc-index.txt`. The old RFC can be marked either "Obsoleted", or just "Updated", in which case both RFCs are valid.

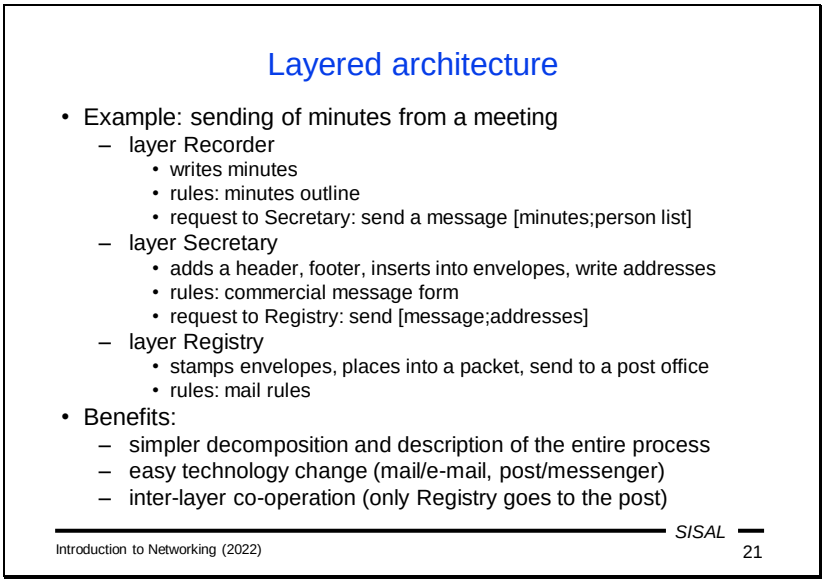
Adherence to the protocol rules defined in the RFC is very important for all implementers. However, some of the rules are a bit restrictive and many clients and servers violate them. The mutual understanding of communicating parties then depends on the extent of the violation. The general recommendation for

implementers is: be as tolerant as possible on the receiving side and be as conservative as possible on the sending side!

Summary 1

- What are the advantages and disadvantages of packet switching?
- How have network protocols been affected by the fact that the idea of networking was initiated by the army to increase the security of communication?
- What is the purpose of the network scalability requirement?
- How do the demands of e-mail and IP telephony differ regarding the transmission parameters of the network?
- What is the definition of a LAN?
- What is the essence of a VPN?

Slide 21



We will use a layered approach to describe network communication. We will try to show the advantages of this approach using a real-life example.

A company holds regular meetings. The minutes of each meeting are recorded and sent to all participants.

Using a non-layered approach, we must describe the process to a new employee as an entire complex task, starting from recording, formatting, and packing into envelopes up to carrying them to a post office.

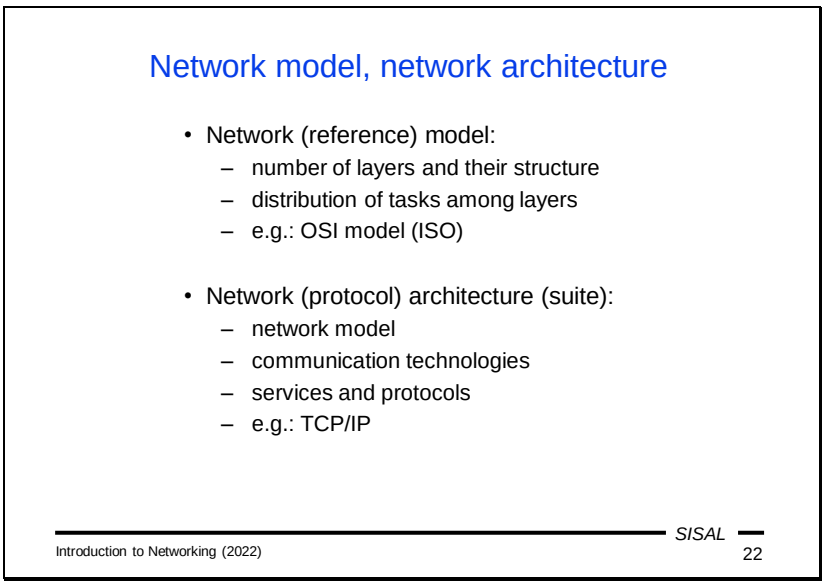
Another principle is to split the task to three layers:
The **Recorder** layer is represented by a person writing the minutes. He or she must follow the company's rules about how the minutes must look like – this can be considered to be a *protocol*. This person's work finishes at the moment when the minutes are ready and are handed over to another layer, the Secretary layer.
The **Secretary** layer does not care what data it received and where it comes from. They are handed over to this layer with instructions to prepare their sending to a list of meeting participants – this can be considered as an inter-layer interface [*data=minutes*; *control data=person list*]. A person at this layer takes the minutes, formats them into a business message (a header with a properly placed address, date, and footer), inserts messages into envelopes and writes addresses on them. The envelopes are then passed to the Registry layer. In principle, the secretary does not need to know which kind of delivery will be used.
The **Registry** layer takes over the envelopes and chooses a delivery method according to the company's current strategy. If the company uses classic mail, the envelopes will be stamped, bundled into a packet and carried to a post office.

What are the benefits of this workflow?

- The whole process is decomposed into simple tasks that can be easily described to new employees.
- Simple tasks can be easily replaced by another solution – if the company decides to stop using mail and to start using a parcel service, no other layer but the Registry needs to know about this change.
- Resources are saved thanks to layer co-operation – e.g. it is not necessary that all secretaries go to the post office.

We get exactly the same benefits – ease of description, layer co-operation and substitution possibility – with layered approach for computer networks.

Slide 22



When talking about a network communication structure, we distinguish two terms:

A *network model* describes the number of layers, their structure and describes the tasks of the layers. An example of such a model is the Open Systems Interconnection (**OSI**) model made by the International Organization for Standardization (ISO).

If we take a model and add concrete services, technologies, inter-node protocols, inter-layer interfaces etc. we get *network architecture*. An example of it is the **TCP/IP** protocol suite.

Slide 23

Open Systems Interconnection

- OSI: Basic Reference Model + set of protocols
- Model: used for documentation and teaching
- Protocols: built from the top, megalomaniac, inconvenient

Layer		Function
7	Application	End application communication
6	Presentation	Data conversions for applications
5	Session	End nodes dialog control
4	Transport	End-to-end data blocks transmission
3	Network	Routing across networks
2	Data link	Transmission of data frames between two nodes
1	Physical	Transmission of bit streams over a medium

The Open Systems Interconnection model was introduced in 1978 by the International Organization for Standardization. It is a network architecture consisting of the Basic Reference Model and a set of protocols. While the model has been designed very well and is used for network suites documentation and teaching until now, the architecture as a whole has been significantly less successful. It was designed from the top, i.e. the decisions were made as in an ideal world, so they were too complex to achieve simple goals and were relatively inflexible, user-unfriendly…

Layer 1, the **physical layer**, is responsible for transmitting and receiving raw bit data between a node and a physical medium. It has no idea about the logic of the transmitted data and deals only with the physical aspects of the transmission.

Layer 2, the **data link layer**, provides data transfer between two directly connected nodes. Its task is to identify the source and destination of the transmission, the range of transmitted data and to detect and possibly correct errors that may occur during the physical transmission.

Layer 3, the **network layer**, ensures the transmission of data blocks with variable but limited length (packets) between two nodes, possibly connected in different networks. If the source and destination are in the same network, the network layer only passes the request to the link layer, but if they are in different networks, the network layer is responsible for finding a path and routing the data through intermediate nodes.

Layer 4, the **transport layer**, is responsible for transmitting and receiving data blocks with unlimited length between source and target applications. The layer still does not understand the data, but it is able to recognize its affiliation to a particular

communication channel between two applications. Some protocols at this layer provide *segmentation* of blocks that are too large and their reassembly at the target station. Some protocols provide *reliable* transmission, i.e. they guarantee the successful transmission of the entire block, or else trigger an error condition.

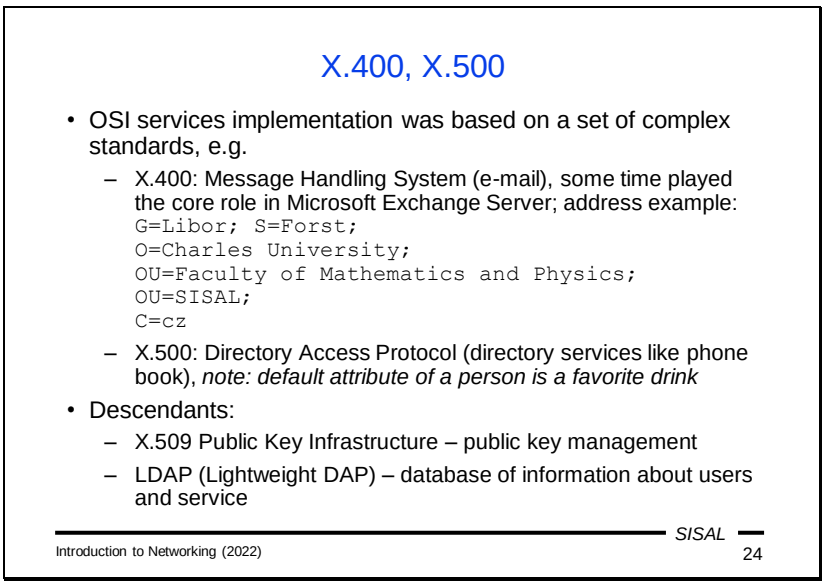
Layer 5, the **session layer**, controls dialogs between applications. In many network models, its work is in fact covered by adjacent layers.

Layer 6, the **presentation layer**, provides means to hide differences between data semantics implementations on different platforms, e.g. integer endianness (an integer value is divided into bytes in different ways on different computers), or text line endings (LF=0x0a on UNIX systems vs. CR+LF=0x0D0A on Windows systems).

Layer 7, the **application layer**, is the closest layer to the end user; applications mediate interactions between the user and the application protocol(s) that implement communications needed to perform tasks requested by the user.

Understanding the OSI model layers is important because when dealing with various issues, we must carefully consider which solution can be used at which layer.

Slide 24



OSI standards were marked by the X.*number* schema. As examples, we can look at two of the most important:

- **X.400 (Message Handling System)** was the OSI implementation of electronic mail. For some time, it formed the core of Microsoft Exchange Server. Interestingly, X.400 used a fairly complex address structure. This complexity was inspired by classic postal addresses – they also have numerous components (name, street, number, zip code, city, country) which does not surprise anybody. And such addresses have one major advantage – they are unique! In the “computer age”, however, nobody wants to write too much and so ambiguous but much shorter internet addresses have won.

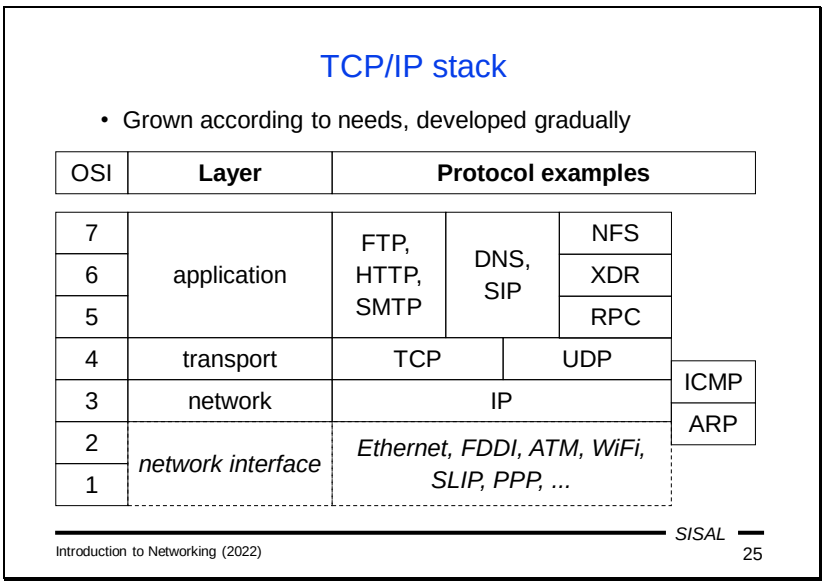
- **X.500 (Directory Access Protocol)** was a directory service, one of the first implementations of something like a phone book, or Yellow Pages. It uses the same person identification structure as X.400. As an illustration of how complex this service can be, the authors decided to include a favorite drink as one of the default attributes of a person!

Although the OSI services have not been widely implemented, some of their ideas have been beneficial and have remained in some modern protocols and data structures:

- Asymmetric cryptography needs an infrastructure for public key management. Each public key must have a completely unique identifier. For that purpose, the structure of addresses designed for OSI e-mail and directory services is ideal.

- The X.500 DAP was too complex to be implemented in personal workstations, although its principle was good. The solution was to create a simpler version while preserving key features. This protocol is called LDAP and currently, it is a common solution for user and services information retrieval and management.

Slide 25



Unlike OSI, the TCP/IP stack was built step by step using smaller building blocks – protocols that solve one compact problem. As we mentioned, we will use the OSI model as a reference model to describe tasks solved by particular layers and protocols of TCP/IP.

OSI layers 1 and 2 are not subject to the TCP/IP stack, they are simply called network interface, and the protocols used here correspond to the type of media used for transmission.

The lowest TCP/IP layer that corresponds to OSI layer 3 is called the **network layer**, just like in OSI, and its function is the same as well. The protocol used at this layer is called the *Internet Protocol* (IP). In fact, there are currently two versions of this protocol (IPv4 and IPv6), but at the moment the differences between them are not important to us.

The next TCP/IP layer, the **transport layer**, also has the same name and function as in the OSI model. Multiple protocols are used at this layer, the most frequent being the *Transmission Control Protocol* (TCP) and the *User Datagram Protocol* (UDP). We will explain the differences between them in the next slide.

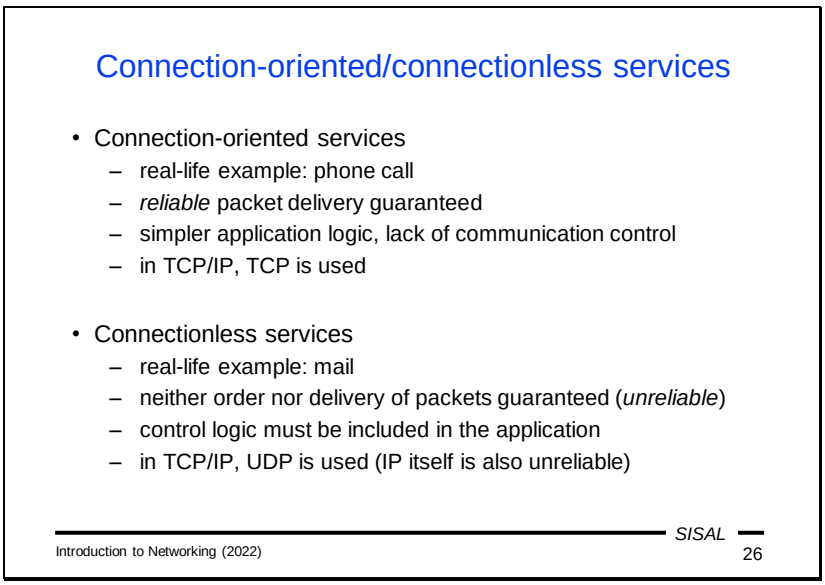
The functions of the last three OSI layers are combined in TCP/IP into a single layer, the **application layer**. For most protocols it was easier to define dialog rules and value semantics directly in the application protocol without using special intermediate layers. There are a few exceptions such as the Network File System which uses special protocols XDR (eXternal Data Representation) and RPC (Remote Procedure Call) at layers 5 and 6. Authors of each application protocol can choose which transport layer protocol is the best for the application, so some protocols (e.g. FTP,

HTTP, SMTP) use TCP, some (e.g. NFS) use UDP and some (e.g. DNS, SIP) can use either.

There are also several management protocols that stand outside the strict hierarchy of layers, namely the Address Resolution Protocol and Internet Control Message Protocol, which we will talk about later.

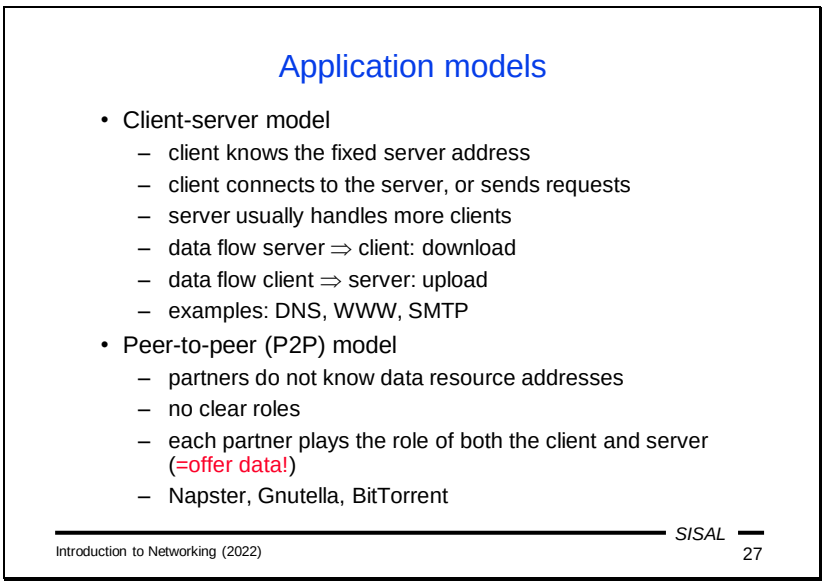
Note that the separate terms TCP and IP refer to specific protocols at the 4th and 3rd layer, while the “TCP/IP” refers to an entire protocol stack (and is therefore somewhat unfair to the neglected UDP protocol).

Slide 26



The **Transmission Control Protocol** (TCP) is designed for connection-oriented services. An example of such a service from real life is a telephone call. When you make a call, the telephone operator establishes a connection and you do not need to worry about whether your sentences will be transferred completely and in order. So the application (you) can be very simple and all the responsibility lies in the lower layer. The same principle is used by TCP. It provides reliable packet delivery, so the application only calls a function “send data block to destination” and waits. TCP handles *segmentation* of data to smaller blocks, sending them in packets, acknowledging their successful delivery, retransmitting them in case of failure and finally it returns control to the application, resulting in a success or failure. The TCP software is therefore very complex, but it is easy for applications to use. The disadvantage of this approach is that the application loses control over the process, it cannot react flexibly to the current state of the network; it just calls the function and waits.

The **User Datagram Protocol** (UDP) is designed for connectionless services. The classic mail service can serve as a real-life example. If you send three messages in three days, you never know whether they will all be delivered and in order. Therefore, the application (you) must pay special attention to confirming the delivery and resending the lost data, or it must be resistant to data loss. This is also the case of UDP. It is very simple, just transports packets and the responsibility for reliable delivery lies in the application. The advantage is that the application can react immediately to the current situation, e.g. when a delivery problem occurs, the application can try another data source, or it can ask the user whether to stop or continue.



As we already mentioned, internetworking has enabled new application models where two parts of an application co-operate remotely.

The **client-server** model. The client knows one or more addresses where a required service can be found, it tries to contact them and, if successful, to exchange data with a server to meet the user's needs. The roles of client and server are well defined (in terms of communication, not necessarily in terms of data flow – a client can upload data to a server, i.e. be a source of data).

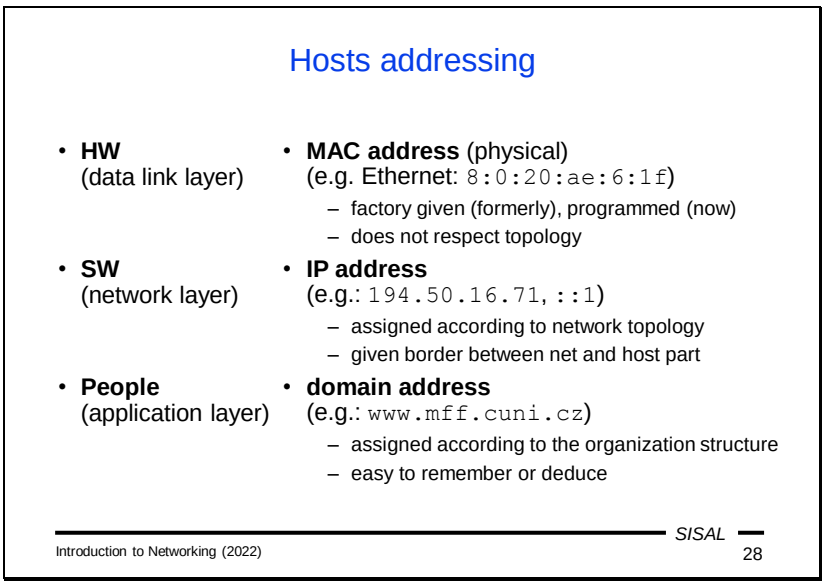
The important fact is that the meaning of the terms client and server is related to a concrete application, not to a computer. It is quite common for a computer to be a server for one type of communication while in another one it plays the role of a client. A typical server implementation is to receive incoming requests from several clients in parallel.

However, the client-server model is not the only possible application model. There is also the so-called **peer-to-peer** model. The key feature of this model is that a user application does not need to know the address of the data source (although it often has some starting addresses to speed up the process). The application is able to find other computers on which the application is running and ready to exchange data. At the same time, the application running on our computer responds to all incoming requests from other instances of the application and sends them requested data. So there are no clear roles, such as client and server.

P2P applications have a bad reputation because they are often used to distribute data that the user does not have the right to distribute. The problem is not in the applications themselves, but in the data. If your application downloads data illegally,

you have not committed any wrongdoing, but once it starts offering data to others, the law has been violated. The only option is to deny the application to distribute the data, if such a denial is possible.

Slide 28



When a client wants to contact a server, it needs an address. Depending on the layer at which the communication takes place, different addresses are used.

At the data link layer, **Media Access Control** (MAC) addresses are used. Since they are quite closely connected to hardware, they are sometimes called “physical” address, even though they have nothing to do with the OSI physical layer! Examples of MAC addresses are **Ethernet** addresses. They are six bytes long and consist of the manufacturer’s prefix (3 bytes) and network interface card (NIC) number (3 bytes). The address was originally burned into the NIC and so it was possible to effectively restrict access to network resources by the sender’s MAC address. However, current NICs have the MAC address stored in memory and so it can be easily faked. Due to their nature, these addresses cannot be used for inter-network communication since they do not respect any particular network topology.

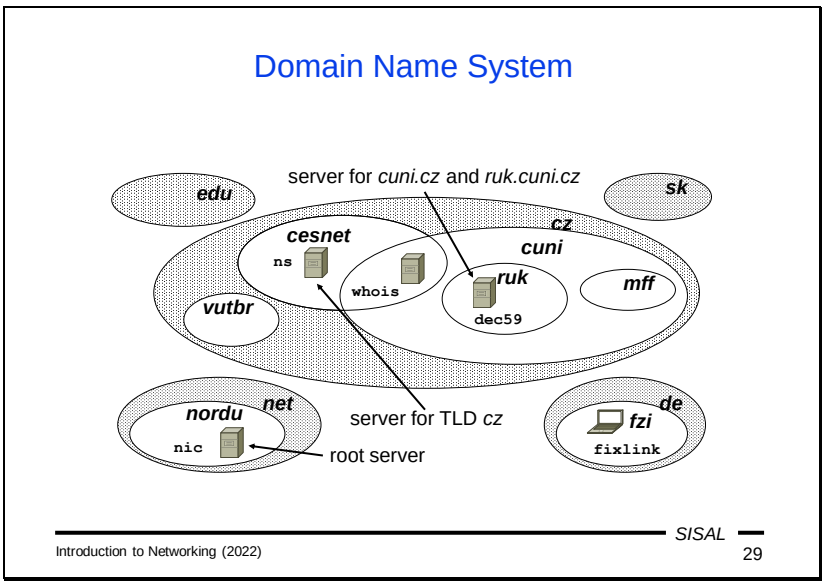
Therefore, we must use an address type that depends on the network topology. In TCP/IP, **IP addresses** are used. We have already mentioned two versions of the protocol, so we have two address formats. Each computer is assigned an address according to where the computer is connected to the network and the address has two parts – a network address (used for routing across the network) and a computer (host) address (used locally). The boundary between these two parts varies according to the concrete network.

IP addresses are sufficient for the proper functioning of network communication, but they are somewhat user-unfriendly. Therefore, in common practice we use yet another address format, **domain addresses**. Instead of numbers, these addresses use a mnemonic format, *domain names*, assigned according to the organizational structure. Therefore, it is much easier to remember or to deduce them. The hierarchy

is right-to-left: the rightmost name is a so-called *top level domain* (TLD), e.g. a country code, while the leftmost name is usually a host or service name.

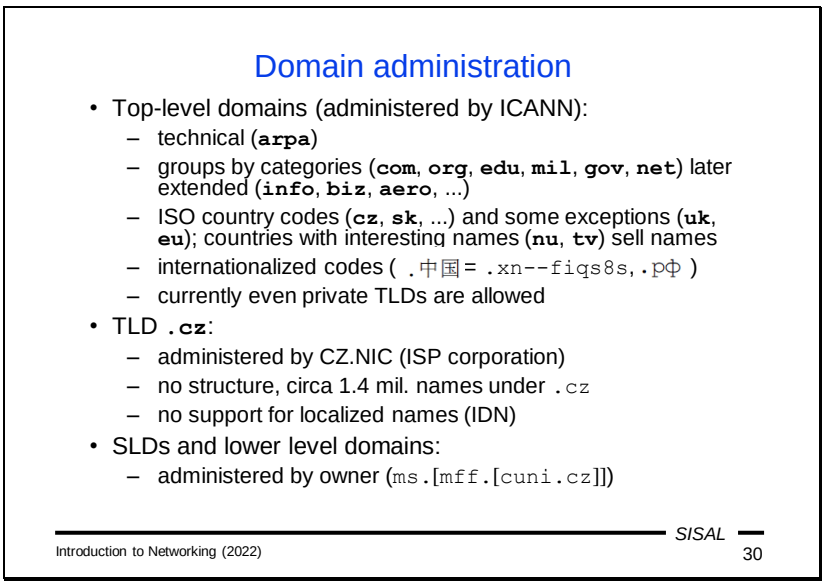
The Domain Name System (DNS) is used to convert between domain and IP addresses. The Address Resolution Protocol (ARP) is used to convert between network and MAC addresses.

Slide 29



The Domain Name System is a hierarchy of *zones* that contain information about child computers and zones, and corresponding *name servers* that provide that information to clients using queries and responses in the DNS protocol.

Every computer should have a domain name that matches the address(es) it currently uses; many computers have multiple names, especially if they are running multiple services for multiple domains.

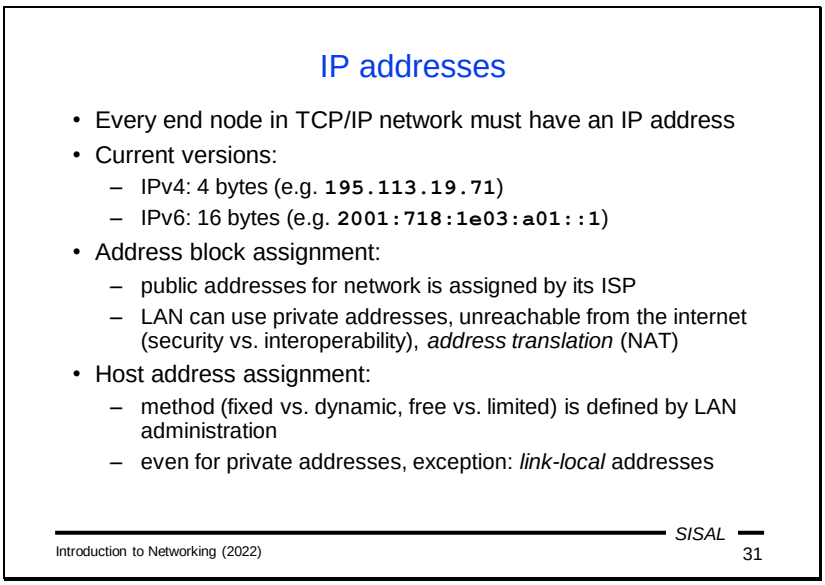


The top-level domains (TLDs) are administered by ICANN (the Internet Corporation for Assigned Names and Numbers). The Domain Name System structure was defined in RFC 920 and the conditions for TLDs were originally relatively strict. There was an **arpa** TLD, marked as "temporary" for old ARPA hosts, but it has been converted to a domain used for technical purposes. The second group of domains were country domains named after the two-letter ISO country codes (with some exceptions such as **uk** or **eu**). Then there were five TLDs collecting institutions by category (**com** for commercial companies, **org** for non-commercial organizations, **edu** for education and research institutions, **mil** for military, **gov** for government and later also **net** for networking organizations). Over time, name conflicts occurred, especially in the **com** domain. Therefore, some new TLDs were added later (**info**, **biz**, **aero**, ...) but of course that could not solve the problem. Under great pressure, ICANN eventually completely opened up the name space, and nowadays even a private entity can request an individual TLD.

The TLD **cz** is administered by a corporation of ISPs called CZ.NIC, It has no underlying category structure like in some other countries (U.K., Austria) and there are over a million names defined as second-level domains (SLD). Although it is currently technically possible to allow localized names, CZ.NIC has not yet allowed it, fortunately.

Second-level domains are administered by their owners who can decide on their structure. At our university, we have decided to maintain the hierarchy and only domains for faculties and university-wide facilities are allowed under the SLD **cuni.cz**. Similarly, our faculty's domain is administered at the faculty and has a

structure according to departments and buildings (so the **`ms`** domain name is an abbreviation of “Malá Strana”).



In order to communicate on a TCP/IP network, each end node must have an IP address. As already mentioned, we currently use two versions of the protocol/address. Version 4 which is still dominant in the Czech Republic, has addresses that are 4 bytes long and the common form of their notation is the so-called *dotted decimal* form. The newer version 6 expanded the address size to 16 bytes and the commonly used format for these addresses is hexadecimal notation in double-byte blocks separated by colons. Due to the large number of zeros in these addresses, the longest series of zero blocks in any address can be replaced by "::" notation.

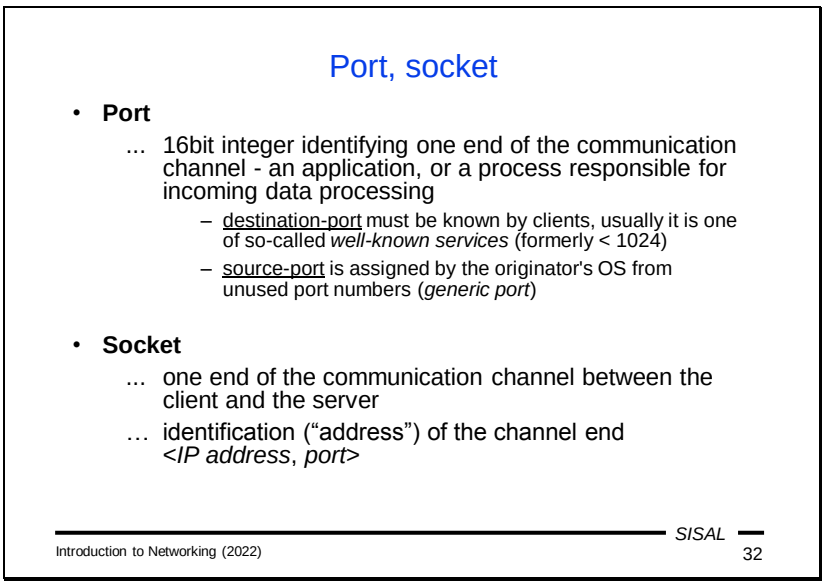
Every address has a prefix defining a network to which the address belongs. The assignment of these *network addresses*, or address blocks, must meet certain conditions:

- Blocks of addresses that should be able to communicate directly with the rest of the world, so-called **public** addresses, are assigned to the network administrator by an ISP (or other authority) that connects the network to the Internet.
- For internal use, a network administrator may choose to use one or more **private** address blocks. They are inaccessible from the Internet (which increases security but complicates interoperability) and in order to enable hosts to access resources on the Internet, a router at the perimeter of the local network must use Network Address Translation (NAT).

A method of assigning addresses to hosts must be chosen by the network administrator and all hosts must follow these rules. The assignment can be static (each node has a predefined IP address) or dynamic (an address is assigned on demand), free (any host can join) or restricted (hosts must authenticate to connect). These rules apply even for **private addresses**. The only exception is so-called *link-*

local addresses – they are freely chosen by the host software (with a risk of duplicate address collisions) and they allow communication only within a directly connected network (and only if some other hosts also use a link-local address).

Slide 32



If we know the address of the host we want to communicate with, we still can't contact a specific service because multiple servers may be running there. We need to somehow identify a service on the target machine. And for the same reason, we must also identify "our end" of the communication channel because when a response comes, it must be clear to which client it should be delivered – for example we must insert an incoming image into the correct web browser window…

A **socket** is one end of a communication channel between a client and a server. A socket is identified by an IP address and a 16-bit integer number called a **port** which is used to distinguish between different services or applications.

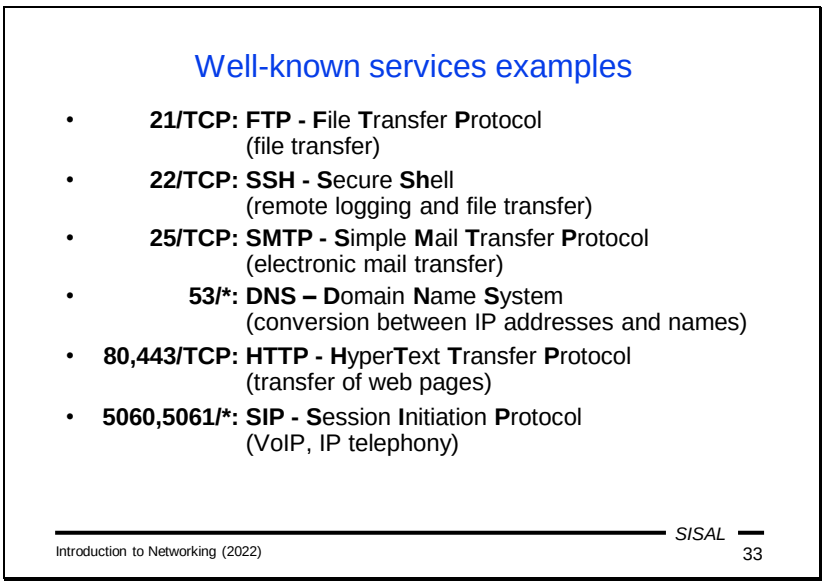
However, clients do not know the situation on the server machine, so they cannot always know which number a particular service has on a server, i.e. which number should be used for the *destination port number*. Therefore, there are so-called **well-known services** and their port numbers. For instance all web servers use port number 80 by default.

Well-known services, however, are not a solution for the *source port number* because we need to identify multiple clients of the same service. The usual solution is that when client software prepares a socket for connecting to a server, it asks the local operating system for an unused "**generic**" port number (out of the well-known services range). The client uses is this number for the entire communication with the server and after the connection is closed, the port is released and is available for other applications.

From a mathematical point of view, the unique identification of a TCP/IP connection is an ordered quintuple <source IP address, source port, destination IP address, destination port, transport layer protocol type>. This fact has two consequences:

- Two different application channels between two computers must differ at least in the source port.
- The same port numbers can be used for two different communication channels as long as they do not use the same transport layer protocol.

Slide 33

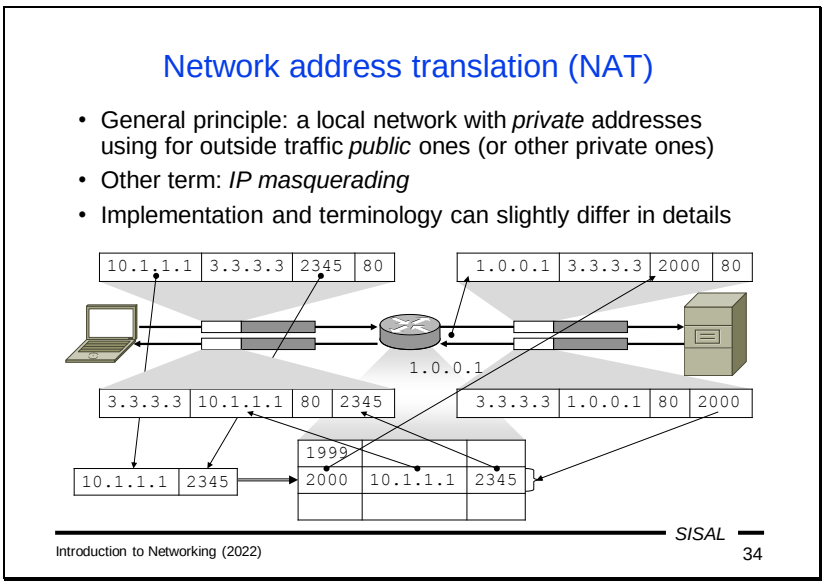


Examples of port numbers on this slide are basic networking literacy, except perhaps for the last bullet which only documents that some well-known services nowadays do not fall in the range from 1 to 1023...

Well-known port numbers are usually reserved for both TCP and UDP even if a particular application protocol uses only one of these transport protocols.

HTTP uses port 80 for unencrypted and port 443 for encrypted connections. SIP uses different ports for different types of messages.

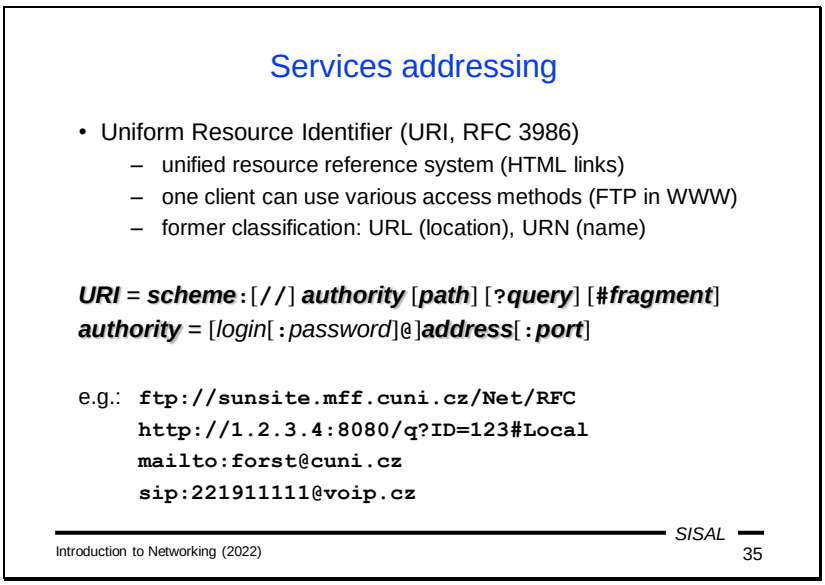
Slide 34



The concept of ports plays a very important role in NAT. Hosts with a private address cannot communicate directly with the world outside the local network because an external host has no idea how to route packets when sending a response.

A common principle of NAT is that when a host establishes a connection to a server, the router at the perimeter of the private network modifies the content of the packet so that the external host can respond. The router stores the socket address (the IP address and port) of the incoming request and replaces it in the packet by its own external address and a port, currently not used at the router. So, when sending a response, the external host uses this translated socket address for the delivery. When the response arrives at the router, it looks for the appropriate original socket address (according to the destination port used by the external host), changes the destination information back to the original data (the IP address and port from the request) and delivers the response to the client.

Slide 35



To get to a particular data source, an application needs to have quite complex information about the required service. The situation before the invention of the WWW was quite broken; there were different applications for different services. Web browsers were the first applications that use various protocols (FTP, Gopher, WWW) and HTML links must be able to address different services. Therefore, we need a common general way to combine a retrieval method type, server address, user credentials, resource location etc. into one compact form. The original idea was to define an addressing method (URI, Uniform Resource Identifier), which can either refer to a specific resource location (URL, Uniform Resource Locator), or just a service name without an explicit location (URN, Uniform Resource Name). However, URNs have never been successfully implemented, so the terms URL and URI are more or less interchangeable.

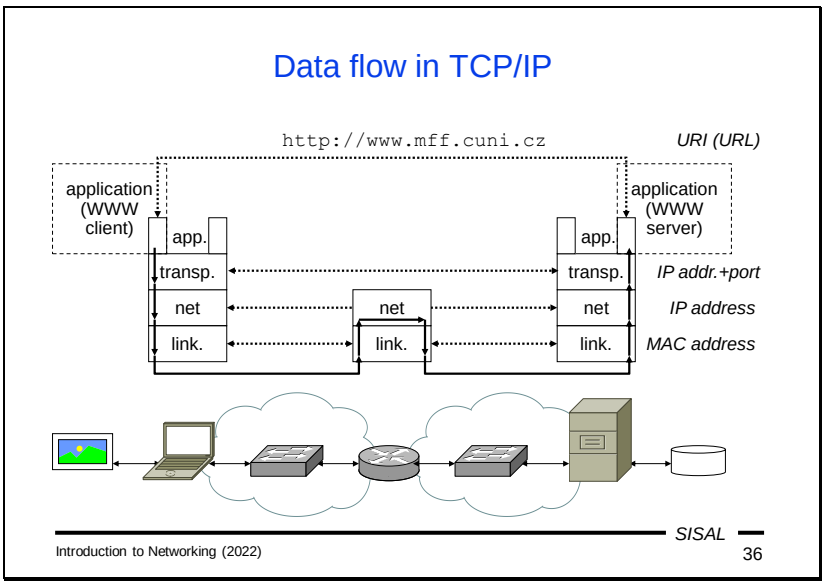
The first part of a URI is a **scheme** (followed by a colon), often incorrectly called a "protocol" because many schemes are synonyms of protocols that should be used to approach a resource. Every URI must include a scheme, unless it is clear from the context – e.g. "http" is the default scheme of a link on a HTML page and we do not need to write it there, in a different context a missing schema can mean something else.

The next part of a URI is an **authority**, in some URI types prefixed by a pair of slashes. An authority specifies a server (or domain) together with information about the user (if needed). If a resource uses a non-standard port, it must be specified here; otherwise the well-known port will be used. For HTML links, if the authority in the link is the same as in the referring page, the authority can be omitted.

Many URI schemes contain a **path** to the requested resource on the server. The path format is similar to paths in filesystems (just using slashes to separate directory names) and the path often actually corresponds to the page's actual location on the server's filesystem. For HTML links, if the authority is omitted, a relative path anchored in the directory of the referring page can be used, similarly to how a relative path is used in a filesystem.

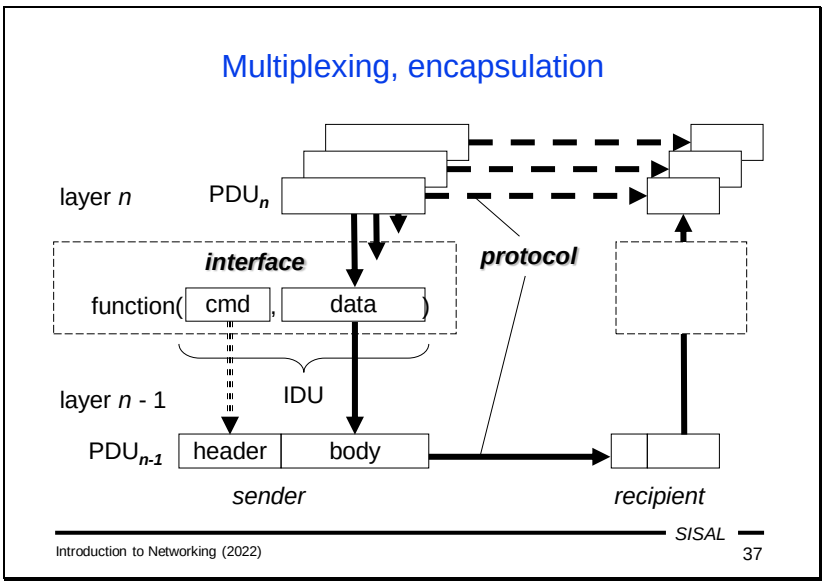
Some URI schemes may contain additional URI parameters used to more precisely specify a request; e.g. an HTTP URI may contain a **query** (with e.g. data from an HTML form) or a **fragment** (the identifier of the location of an exact point inside the page that the browser should focus on).

Slide 36



- At the application layer, a client usually addresses a server using a URL. The application layer passes data together with the destination address and port to the transport layer.
- At the transport layer, the logical communication channel is addressed by the socket addresses (i.e. IP address + port number) of the client and server. The transport layer hands over the data along with the target IP address to the network layer.
- The logical communication channel at the network layer is addressed by the client and server IP addresses. Before sending data, the client must decide whether the destination address belongs to the same network. According to this decision, a *next-hop* node is chosen and data plus the next-hop address are passed to the data link layer.
    - If the server is in the same network as the client, the next-hop node is the server.
    - Otherwise, a router capable of forwarding packets to the destination network must be found and this router is the next-hop node.
- The communication channel at the data link layer connects a node with a next-hop node (i.e. not necessarily with the target host), so the destination address at the data link layer is the next-hop MAC address.
- After delivering to the end of the data link layer channel (target host or router) through the physical layer, the behavior differs according to the nature of the node:
    - If the node is the server, data is passed to higher layers for further processing.
    - If the next hop is a router, data is passed only to the network layer and this layer again decides which next hop has to be used further. The data with the next-hop address is then passed back to the data link layer for delivery.

Slide 37



An important method used for the efficient use of a network is called **multiplexing**. This method means that several communication channels at one layer use the same communication channel at the underlying layer.

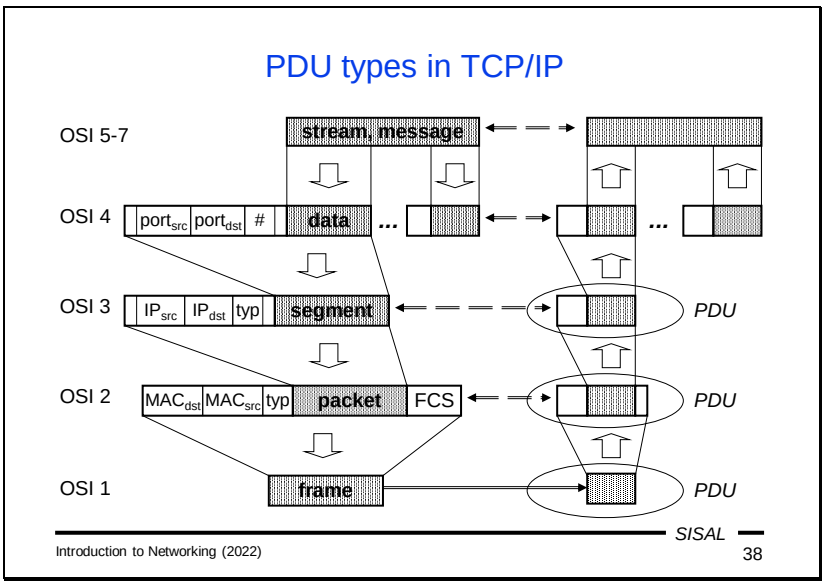
For the correct behavior, both sides of each communication channel at a particular layer must follow a set of rules, a so-called **protocol**. Multiple protocols can be used simultaneously at one layer.

Let's imagine that we have a protocol at layer *n* which defines its **Protocol Data Unit** (PDU) format of this layer. The software at layer *n* has finished its work and has decided to call layer *n*-1 to perform the transmission. It takes a data unit (PDU_n) and adds control information (such as destination, size, delivery options etc.) and makes a call to the next layer service. Usually, such a call actually takes the form of calling a library function with the data and command as parameters. Upon return, the caller receives information about the success or failure of the work of layer *n*-1. This data exchange between layers is called an **interface** and the exchange format is the **Interface Data Unit** (IDU).

Layer *n*-1 takes the control information, processes it and then prepares a PDU according to the particular protocol of that layer. This PDU usually contains a *body* containing PDU_n plus a *header* with control information. We call this method the **encapsulation** of the layer *n* data into the data at layer *n*-1. This is the most common case of encapsulation, but in general it is possible to encapsulate a PDU of any layer into any layer, even including the same layer or higher layers. For instance if we need to send an IPv6 packet over an IPv4 network, we must encapsulate the PDU_3 into another PDU_3.

The header must also contain an identifier of the layer *n* because **decapsulation** and **demultiplexing** must be performed on the recipient side. Layer *n*-1 checks the control information and passes the data (body) to appropriate software at layer *n*.

Slide 38



Let's look at how multiplexing and encapsulation work in the TCP/IP protocol suite.

The application layer sends either separate **messages** (in connectionless mode), or a **stream** of data (in connection-oriented mode). It passes the data together with the target socket address to the appropriate protocol at the transport layer.

If TCP is used, it takes the data from the stream and breaks (**segments**) it into smaller blocks and prepends a header containing, among other information, the source and destination port numbers and an "offset" into the stream so that the data can be sorted and reassembled upon delivery. UDP has much simpler work to do; it just prepends a header with port numbers to each message. The PDU_4 is then passed to the network layer along with the destination IP address.

The network layer takes a segment and prepends a header with the source and destination IP addresses and the transport layer protocol number, creating a **packet**. It then takes the destination address and checks if the packet can be delivered directly to the destination host or if a next-hop router must be used for forwarding. The packet, along with the data link layer destination node address is passed to the data link layer.

Warning: be careful when talking about "packets", be sure to clearly indicate whether you mean a well-defined PDU_3, or a loose term of "packet switching" networking mode.

The data link layer takes a packet and prepends a header with the destination and source MAC addresses and the network layer protocol number. Since this is the last "software" layer before the data is passed to the "hardware" (the physical layer), we

need some way to check whether the data has been transferred correctly. Therefore, the data link layer PDU (called a **frame**) contains also a suffix, the so-called Frame Check Sequence (FCS), the value of which is calculated from the contents of the rest of the frame.

The last step is to pass the frame to the physical layer to be transferred.

Upon the delivery to the destination node, the physical layer decodes the data and passes it to the data link layer.

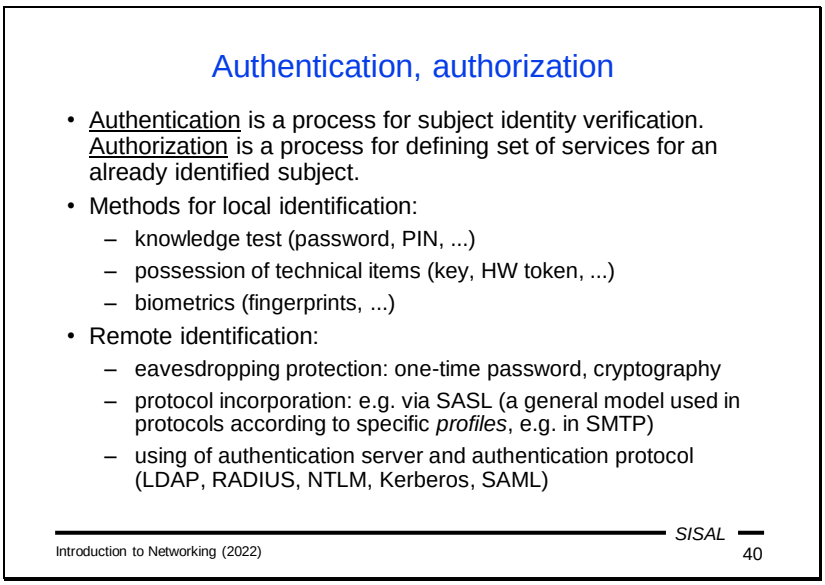
The data link layer recalculates the value of the FCS and checks whether it is equal to the value stored in the frame. It also checks the destination MAC address to see whether the data belongs to this node. The decapsulated packet is then passed to the network layer software chosen according to the network layer protocol code stored in the header.

The network layer checks the destination IP address and passes the decapsulated segment to the appropriate transport layer software.

The work of the transport layer depends on the protocol used. In the UDP case, the message is simply delivered to the application. In the TCP case, the segment is saved and the entire data block will be delivered to the application after all segments have been received.

Summary 2

- Describe the differences between TCP, UDP and IP.
- Which addresses are used at the application, transport, network, data link and physical layer?
- What is the difference between the client-server and peer-to-peer application mode?
- How are domain names administered?
- How are IP addresses assigned?
- What is the meaning and principle of NAT?
- What is multiplexing and encapsulation?



Before we talk about security, we should clarify two terms that are often confused.
- Authentication is a process used by subjects (usually users) to prove their identity, i.e. it is an answer to the question “who am I”.
- Authorization is a process by which an authority (usually a server) assigns permissions to a subject that has already succeeded in the authentication process, i.e. it is an answer to the question “what can I do”.

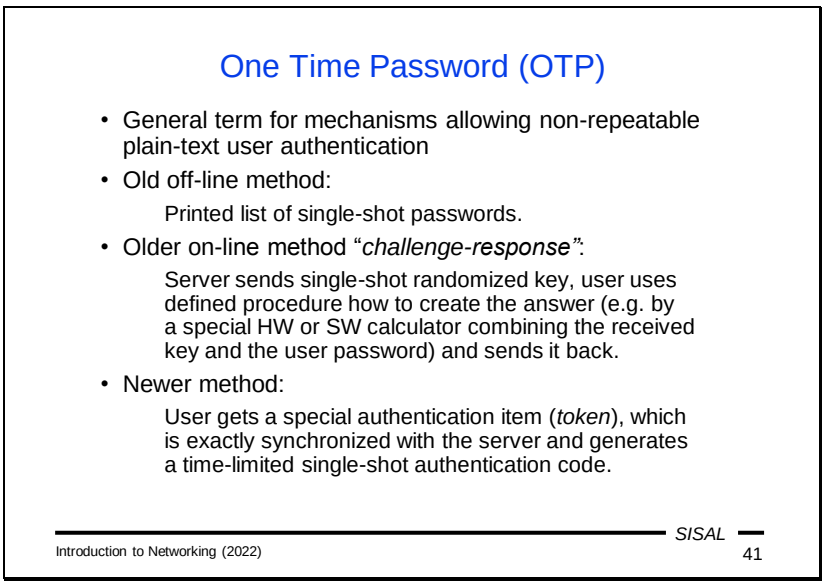
In general, there are three basic methods of proving identity:
- “What do I know”. The easiest way is to test a piece of knowledge, a password (or in a simpler version only a number, a PIN), an answer to one of a set of predefined questions etc. The big disadvantage of these methods is that the knowledge can be revealed. On the other hand, there are also some advantages, especially when authentication is used to access a less important resource: easy implementation, simple sharing of the knowledge and easy use of remote resources.
- “What do I have”. It is a bit safer to prove identity by holding a technical item. It can be either a physical object (a hardware token) or a piece of data (an electronic key). It is a bit more complicated to steal or copy the item.
- The safest proof of identity is biometrics – a fingerprint, a retinal scan, voice recognition etc. – however, if you need to prove your identity for a remote source, these methods have a difficulty in connecting the device that retrieves the biometric data to the server that verifies it.

A combination of these methods is also used.

If we want to authenticate for a remote service (a server), there are several other factors to keep in mind.
If we use the knowledge test, there is a risk of eavesdropping if we are using an insecure channel. The oldest way to protect against reusing revealed information is to

use modified information that is valid for one access only (a one-time password). Another way is to eliminate the problem of an insecure channel and use cryptography to create a secure channel.
Another problem is that most protocols have no means for secure authentication and they need to be extended to be able to transmit the necessary information. There is a framework called Simple Authentication and Security Layer that allows using many authentication methods and incorporates authentication into most of the important protocols. For each protocol, there is a special *profile* defining how to use it in the protocol.
It is also a common requirement to centralize the authentication for a set of resources, to create a special central authentication server (an *identity provider*) that uses a special protocol to communicate either with a client (to get authentication data from him/her and then to provide him/her an authentication result that can be used to prove identity to a server), or with a server (a *service provider*) who communicates with the client by itself.

Slide 41

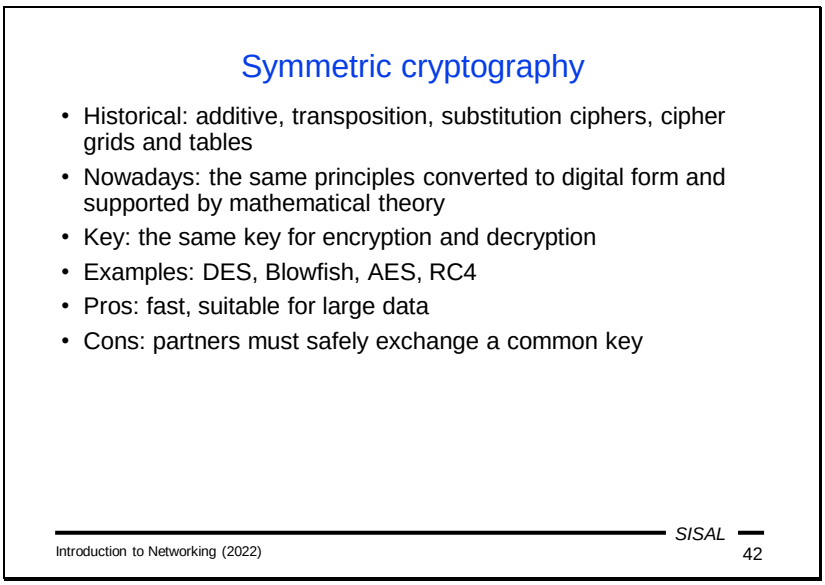


In the past, a very simple method for plain-text authentication was used. A server generated a list of passwords, a user printed this list and used the printed passwords one by one for subsequent authentication attempts.

Later, the so-called *challenge-response* method was developed. A server sends a randomly generated but unique sequence of characters (a challenge) and a client must send an exact response. The response could be generated either by a special hardware calculator, or by a piece of software. The challenge-response principle is widely used in various communication schemes. A conceptually similar system is also used nowadays to validate various web application operations by sending a mail with a URL that need to be visited to complete the operation.

A newer method uses special hardware items (tokens) that are precisely synchronized with a server and generate a special code used for identification. The validity of the code is limited to a few seconds and a single usage so that it cannot be stolen and misused.

Slide 42



Cryptography plays an important role in contemporary communications. There are three types of cryptographic algorithms that are used in collaboration to achieve the goals of data encryption and origin verification.

The first group are **symmetric encryption** methods.

The history of these algorithms is really very long. Since ancient times people have needed to protect information transported to another place in the world. There are several methods of encoding by substituting or transposing characters. Another approach is to use a grid mask that shows only a part of the text written into a grid. This part of the text is written and by turning the grid mask repeatedly we can encode the entire text. A common feature is that our partner must have the same piece of information (the substitution or transposition key, the grid mask etc.) for decoding that we use for encoding. Of course, nowadays we need different methods of encryption based on complicated mathematical principles.

The basic advantage of current symmetric cryptography methods is that they are very fast and therefore suitable for encrypting large amounts of data. However, the catch is that we haven't solved our problem – the parties needed to securely transfer a piece of information (the message), now they can send it encrypted, but they still need to securely transfer another piece of information (the key). Of course, they can use another, safer method to transport the (small) key, but the fact is that we have only **reduced the size** of the problem.

Slide 43

Asymmetric cryptography

- Keys: a pair of (mutually nondeductible) keys for encryption and decryption
- Mathematical principle: one-way functions
  - multiplication vs. factoring
  - discrete logarithm $m = p^k \bmod q$
- Examples: RSA, DSA, ECDSA
- Pros: no need of shared secret, one key (public) is free for distribution, the other (private) one must be secured
- Cons: slow, suitable only for small data
- Crucial problem: the public key must be carefully verified

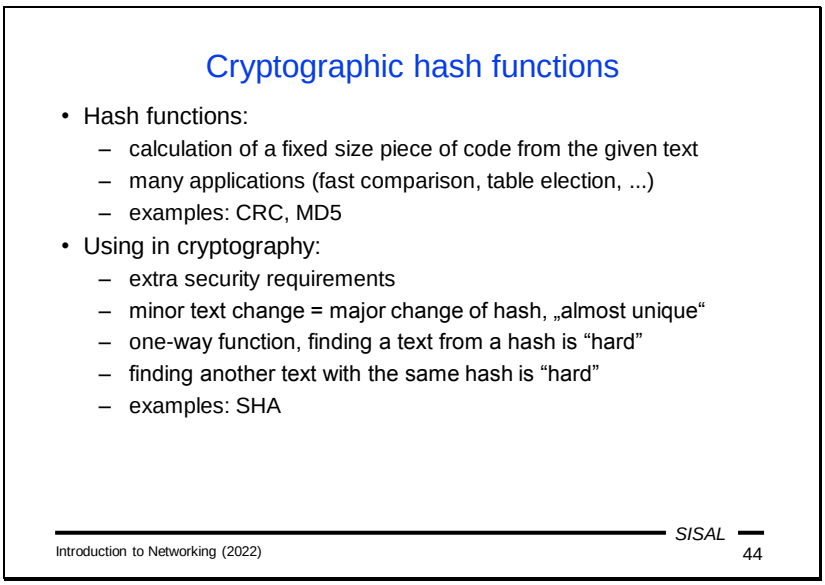
The solution of the problem how to transport securely encrypted data over an insecure channel comes with the **asymmetric cryptography**.

The basic idea is to use two different keys, one for encryption and one for decryption. Of course, it must not be possible to deduce one key from the other. Then you can simply publish one key from the pair while the other should be stored in a safe place.

Just to get an idea of how it works, consider ordinary multiplication – you can easily multiply two numbers but if you have a product, it can be difficult to find its factors (if they are large prime numbers). Such functions are called **one-way** functions. A more sophisticated mathematical one-way function is a discrete logarithm.

The advantages over symmetric cryptography are clear – there is no need to transfer a *shared secret* between the parties. However, a big complication is that these methods are extremely slow and thus only suitable for small amounts of data. But we already solved the problem of reducing the size of transported data using symmetric cryptography!

So we avoid the problem of necessity of sharing a common secret, but there is another problem: we need to **trust** the published key, i.e. to believe that it is correct and that the corresponding secure key really belongs to the person or service we expected.



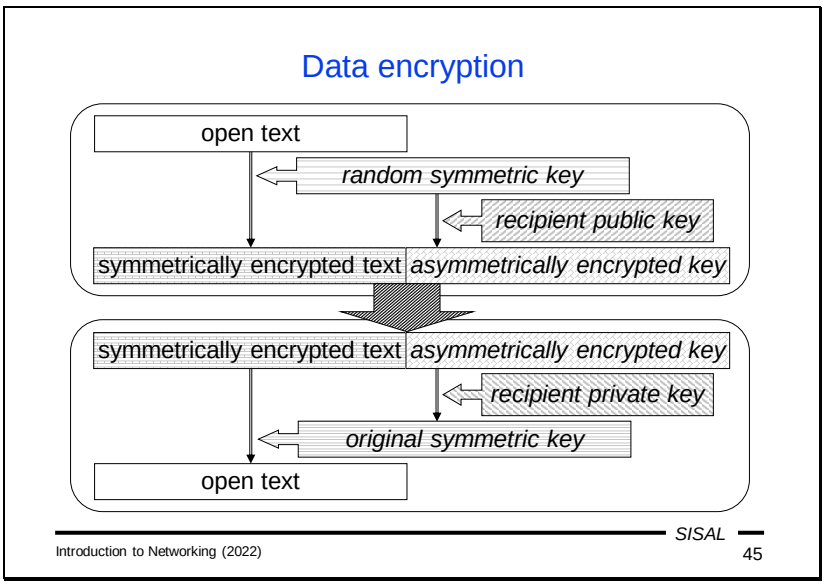
The last type of cryptographic algorithm we will talk about is a **hash function**.

In general, a *hash function* is a function that creates a code of short fixed length from any data. These functions are widely used in programming, e.g. for a fast search through a table, fast comparison of texts etc. The principle is that instead of working with an entire piece of data, we work with its hash only. Of course, the code cannot be unambiguous, so either we accept the risk (e.g. when we compare two versions of a source code file, the probability that they differ but their hash values are the same, is so low that we can ignore it), or we use the hash only to reduce the number of elements we have to deal with and check completely.

However, such weak algorithms are not acceptable for use in cryptography; there are additional conditions for a good cryptographic hash algorithm:

- It is important that even a minor change in the original data causes a fundamental change in the hash value, so the hash is “almost unique”.
- The function must be a one-way function; finding a text that corresponds to a given hash must be “difficult”, as must be finding another text that has the same hash value as a given text.

Slide 45



Now we can look at two basic operations used in cryptography, data encryption and origin verification.

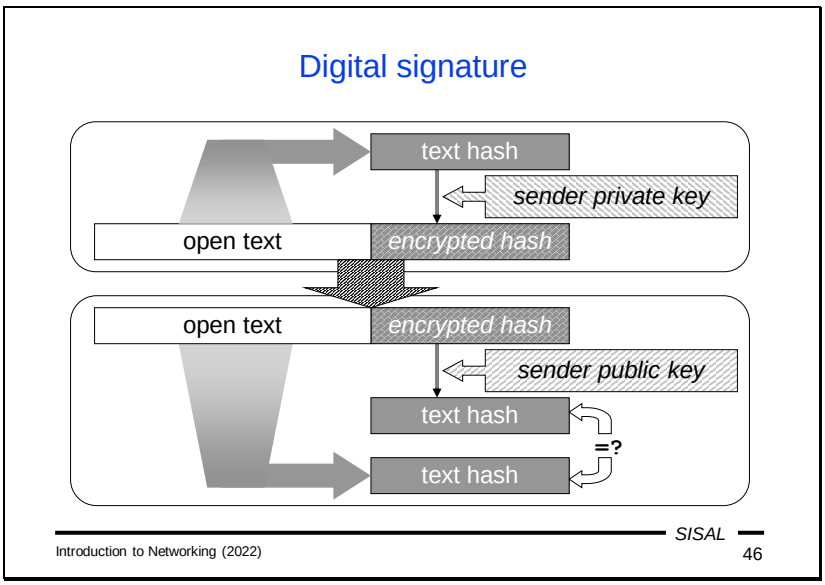
For efficient **data encryption**, we use a combination of symmetric and asymmetric algorithms.

Suppose that we have a text that should be sent encrypted over an insecure channel (e.g. e-mail). We generate a random key key_{sym} and symmetrically encrypt the text. The text can be large since we use symmetric cryptography which is fast enough. Now we need to pass key_{sym} to the recipient, so we need to encrypt it. But the key is short, so we can now use asymmetric cryptography (the speed is not an issue here) and encrypt key_{sym} with the **recipient's public key**. This encrypted key is then attached to the encrypted text.

After successful delivery, the encrypted key is taken and the **recipient's private key** is used to decrypt it. Now we have the original key_{sym} and the data can be symmetrically decrypted.

Note: Especially for e-mail, this approach has also another big advantage – if you have more than one recipient, you encrypt the data **just once**, then encrypt key_{sym} separately for each recipient using his/her public key and attach all the encrypted keys for all recipients to the encrypted data.

Slide 46



For **digital signatures**, we use a combination of asymmetric cryptography and a hash function.

The sender selects a hash function, takes the text (either plaintext or encrypted) and calculates its hash. Then it takes the **sender's private key** and encrypts the hash. The encrypted hash is then attached to the original text (along with the hash algorithm parameters that were used).

The recipient takes the text and uses the given hash function and parameters for a new hash value calculation. Then it takes the **sender's public key** and decrypts the hash originally calculated and encrypted by the sender. If both hashes are equal, we believe that the text was really sent by a person who had access to the sender's private key and that the message has not been modified.

Note that I didn't say "mail was sent by the right person"; we only know that the sender was able to work with the private key, not that it was really the right person…!

Be careful about the properties of these two cryptographic operations: Data encryption protects data from being read by anyone while a digital signature prevents anyone from modifying data sent by the original sender, or from sending completely different data without the recipient revealing it.

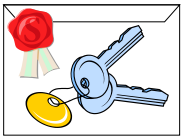
Diffie-Hellman algorithm

- Method of information exchange between two partners via an open channel to get a common secret (e.g. a symmetric key)
- Used in many protocols exploring the symmetric cryptography
- Procedure description:
  1. Alice picks a secret number a and public primes p and q.
  2. She computes $A = p^a \bmod q$ and sends p, q, and A to Bob.
  3. Bob picks a secret number b, computes $B = p^b \bmod q$ and sends B to Alice.
  4. Alice computes $s = B^a \bmod q$ and Bob the same $s = A^b \bmod q$.
- Principle:
  - $A^b = (p^a)^b = p^{ab} = p^{ba} = (p^b)^a = B^a$
  - Without knowing of secret numbers a and b and when choosing large prime numbers p and q, computing the s from A and B is considered to be “hard”.

There is another widely used mathematical principle that allows two partners to agree on a common secret using communication over an open plain-text channel, the so-called **Diffie-Hellman** algorithm. It is also based on one-way functions and the idea is that if both sides keep a secret number and only products of the discrete logarithm are sent via the open channel, then no one who does not know the secret factors, can calculate the final shared secret.

Slide 48

Public key authenticity



- Does the identity tag belong to the key?
  - people usually can make sure that they communicate with the proper partner prior to disclosing some secret
  - the key may be and should be verified from independent sources
  - applications must use some automated way
- Authenticity is verified by a third party and its signature is appended; this can be
  - someone who is known by me and I have his or her key verified ("*web of trust*")
  - well-known public certification authority; listed e.g. in browsers, however such list's credibility is not absolutely sure

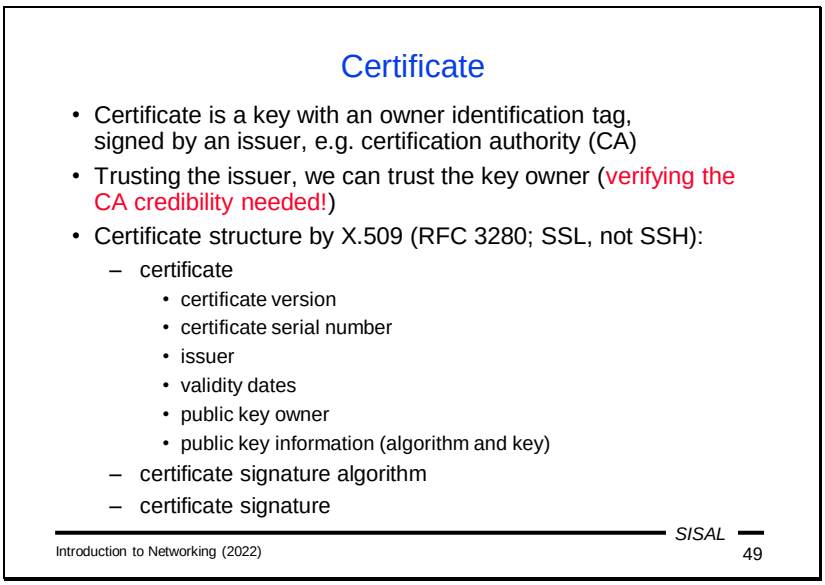
As mentioned earlier, asymmetric cryptography has one fundamental problem: how to prove or verify whether a (public) key belongs to a specific identity (person or service)? If we communicate in person with someone known to us, we can usually recognize whether the person is really our partner – by their greeting, speech style, referring to some common experience etc. If we are not absolutely sure, we can use another independent source to verify the identity. However, if we need to prove identity in software, we must involve some technical means.

In real life, a similar situation is using normal keys. If you have many keys, you will probably make tags to identify them and attach the tags to keyrings. And because you did this by yourself, you will probably believe that the tags match the keys. However, if you are dealing with a key received from someone else, you must believe that its tag is correct. If you want to be sure, you can e.g. arrange that the key with the tag is to be placed in an envelope and sealed by an authority you trust.

The same principle applies to software keys. You can attach **an identity tag** to a key and let someone **confirm** that this coupling is correct. There are various approaches to this type of verification.

- For example, PGP keys for e-mail signing use the method of a *web of trust*. A user creates a key and an identity tag and asks some other users to sign his key+tag. This signed key is then published and all users that trust any of the signers' keys, may decide to trust your key as well from now on. And when you sign other people's keys, the web of trust grows.
- The Public Key Infrastructure framework instead uses special organizations called Certification Authorities (CA) for signing. A CA signs your key+tag pair and if someone trusts this CA, your key is trusted, too.

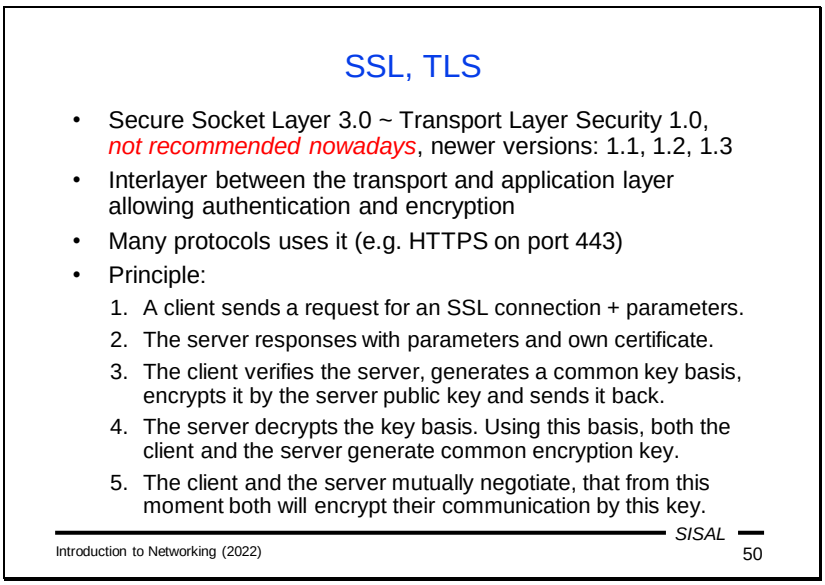
Slide 49



A **certificate** is a public key with a corresponding identity tag attached and signed by a Certification Authority. A user or client that needs to validate the authenticity of a key, downloads its certificate and checks the signature in it using the public key of the signing CA (CA_1). If this key is trusted by the application performing the validation (either it is stored locally in the application, or in the operating system), the original key is trusted as well. If the key of CA_1 is not trusted, the application must get a certificate of it (it can be attached to the original certificate, or the application must download it). This certificate is signed by another CA; let's call it CA_2. Now we must validate the signature of CA_2 using a similar principle. This is a so-called "*trust chain*" that ends either with a known CA, or with failure.

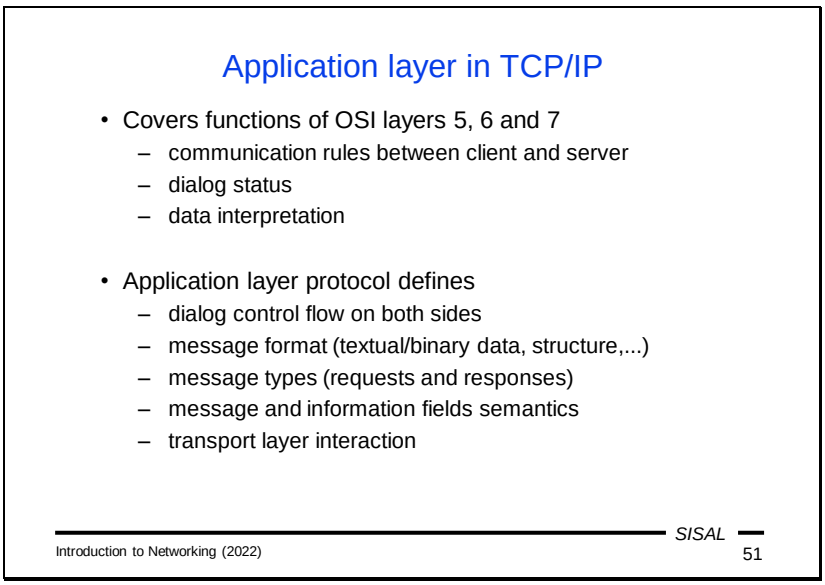
The list of "trusted" CAs is distributed together with the operating system or with software that will use it. For instance, if you install a web browser, a list of CAs trusted by the browser manufacturer is installed at the same time. There are some changes in the list from time to time and the browser automatically updates the list to keep it up-to-date. The problem is that from time to time a CA with a bad reputation (e.g. a CA that does not sufficiently verify key ownership) gets onto a list of trusted CAs. Once this is discovered, all the keys signed by this CA must be revoked.

Slide 50



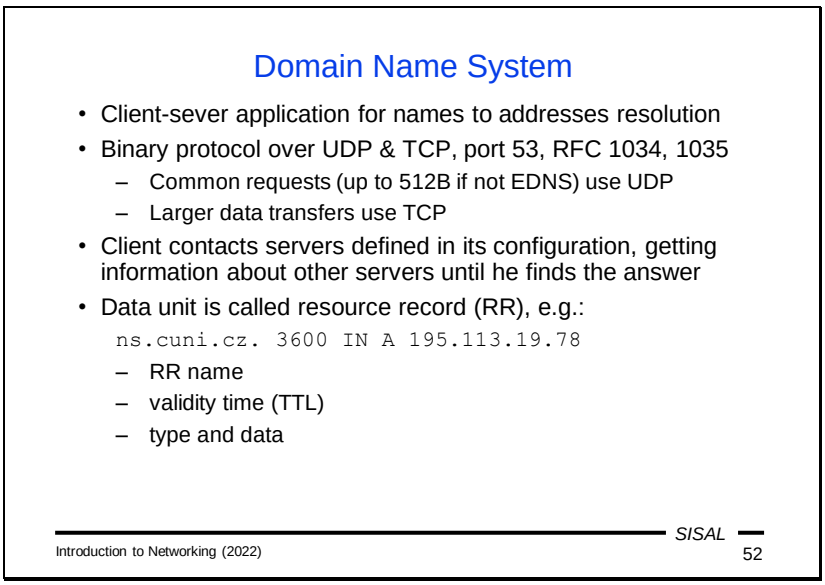
Now we have enough information to explain how the old TCP/IP protocols are currently used in a way that ensures their security. The general method is to insert a special intermediate layer in between the transport and application layers; this layer manages endpoint identification and data encryption. The layer was originally called Secure Socket Layer and the message “s” was appended to the names of the protocols that used it. For example, the URL scheme “https” means “HTTP over SSL”. The important fact is that it is **not a new protocol**; the scheme only says that the SSL interlayer is used. In version 3.0 SSL was renamed to Transport Layer Security 1.0 after slight modifications, but the terms SSL and TLS are still both used interchangeably (except when talking about their version numbers). TLS development still continues and it is now not recommended to use the oldest version 1.0, which was based on SSL.

Slide 51



After the preceding introductory chapters we can now start to talk about the most important protocols at the **application layer**. We already know that in TCP/IP, this layer covers the work of the top three OSI layers. It means that an application protocol in TCP/IP must specify:

- How the dialog can proceed. This means information such as who initiates the connection, who starts the dialog, what kind of operations can be requested and what responses to such requests can be etc.
- A message format. In general, the protocols are either
  - *textual* – i.e. you can look at the data sent by both sides in a text editor, e.g. integer values are sent in their text interpretation
  - *binary* – i.e. the data is structured as a stream of blocks, bytes or even bits, so to understand it you must use a piece of software that converts it into a textual form, e.g. integer values are sent as a sequence of bits representing the value
- Message types. A set of message types for different requests and a set of possible responses to them.
- Data semantics. Each data field (both text and binary) must be described so that there is solely one interpretation of it possible.
- Which transport protocol will be used in which situation and how. In the case of UDP, e.g. a maximum message size should be defined, in the TCP case, e.g. a grouping or splitting of messages into blocks can be described.



The Domain Name System (DNS) is a service for *resolving* domain names to addresses and vice versa. In fact, it provides even more information about stored domain names, but IP address queries are the most frequent.

The service is implemented by the protocol of the same name, which uses both the UDP and TCP transport protocols.

- Common queries are handled in UDP. Questions and answers are short and the overhead of establishing a TCP connection is unnecessary. In addition, UDP allows the client to respond more quickly to any slow or missing server responses. Originally, the limit for a UDP message was 512 bytes, but today the extended protocol (EDNS) is commonly used, which allows the client and server to agree on larger sizes.
- If a response exceeds the size limit, the server sends only the part that fits in the message and sets the TC (truncated) flag. The client can then repeat the query using TCP to obtain a complete answer. Some data exchanges (e.g. database updates between servers) take place directly in TCP.

A common client implementation is to sequentially query the servers it has in its configuration (think about why it has **addresses** of servers there, not their names), gradually increasing its timeout for receiving a response (usually starting at 0.5 sec) until a response is received. If the response does not contain the necessary information, it should include a link to other servers that should be queried to get it.

The protocol is binary; each message contains a header and then a number of so-called *resource records* (RR). Each record contains the name, time-to-live (TTL) in seconds, record type, and corresponding data.

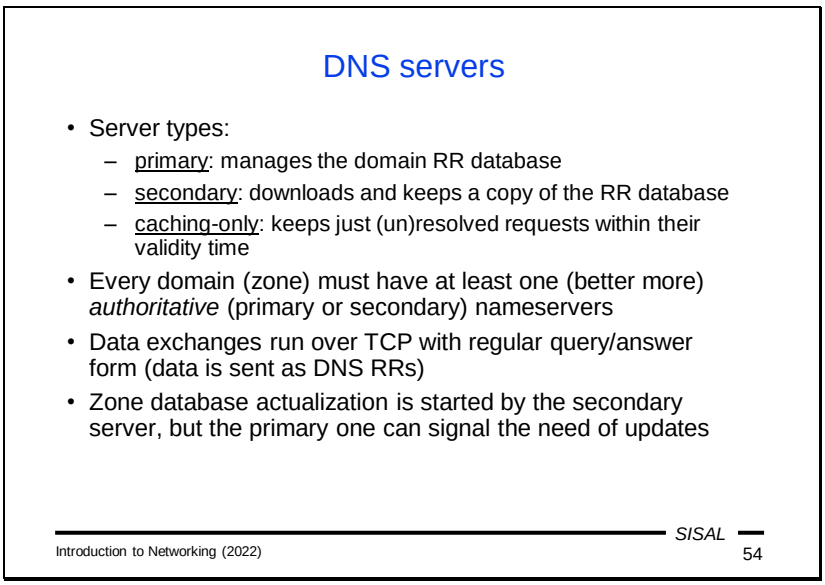
DNS resource records

Type	RR name semantics	Data semantics
SOA	domain name	general domain attributes
NS	domain name	domain nameserver name
A	host name	host IPv4 address
AAAA	host name	host IPv6 address
PTR	reverse name (e.g. IP address 1.2.3.4 is represented by `4.3.2.1.in-addr.arpa`, ::1 by `1.0...0.ip6.arpa`)	host domain name
CNAME	alias name	canonical (real) host name
MX	domain/host name	mailserver name and priority

Overview of the most important record types:

- SOA (Start Of Authority) is the initial record of each domain and contains information such as the data version number, administrator address, etc.
- NS is a type of record for name servers that maintain a database of records for a given domain (usually each domain has more than one NS RR).
- A is a record containing the IPv4 address for a given name.
- AAAA is a record for an IPv6 address. The name of the record is perhaps a bit strange and one would expect something like A6. Indeed, this type of record was introduced as a first draft, but did not work well, and was therefore replaced. The name AAAA indicates that the record contains 4x longer data than record A…
- PTR is a reverse record that is used to convert addresses to names. Since there must be a name and not an address on the left side of each RR, it is necessary to somehow convert the address into the name format. For IPv4, the conversion $IP_1.IP_2.IP_3.IP_4 \rightarrow IP_4.IP_3.IP_2.IP_1$.in-addr.arpa is used. The reason for the reverse byte order is the reverse hierarchy of names (right to left) and addresses (left to right). This allows, for example, placing the server responsible for the $IP_1.IP_2.IP_3.*$ network, and thus the $IP_3.IP_2.IP_1$.in-addr.arpa domain, directly in this network. For IPv6, the address is divided into a series of half bytes; their hexadecimal values are separated by periods, and the ip6.arpa domain is appended. Every computer connected to the Internet should have a reverse record, because some servers require clients that connect to them to have such an entry.
- CNAME (Canonical NAME) is an alias creation record. On the left side is the name of the alias, on the right side is the canonical (or real) name of the computer (beware, there is a common misunderstanding of the left and right sides). Both the record name and the description in the RFC clearly specify that the alias should not lead to another alias. The reason for this is the reduction of additional queries. Unfortunately, this restriction is inconvenient and this rule is commonly violated.

- MX (Mail eXchanger) is a record defining which server receives mail for a given domain or computer (usually more than one MX exists for a domain). This is a nice example of how to separate a domain record from the record of a particular service in that domain. It works very well with e-mail and it is a pity that a similar simple mechanism has not been introduced for the WWW. As a result, **host**-specific (A or AAAA) records are now commonly defined for a **domain** so that users can omit writing "www" in browsers. A solution using a special "web server" record would be more conceptual.

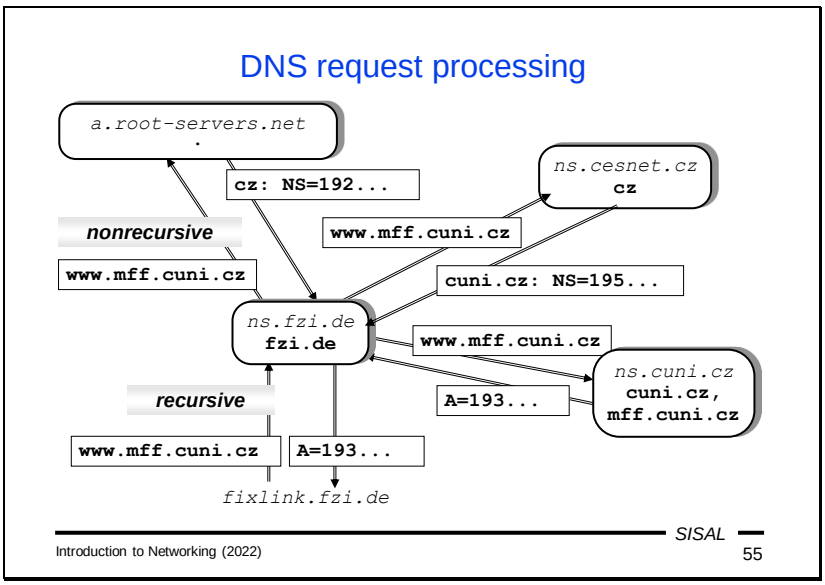


For handling queries belonging to a particular domain, we divide servers into three groups:

- The most important server is the so-called primary (or "master") server. It manages the domain RR database.
- Several secondary (or "slave") servers are usually set up as backups. They periodically download the current contents of the database from the primary server.
- All other servers, if they access any records for a foreign domain, usually cache them for some time, which is why we call them caching-only.

The first two types are collectively referred to as authoritative servers because their data comes from a trusted source. All authoritative servers return answers marked with the authority flag, and clients can choose any of them for making a query.

Database updates are initiated by secondary servers depending on a period defined in the SOA record. Outside of these periodic times, the primary server may inform the secondary servers that the data has changed to such an extent that it is appropriate to perform an out-of-order update. This mechanism reduces the volume of communication between servers, but a consequence is that even an authoritative server can return outdated data to clients for a short time after a data change.



Let's now look at an example of the query flow.

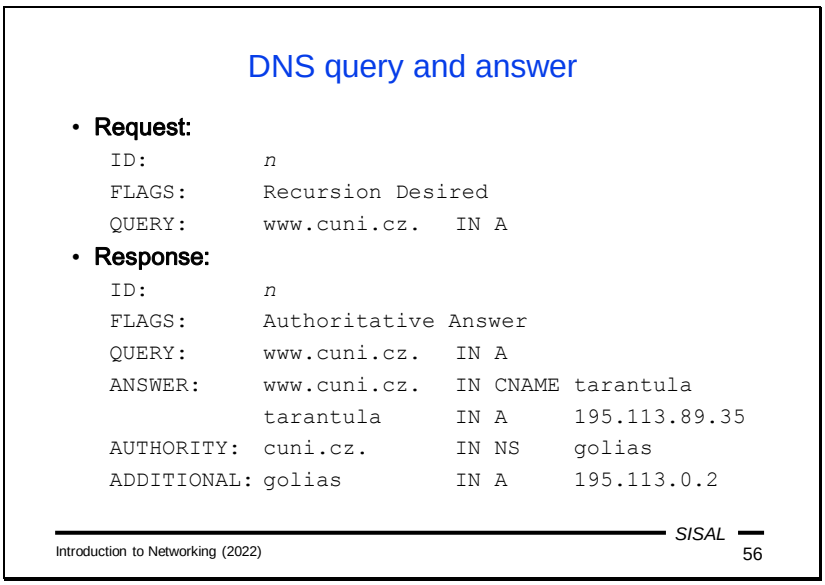
- Let's imagine that a user on a computer in a domain types the address www.mff.cuni.cz into the browser's address bar. In the usual implementation of a client on a workstation, a query is sent to a name server in the domain where the query originated. The query will be **recursive**, which means that the server takes responsibility for complete processing the query and sending the response. If the selected server does not currently have any relevant information in its cache, it needs to start looking for it.
- The server has no information about the domains mff.cuni.cz, cuni.cz or cz. It must therefore contact one of the so-called root name servers, whose addresses it has in its configuration. However, these servers do not handle queries recursively – they will only find the most relevant answer to the question in their database and send it. In our case, it will be an answer such as "send your query to the name server named… cz, whose address is…".
- The server caches this information and continues by querying the proposed server. In this way, all intermediate servers are cached until the query arrives at some authoritative server, which sends the final response.
- This response is also cached and finally sent to the client that originally requested it.

You may be interested in why we send the entire query to the root server, when it will most likely answer us only with a record for cz. There are several reasons for this:

- The basic problem is that DNS is designed so that a **dot** can be part of a **name**! It's as if you could use slashes or backslashes in file names in the file system. This decision brings certain advantages (I can set up a website www.group.abc.com despite the non-existent domain group.abc.com), but also a number of disadvantages. In short, a DNS client cannot decide which dots in a name are

really domain delimiters and which are not. Therefore, the query must be sent in its entirety – to someone who has this information.

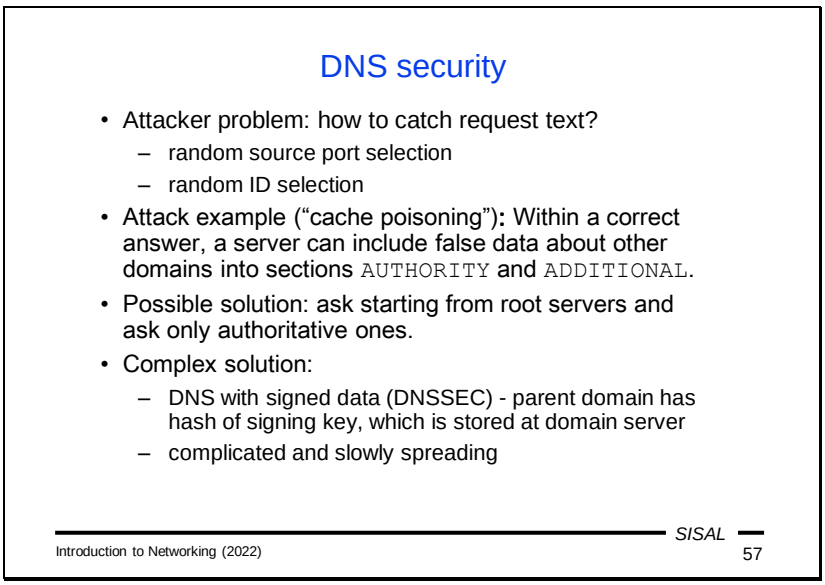
- In the example in the picture, we see that we saved one step by sending the entire query instead of a partial query "where is the server for mff.cuni.cz“ to the server for cuni.cz. The server for cuni.cz is a secondary server for mff.cuni.cz, so it could immediately send us the required final authoritative response.



Before discussing DNS security, let's look at what a request and response look like.

A DNS query consists of a header that contains, among other things, a 2-byte random number (identifier) of the query and flags (e.g. a recursive response request), and a so-called QUERY section with a single RR containing the queried name and type (this record is exceptional in that it does not have a data part).

The answer again contains the query identifier and flags (e.g. the authority of the answer) in the header and the repeated query in the QUERY section. This is followed by an ANSWER section containing RRs with answers, an AUTHORITY section containing a list of name servers that can give an authoritative answer or at least authoritative information about some of the parent domains contained in the query, and an ADDITIONAL section containing additional information (addresses of name servers from the AUTHORITY section, addresses of the MX servers listed in the ANSWER section, etc.).



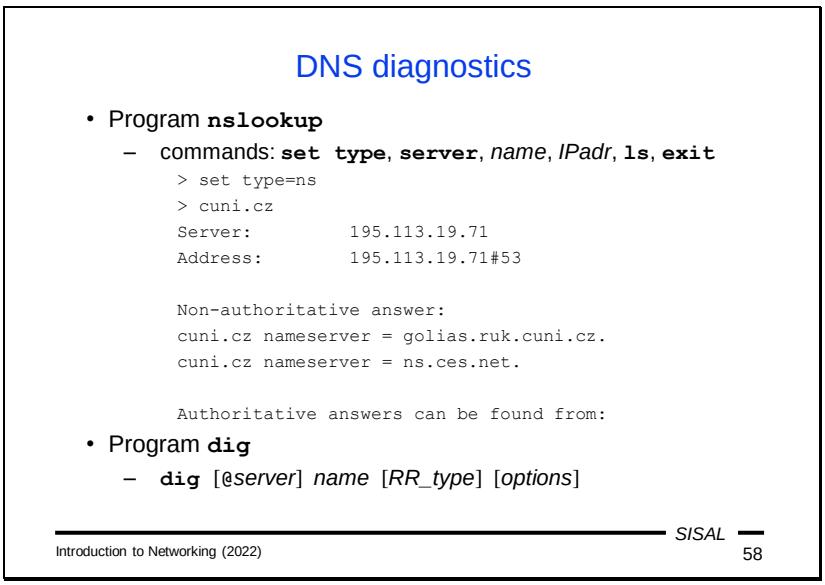
DNS security is based on the fact that it is not easy for a potential attacker to get to the exact content of the query in order to spoof the answer. If it can attack and eavesdrop on the local network, an attack is possible. However, if the query leaves the local network, it is routed between the network routers to the destination server, and it is difficult for an attacker to get to the query there. The second option for an attacker is to expect the question, guess its content and send a false answer. This procedure is complicated by the random selection of a source port of the UDP query and a random ID – this gives billions of possibilities.

It is therefore necessary for an attacker to choose other ways. As an example of a protocol weakness, we can mention the so-called *cache poisoning* attack. An attacker legally operates a server for a domain and forces the client to send him a DNS query (for example, by a spamming advertisement). The query contains a correct answer (making the attack more insidious), but the AUTHORITY section contains faked information that e.g. the server for the cz domain is also this attacker's server. And if the client is naive, it caches this false information and from then on the attacker gains complete control over queries directed to the cz domain from the given client (or worse – from the whole network).

The solution for the client, of course, is to proceed with queries from the root servers and cache only the information to which authoritative responses lead.

Weak DNS security has led to the creation of another extension (DNSSEC), the essence of which is the signing of all records in the domain with a key that is stored in the parent domain. An attacker who does not know the key cannot forge records, and cannot forge the key without access to the server of the parent domain (e.g. to forge data about the domain cuni.cz, he would have to attack the server for the

domain cz). The problem with DNSSEC is that to ensure sufficient security, the protocol became extremely complicated, as is the management of signed domains, and as a result it has spread very slowly.



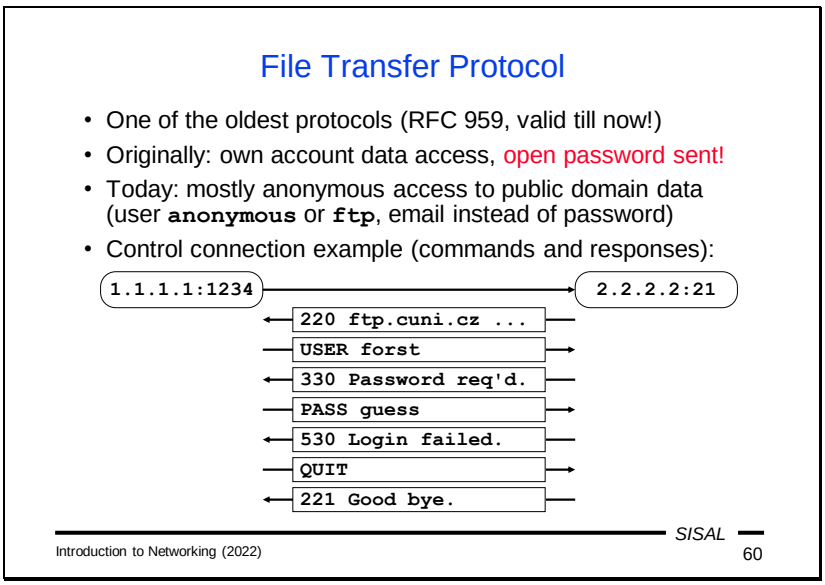
If we need to check the behavior of the *resolver* (the operating system component responsible for using the DNS service), we have the **nslookup** program available. After starting, the program displays a prompt and we can enter commands for specifying the server we are querying and query parameters, as well as the names we want to resolve. The program can also be called from the command line and the query can be specified using parameters.

On UNIX computers we usually have a program with a similar function called **dig** or **drill**.

Summary 3

- Describe three basic groups of cryptographic algorithms and the differences between them.
- Describe the principles of encryption and digital signatures.
- Describe the purpose and principle of the Diffie-Hellman algorithm.
- How is the authenticity of a public key verified?
- What is the difference between HTTP and HTTPS?

- When is UDP used in DNS and when is TCP used?
- Describe the purpose and behavior of the secondary name server of a domain.
- Describe the procedure for resolving a DNS query.
- Describe the security risks of DNS.



The File Transfer Protocol (FTP) is one of the oldest protocols still in use today. Its original purpose was to allow a user account to transfer files from or to a remote computer. To log in in this way, it is necessary to authenticate the user, and in FTP, this is done by transmitting a password in the clear, which poses an obvious security risk. However, this approach was soon supplemented by an equally (and perhaps more) important but less risky variant, in which the user logs in as an anonymous user and instead of a password, a user provides an email address (for statistical purposes only). A user logged in in this way has access to freely available documents, programs, etc. In its time, FTP was an information source that was as important as the WWW is today. And some servers still use FTP archives without the user's knowledge: web browsers display a link to a resource available via FTP in the same way as a link to HTTP, and when the user clicks on such a link, the browser establishes an FTP connection and makes the result available to the user. A typical user does not have to know at all that the whole transmission took place by another protocol.

FTP is a text based protocol; the client establishes a so-called *control connection* to the server on port 21 and then sends command lines, while the server sends response lines over the same channel.

Slide 61

Response codes

- For simpler automated processing of responses, they start with 3-digit number
- The first digit expresses response severity:
  - 1xx **positive preliminary reply** (action was started, further responses expected)
  - 2xx **positive completion reply**
  - 3xx **positive intermediate reply** (further commands necessary)
  - 4xx **transient negative completion reply** (action failed, however repeating later makes sense)
  - 5xx **permanent negative completion reply** (action failed and will fail later, too)
- A similar schema adopted by many protocols

FTP was one of the first protocols designed to identify responses with a three-digit code, where the first digit expresses the nature of the response. This method allows clients to easily process responses without having to analyze the text of the response, which can also be localized into another language.

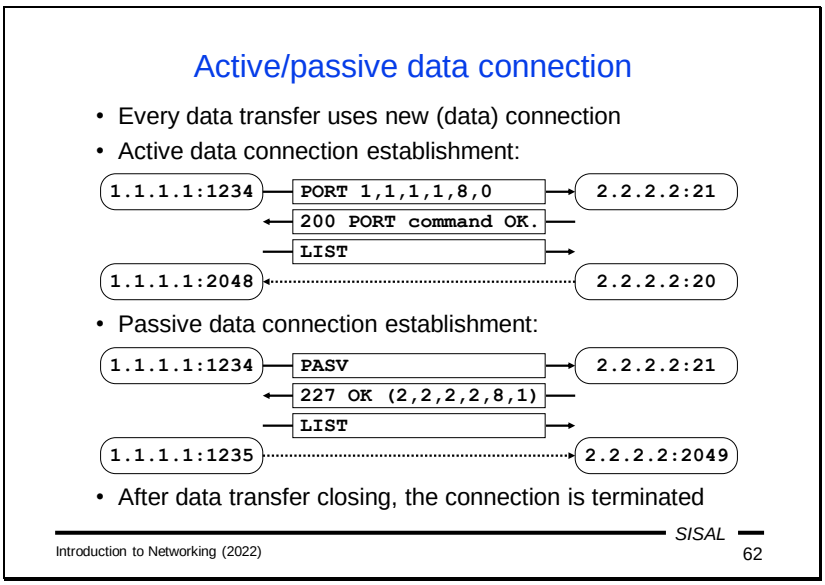
Answer types:

- A positive preliminary reply. The answer means that the server has accepted a request and has started processing it. No further action is expected from the client at the moment; the ball is in the server's court.
- A positive completion reply. The server has successfully completed the requested operation.
- A positive intermediate reply. The answer means that the server has accepted the request, but needs more information from the client in order to complete it. The client should know what information is needed according to the operation that was requested. For instance, during a user login operation, the client must respond to a 3xx response by sending a command containing a password. In other words – everything is fine, but the server is waiting for client activity.
- A transient negative completion reply. The operation failed; however, the error is not fatal. The usual reason is a temporary server overload. The client can repeat the request after some time without any change and there is a chance that the request will succeed.
- A permanent negative completion reply. This error is fatal and even resubmitting the same request will not succeed.

A similar mechanism was later adopted by a number of other protocols. In some of these (e.g. SMTP) the three-digit prefix has a similar meaning as in FTP, in others

(e.g. HTTP) there are slight differences (surely at some point you have received a 404 response from an HTTP server – page not found).

Slide 62



Another aspect of FTP which overlaps with current protocols is the use of additional data channels.

In addition to the control connection, used only for sending client requests and server responses, special *data connections* are opened, through which all data content is transmitted. For each new transfer (a file download, file upload or directory listing), a new TCP connection must be opened, which will be closed again after the transfer is complete. But there are problems with opening additional data channels – both parties have to agree on **who** will open the data channel and to which **address and port** the connection should be established.

FTP distinguishes between two ways of opening a data channel, depending on the server's role:

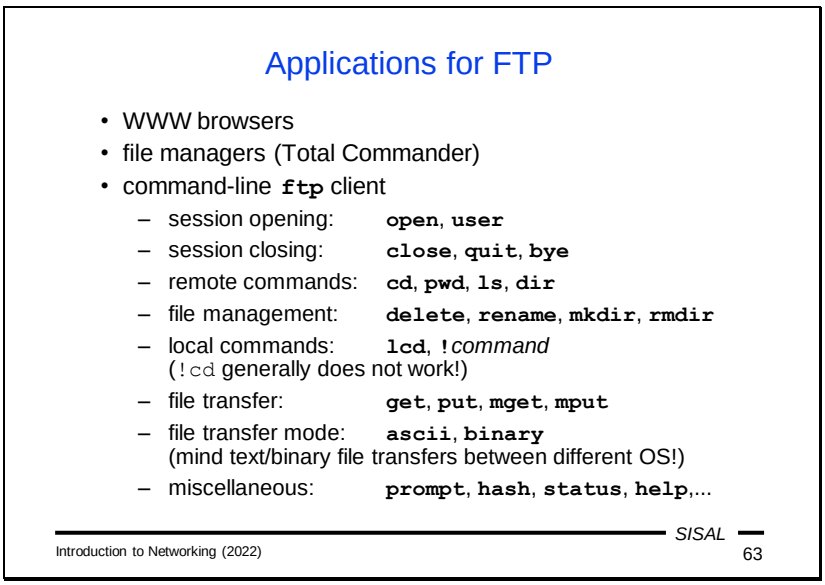
- An **active** data connection is established by the server. The server theoretically does not need any additional information – according to the original concept, it connects to the same address and port that the client uses. On the server's side, the connection will use the server's IP address, but it cannot use port 21 (all parameters of the new connection would be the same as the control connection and this is not possible), so another port 20 called *ftp-data* was reserved, which the server can use as the source port. An alternative, of course, is to use a generic port, like a regular client does when establishing a connection. However, the solution using the client's address and port as the destination address for the data connection is not entirely practical, so today most implementations initiate an active data connection with the PORT command (or EPRT for IPv6), which lets the server know another address and/or port. The syntax of the command may seem a little strange since the notation is a bit harder to read, but it is quite efficient – it's

just six bytes of the socket address (4 bytes of the IP address and 2 bytes of the port number).
- In addition to the active method of establishing a data connection, there is also a **passive** method. In this case, the connection will be established by the client and will therefore need an address and port from the server. The client requests them with the PASV command (or EPSV for IPv6) and the server responds with a response that contains the required socket address in parentheses.

The proposed solution for establishing data channels corresponds to the state of network security in the seventies of the last century. All addresses were public and every address was accessible from anywhere. But at the moment when we start implementing security restrictions and using private addresses, the situation becomes more complicated. In the case of an active connection, the server will probably not be allowed to open a connection to the client, even if the client has a public address. However, the client is likely to have a private address today, to which the server cannot connect at all. The solution is to involve higher logic in the network address translation, where the NAT router must modify even the **content** of the messages between the client and the server and change the addresses sent in them accordingly. With a passive connection, the situation is slightly better: the server sends its public address to the client, so the only restriction may be if the router on the perimeter of our network allows opening the connection only to a **defined list** of ports.

The issue of opening additional data channels is not specific to FTP only. For instance, the SIP protocol, which is most often used today for voice transmission over a TCP/IP network, faces exactly the same problems.

Just as a reminder, the direction in which the data connection is being opened is in no way related to the direction in which the data will then flow.



You can currently use the following ways to access the FTP protocol:

- Web browsers. To download a file or list a directory, just enter the appropriate URL; for uploading a file, it is often necessary to install an extension.
- File managers. Many file management applications can work with FTP. For instance, Total Commander can display the contents of a directory on an FTP server (instead of the contents of a local disk) in a panel and perform common operations with files. For the study of FTP, it is interesting that in an additional window it is possible to monitor the FTP commands by which individual operations are performed.
- The last resort is the oldest application, the interactive **ftp** program, which accepts line based commands. Most of them are logical, you just need to be careful when transferring files between operating systems that use different line endings of text files (e.g. Windows vs. UNIX) – unless we choose the correct transfer mode (**ascii** for text files and **binary** for binary ones), the files on the target computer will be unreadable or damaged.

Electronic mail

- General service, exists also out of the Internet
  - off-line sending of messages or files
  - off-line usage of information services
  - mailing-lists, conferences
  - communication outside the Internet
- In TCP/IP based on RFC 821, 2821 and 5321 (SMTP, or ESMTP) and RFC 822, 2822 and. 5322 (message format) on the port 25
- General form of e-mail address in the Internet:

  login`@`*host* or *alias*`@`*domain*

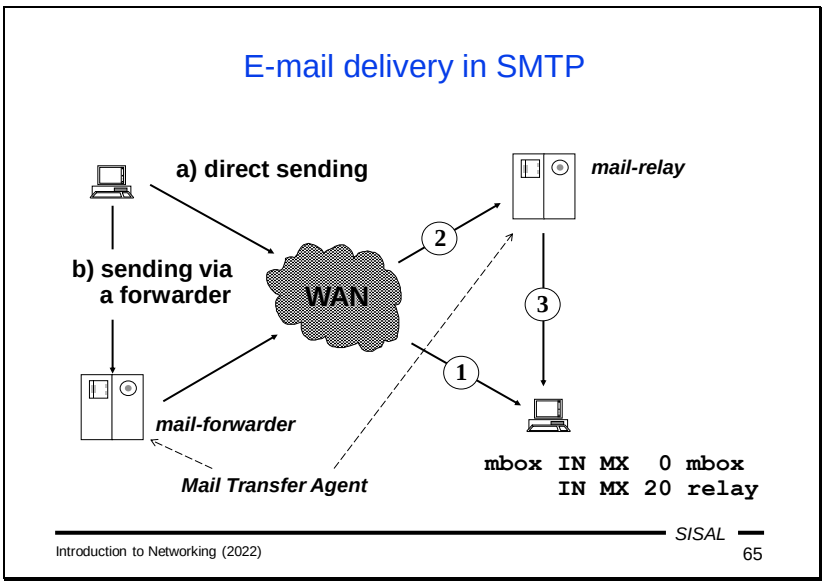
  e.g.:

  `forst@ms.ms.mff.cuni.cz` or **`Libor.Forst@cuni.cz`**

Electronic mail is another very old service, dating back to before the “modern Internet”. Already in computer prehistory, it served as one of the few communication bridges between computer networks of various kinds, and the variety of postal address formats also reflected this. At present, a form composed of two parts separated by an “at” symbol (“@”) become standard on the Internet. In its basic form, the at symbol is preceded by a mailbox name and followed by the name of the server where the mailbox is located. However, this form is both too dependent on the current state (changing the server name would change the email addresses of all of its users) and poses a risk, because a potential attacker can read both the server name and the account name from the address and can try to attack the account. Therefore, for both reasons, an alias is more often used instead of an account name, and a domain or service name is used instead of a specific server name.

The original concept of email was to serve for the transmission of short messages (up to 64 kB), either for personal correspondence or for correspondence within a certain group of people (a mailing list), but also as an off-line method of using various services (e.g. searching for documents in FTP archives). Over time, however, email has begun to be used more frequently also for file transfer, which brings some complications.

The Internet uses the SMTP protocol for mail transmission. It is a text based protocol on TCP port 25 with the principle of sending messages and replies, similar to FTP.



Let's now look at what happens when a user issues a command to send an email message.

The mail program checks the part of the address after the at symbol and finds out which server receives mail for the given domain. Theoretically, it is possible that there are no obstacles in the path between the client computer and the destination server, so the program can establish an SMTP connection to the destination server and try to forward the message directly. However, this is not typical; usually the message's path includes several nodes which pass the message to each other in turn.

On the sender side, the reason is usually that the mail program has a simplified configuration and instead of complex rules, it only indicates that mail is to be forwarded via SMTP to a special server on the local network (a so-called *mail-forwarder*) intended for further mail processing. This step is called *mail submission*. There may be even more such forwarders, e.g. due to several levels of NAT.

Each node that receives and delivers mail is called a Mail Transfer Agent (MTA). Individual MTAs forward the message using the SMTP protocol, where the sending MTA temporarily acts as a client and the receiving MTA as a server. If the server is not the final destination, it queues the message and periodically attempts to deliver it to the next MTA. Note that the number of MTAs along the way has nothing to do with the number of networks or routers.

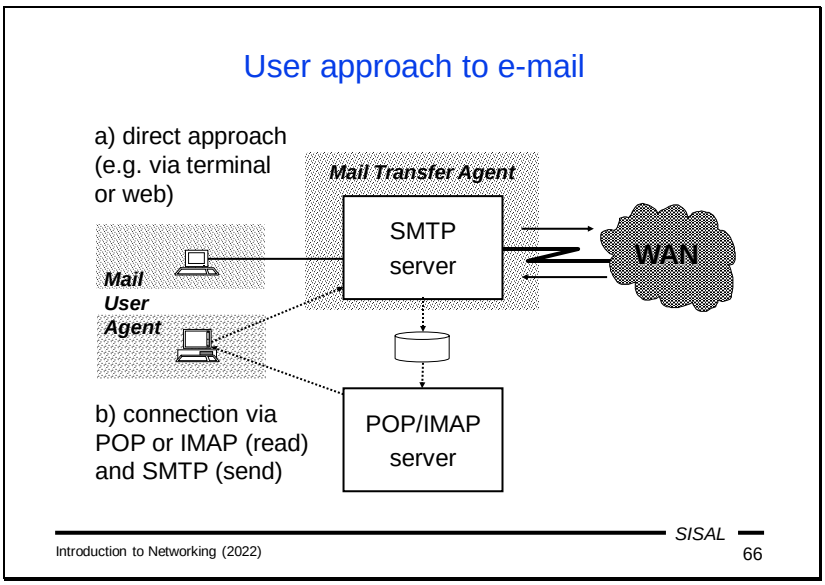
If the destination mailbox is on a server that is freely accessible from the Internet, the last MTA will try to deliver the mail to that server. If delivery is not possible, the mail remains in the queue on the last MTA (generally "somewhere on the Internet"). If the mail server administrator wants to prevent this, he can set up a DNS MX record for

his server. MX records contain the name of the *mail exchanger* that is temporarily or permanently authorized to receive mail for a given machine or domain plus its priority (the lower the number, the higher the priority). In such a case, the delivering MTA sequentially tries to contact individual exchangers, according to priorities, until it succeeds.

The recipient can follow the entire path of the message over the Internet using the Received headers in the delivered message.

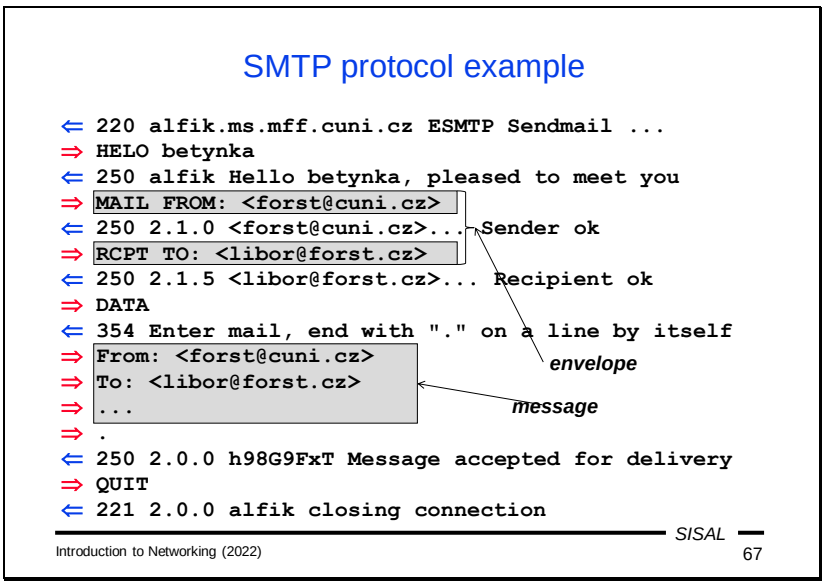
Given how messages are delivered, it is probably clear why this protocol is not suitable for transferring large files. Each MTA node on the path must store the entire message in its queue just to pick up it from the queue a while later and send it on.

Slide 66



From the user's point of view, the mail system is accessible through a "mailer program“, a so-called Mail User Agent (MUA). In principle, there are two ways in which an MUA is connected to the mail system:

- The first option is a direct connection. The user connects from his computer to the MTA, where he has his mailbox. Various connection methods are possible, such as a remote login to the system, or visiting a web application running on the mail server, and the user authentication options also correspond to the type of connection method. In this case, the application itself has direct access to the messages in the user's mailbox and at the same time to the local services of the MTA, so it directly places messages to be sent in the MTA queue.
- An alternative is to connect to the mailbox using one of the special mail protocols, POP or IMAP. A client of such a protocol connects to a server running on the MTA, authenticates itself using this protocol and, based on user requests, sends commands to the server for handling the mailbox. However, these commands only work with received messages. For sending messages, the client must submit mail normally using SMTP, not necessarily to the same server. Therefore, we will also find separate sections for POP/IMAP and SMTP in the client application configuration. For example, in a situation where a user travels and connects to his own server "from the outside", his server may not be willing to receive a message from him due to lack of authentication in SMTP, so the user may not be trustworthy enough for the server. This can be solved, for example, by temporarily using a local MTA of the ISP to send mails.



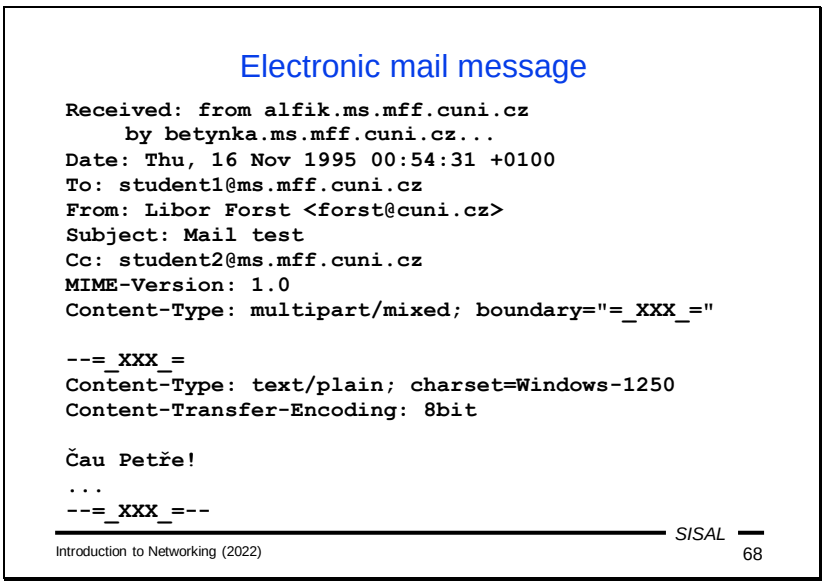
The SMTP protocol is similar to FTP: the client sends individual commands as text lines and the server responds similarly, even including a three-digit code with the same meaning of the first digit.

Sending a new message begins with the client's MAIL FROM command, which includes the sender's address as a parameter (in angle brackets). This command is followed by one or more RCPT TO commands, each with an address of one recipient. The server confirms each recipient separately – if it decides not to accept one, it responds with message code 450 (temporary error) or 550 (permanent error) instead of message code 250. However, if it answers with 250, he **takes over the responsibility** for the delivery of the message or the sending of a message about a delivery failure, a so-called Delivery Status Notification (DSN). The command with the last recipient is followed by a DATA command, to which the server responds with message code 354, after which the client starts sending the text of the message, i.e. roughly what the recipient will then see in his mailbox. The protocol is textual, because the messages were originally so intended, and ends with a line that contains only a dot itself. Therefore, SMTP clients must pay attention to the mail lines that start with a dot (or binary bytes which coincidentally form the sequence CR LF dot!). They send such lines to the server with one extra dot prepended, which the server deletes when saving the message. The server responds to the terminating line with a standard response (250, 450, or 550), and the client can finish with a QUIT command or continue with another message.

Note 1: What relationship is there between the addresses that the client sends to the server as the sender's and recipients' addresses and those that the recipient then sees in the message? **There is none!** The addresses used in the protocol are called the *envelope*, and their relationship to the body of the message is the same as that of

the address written on a real mail envelope and the addresses written on the message inserted in the envelope. It is simply just text that the sender can fake; however, unfortunately, your mail program is then driven by it…

Note 2: An MTA that accepted a message (i.e. replied with code 250 to the specific addressee and to the body of the message as well), but who failed to deliver the message or pass it on to another MTA, has a duty to generate a DSN (Delivery Status Notification) message. Because the sender in this case is the mail system itself, no address is filled in the MAIL FROM statement. Such messages tend to be easier to pass on some nodes (for example, they are less rigorously checked for viruses or spam), which is why some spam machines abuse this method of sending. Unfortunately, this leads some administrators to simply disable DSN delivery. However, this is a gross violation of the rights of their users, because the entire e-mail system is built on the principle of *best effort*. The key element of it is that the sender will be notified of any failure of delivery.



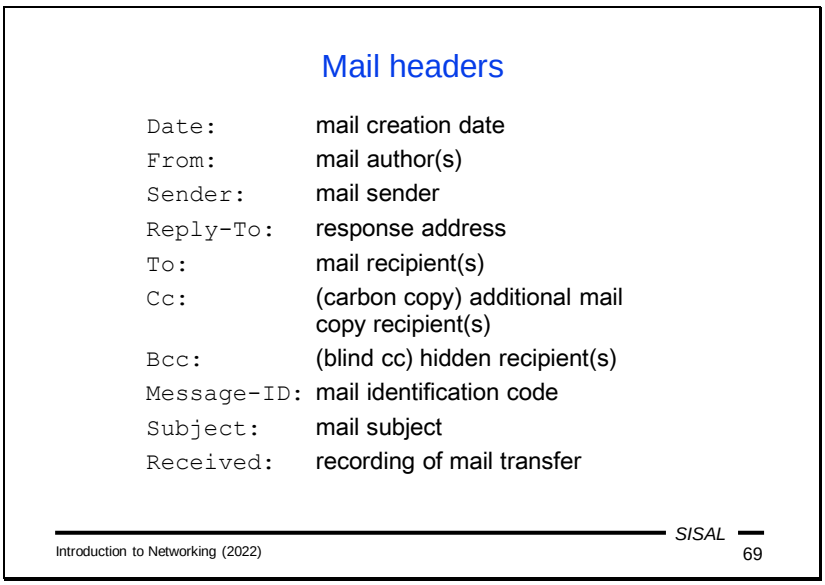
An electronic mail message consists of two parts separated by an empty line.

The first part contains *headers*, lines with control information, with a relatively fixed format and characters exclusively from the ASCII table (i.e. only 7-bit characters 0x00 to 0x7f, no characters of national alphabets). Headers are intended for both end-user information and the operation of e-mail applications, which usually display only a selected portion of the headers to the user, but tend to have an operation or setting that allows the user to see the entire header list.

The second part is the text of the message. In the original concept, it was possible to use only ASCII characters and write only plain text.

Over time, however, the possibility of using non-Latin characters in the text of messages became popular. There is an extension of the protocol called ESMTP, in which the client and server can agree that both are able to accept 8-bit characters – the client can then use the so-called **8-bit** content transfer encoding.

The second major innovation was the ability to define the **structure and semantics** of the message body using the MIME extension. Thanks to this, it was possible to start working, for example, with attachments containing files.

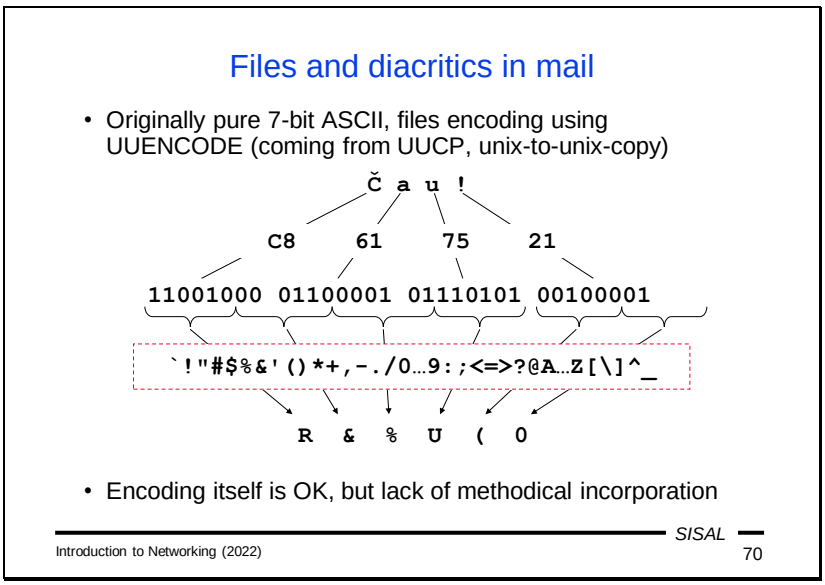


The most important mail headers include:

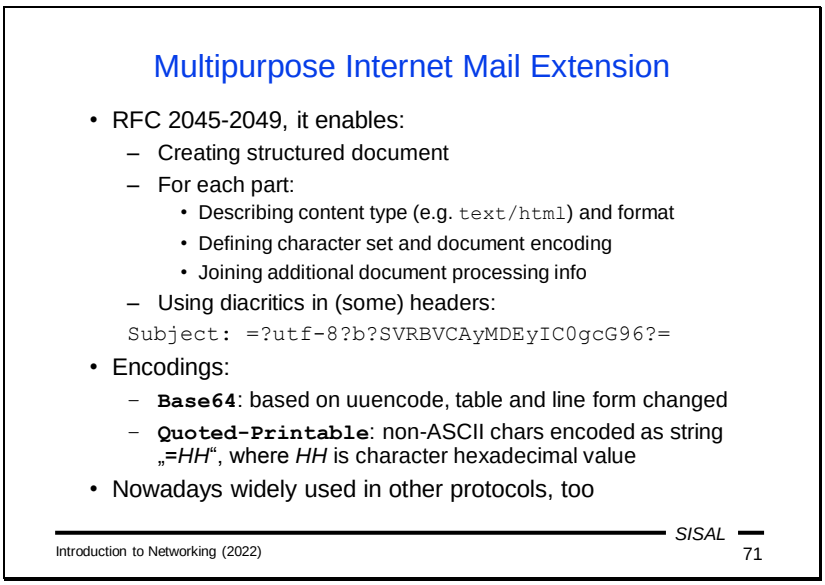
- Date – contains the date of origin of the message in a fixed (American) format.
- From – contains the address of the author (or authors) of the message; either a simple e-mail address or an address accompanied by a comment (usually a full name), most often in the format "comment <address>" or "(comment) address". The header was really intended as a list of authors, that is, if the message is written by more people, everyone should be listed here. But mail applications usually don't have any sophisticated support for this, so users ignore it and usually write only one author in the From header. On the contrary, mentioning more "authors" is often a symptom of spam - spam machines are based on the dubious assumption that if they use a "known address" in From, various mail checking automata will be more tolerant, and therefore they commonly have a long list of addresses in the From header.
- Sender – is, on the other hand, a header where there can be only one address. It indicates who in the From list has actually sent the message, or possibly a sender who is not actually in the From list (e.g. a secretary sending a message for her boss).
- Reply-To – contains the address that the recipient should use when sending a reply (e.g. the address of the mailing list from which the message came).
- To – contains a list of message recipients.
- Cc – contains a list of recipients to whom a copy of the message is sent. Logically, these should be people who are not expected to take part in the dialogue and are just "in copy". Note, however, that the average user does not look at whether he or she has been listed as To or Cc when receiving a message. Therefore, it is not a good idea to write a message to your friends starting with "Hey, guys" and send it Cc-ed to Mr. Dean. A practical problem is that many mail applications mix the original To and Cc recipients when creating the response. The name of the header

is derived from the name of carbon copy paper used to make copies when typing on a physical typewriter.

- Bcc – ("blind carbon copy") is a header that does not appear in the actual message, it is displayed only by the mail program and the recipient is added to the SMTP envelope, but not to the text. Therefore, the other recipients do not know that the person was also among the addressees.
- Message-ID – is a technical header produced by the sending mail program. The recipient's program should use it when preparing a response. It allows the logical grouping of answers into discussion "threads".
- Subject – contains a brief summary of the content of the message (such as the "Subject“ used in business messages).
- Received – is a technical header, which is usually added to the message by each node that forwards it. The header contains the name of the MTA, the time of delivery, the local identifier and possibly other data that may facilitate the tracing of the message.



As already mentioned, in the original email proposal it was possible to use only ASCII characters in the entire message. However, users sometimes needed to transfer files that were binary. Users of the UNIX-to-UNIX copy (UUCP) network already had a similar problem at the time. They also needed to send files over a 7-bit channel and they invented the UUENCODE encoding that allowed them to do so. In this encoding, three bytes of the original file are taken, these 24 bits are divided into four groups of six bits, and these six-bit codes are converted to four printable characters using a fixed table. The encoded file is therefore 33 % larger than the original. UUENCODE was created at a time when it was not common to distinguish between lowercase and uppercase messages, so the table has only 26 (uppercase) messages, 10 digits and the remaining 28 places are occupied by almost all other characters, including those with special meanings in various environments (a semicolon, an apostrophe, a quotation mark, an asterisk, square brackets, at symbol, etc.). However, the encoding already existed, and it was possible to insert an encoded file into the text of an email message merely by surrounding it with begin and end lines. The main problem with this solution was that the structure of a message, i.e. whether it contained embedded files and the attributes of those files, could be determined only by analyzing the entire text of the message.



The problem of structuring email messages was solved by the Multipurpose Internet Mail Extension (MIME), which was later, despite its name, used in many other protocols (e.g. HTTP).

Every document in MIME format contains a header and a body (similar to the mail itself), where the header defines attributes such as:
- a document type; MIME types consist of a main type plus a subtype (e.g. text/html, image/jpeg, audio/mp3, application/pdf),
- the character set used,
- an encoding method,
- the original file name,
- the intended method of processing (to display as an attachment or to insert into the text)…

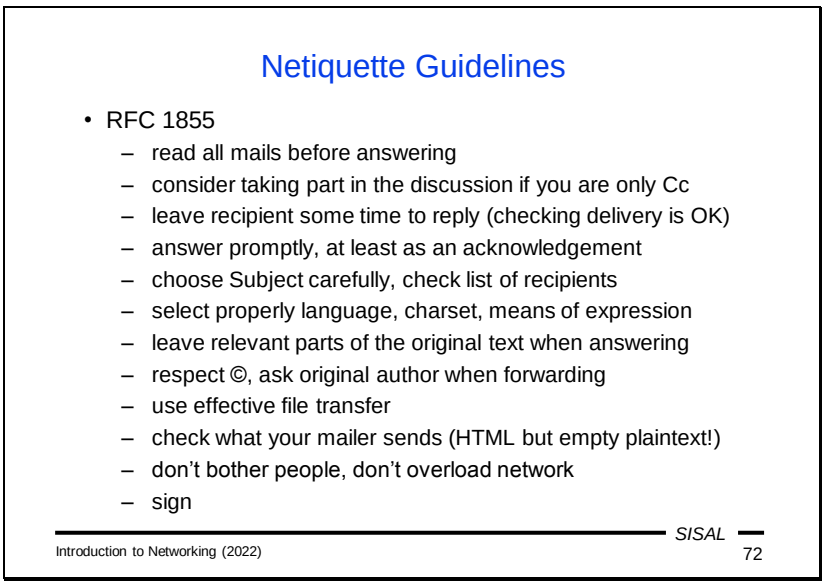
However, one of the key document types is **multipart**, which means that the body of the document is further structured. The sending application generates a random string, which is used as a separator for each part of the structured document, and each of these parts is again a standard MIME document. If you attach two files to a message in a typical current mail program, the message will be sent as a MIME multipart document, where the first part will be the message text and the other parts will be the attached files.

MIME defines two basic encoding methods:
- The method called **Base64** is essentially nothing more than the old known UUENCODE. MIME only replaced the original coding table with a more modern one (today's computers do not have a problem with lowercase and uppercase messages, which gives us 52 characters instead of 26 and if we add digits, we

need to add only two non-alphanumeric characters, plus and slash) and slightly changed the line format.

- The **Quoted-Printable** method is mainly used for text files, where most of the text consists of ASCII characters. Each non-ASCII character is converted to the sequence "=HH", where HH is the hexadecimal value of the character (e.g. the equal sign itself must be encoded as "=3D"). The advantage over Base64 is that the text remains at least a little bit readable. From the point of view of coding efficiency, the ratio of ASCII and non-ASCII characters is important – if we encode text containing only non-ASCII characters, we will get a code size of 300 % of the original (compared to fixed 133 % for Base64), and for a pure ASCII text we get only slightly above 100 %.

In addition to the advantages for transferring documents, MIME also brought the ability to insert non-ASCII characters into (some!) headers (e.g. the Subject). The encoding starts with the characters "=?" followed by an identifier of the character set used, a question mark, an identifier of the encoding ("b" or "q"), a question mark, the encoded text, and the final sequence "?=".



The style in which e-mail should be used is subject to certain rules. Some of them are similar to the general rules of non-electronic communication, some are completely new. Their nature is a recommendation, but most of them should be really followed under normal circumstances. The specificity of electronic communication has even led to the most important rules being written in the form of an RFC.

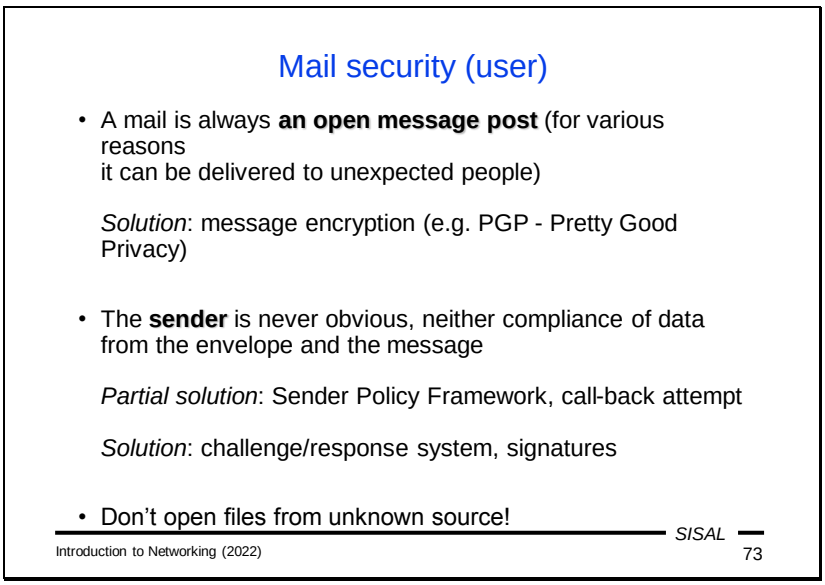
So here is how a user should behave:

- When you open a mailbox with incoming messages, you should first look at all the messages received and only then start replying to them. It often happens that the discussion in a thread has already progressed in such a way that your answer to the first message in the sequence would be useless or confusing.
- The RFC says that if you are only a Cc recipient, you should carefully consider whether you want to actively intervene in the discussion. The problem with current mail programs, however, is that after several replies, you usually don't know if you were originally on the Cc or To list. But you should usually know this from the nature of the dialogue.
- If you send someone a question or request, you should give them time to answer. Mail is an off-line service and you do not know the recipient’s situation, how soon he or she will get to reading the mail. Of course, you can, for example, send a decently worded message the next day, asking at least for a confirmation that the message has arrived, because you are never absolutely sure about this. Of course, it is also possible in an emergency to contact the recipient in another way and notify him of your message.
- A complementary rule, however, is that you should respond quickly. It means that after reading the message, decide whether you are able to reply within a reasonable time (depending on the nature of the message, it is usually several

hours, no later than the next day), and in the negative case at least acknowledge receipt of the message with an estimation of when you will be able to reply.

- Work with the Subject header. Try to express the essence of the message in a few words so that the recipient knows how important it is and what it is about. A message with the Subject "Query" is the same as a message without a Subject. If the nature of the message changes during the reply, change the Subject.
- Check the mailing list carefully to confirm that you are sending the message to everyone you wanted, and that you are not sending it to someone to whom it should not be sent.
- Carefully select the language and style of the message according to the nature of the content and the list of recipients (think of Cc recipients as well).
- When sending a reply, consider whether it is necessary to enclose the original message (if your reply is "Yes", it is probably necessary) and whether it is necessary to enclose it in its entirety. Especially if the original message is long and you only respond to a few sentences from it, consider copying only the relevant sentences into your answer.
- Respect the rights of other participants in the discussion. You are not formally entitled to forward a text or image of another author to a third party without the author's consent. Of course, you won't worry about this among a group of friends, but when the communication is more official, you have to consider it.
- If you are going to send a larger file to multiple recipients, consider whether really almost all of them need it. Otherwise, save the file in a convenient location (your website, Dropbox, etc.) and send only a link.
- Even if your message does not contain a large file, but has many recipients, carefully consider whether you really want to send it to everyone. A separate chapter is, of course, the case of chain messages.
- Review the content of your message and adjust the format accordingly. A frightening case is when someone creates a document (e.g. in Word) containing two sentences and attaches it as a file to a message, instead of writing the two sentences directly in the message itself. Current e-mail applications approach the problem similarly and often send a message as an HTML document even when it is not absolutely necessary. This can be a problem for some recipients, especially if the mailer sends a multipart/alternative MIME document, which means that the message contains two interchangeable variants, one of which is of type text/html and the other of which is text/plain, which is empty!
- Sign the message.

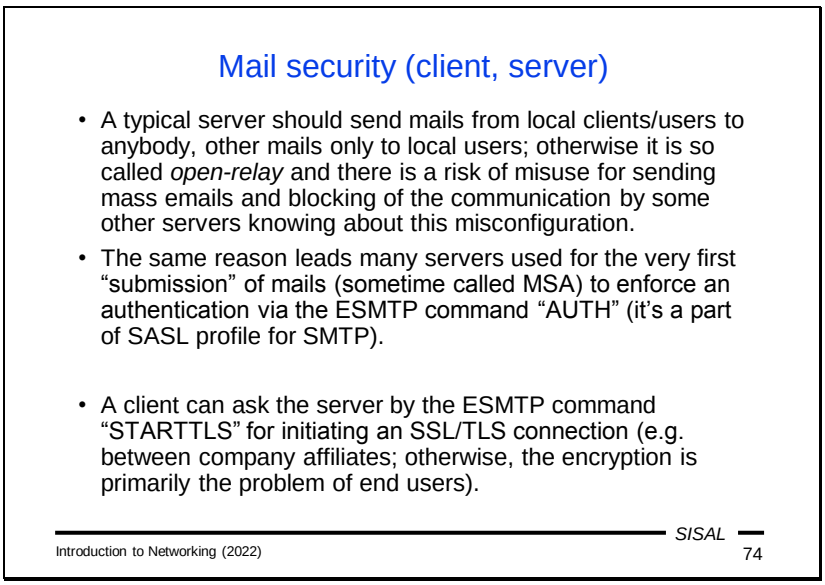
Slide 73



An email user should keep in mind that the content of a message is not protected during transport and due to technical errors or an attack, it may get into the hands of people for whom it was not intended. Not to mention the possibility that the sender may make an error when typing the recipient's address. The sender should adapt the content of the message accordingly. And if the content of the message is confidential, the user should use some encryption system.

It is also clear that thanks to the open mail protocol, anyone can write anything into the envelope or into the message. The recipient is never sure who the real sender is, and he/she should never blindly believe what is written in the message. As we have already said, if it is not serious, the recipient can probably estimate the authenticity according to the style of the text, but it is necessary to be careful. If a friend sends me a file that we talked about at lunch the day before, I'll probably believe it's from him. However, if he sends a file without notifying me about it and if I cannot tell from the text that it is genuine, I will **definitely not open** the file until, for example, I call him.

Mail servers intended for receiving mails for end users attempt to prevent mails from an insecure sender (for example, by trying to send a DSN message to that sender), but such protection is not entirely reliable. Some automated systems use the well-known challenge/response method, but only an electronic signature provides fully reliable protection.

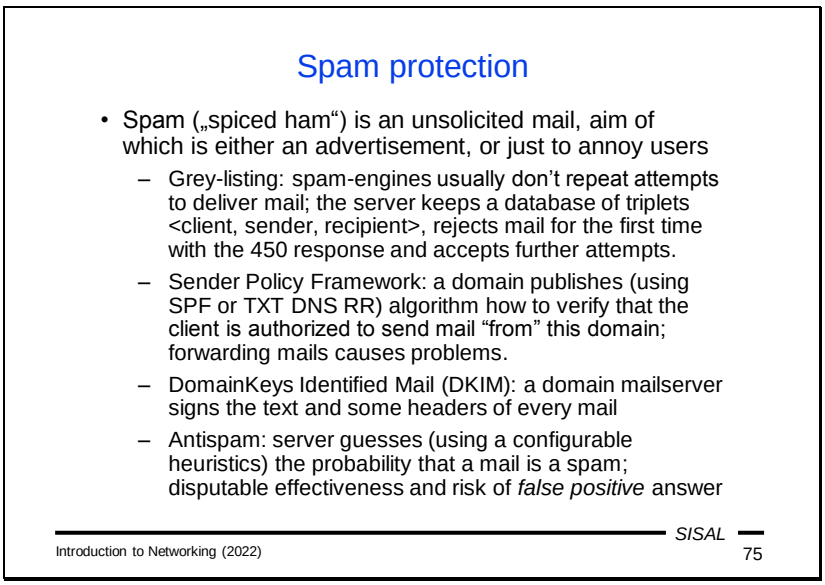


A properly configured mail server should distinguish between mails sent by „its" users, which it should deliver without restriction, and mails coming "from the outside", which should only be accepted if they are addressed to some of „its" users. If the server allows anyone to connect and send a mail anywhere, it is called an *open-relay server* and runs the risk of being misused to send bulk mail. Spammers try to cover their tracks by forwarding mail via several open-relay servers, so finding the real originator is complicated. Conversely, there are organizations that scan servers around the world, look for those that are open-relay, and compile a list of them that mail server administrators can use to block the receipt of mails from any of the open-relay servers.

The problem occurs when one of the server's own users needs to connect to his mail server "from the outside", because he is just travelling outside the local network. From the server's point of view, it is a foreign client and sending the mail will be rejected. And the user has no way to prove his identity to the server, because the SMTP protocol does not use authentication by itself. However, there is an extension that the server can use (usually on a different port, 587) to force authentication.

Since the transmission of a message between the sender and the recipient is not a matter of a single connection, but the message is stored several times along the way, the question of securing the content of the message is entirely a matter for the end users. Therefore, the connection between MTAs is usually not encrypted in any way. However, there are exceptions – e.g. if we have two corporate mail servers connected by a public network, it makes sense to encrypt the connection between them so that all corporate mails do not have to be encrypted by their senders. The ESMTP extension includes a STARTTLS command that asks the server to start encryption using the Transport Layer Security interlayer.

Slide 75



Spam has become a very unpleasant part of electronic mail communications. Protection against it is difficult because any restriction will partly affect real messages sent by real users.

The **grey-listing** method uses protocol properties and the experience that spam machines do not check delivery success (there are certainly a number of invalid addresses among a huge number of recipients' addresses, but they form an insignificant part of the overall effect, so it makes no sense for spammers to detect and repeat unsuccessful delivery). The server will therefore respond to the first attempt to deliver a message to a recipient, from a sender, and coming from an IP address with the 450 response to the RCPT TO command. If it was a genuine message, the client will attempt a new delivery after a certain time (typically 15 minutes), the server will discover that there has been a repeated attempt, and the new request will be answered normally with a 250 response and the message will be delivered. At the same time, it will mark the time of the last delivery attempt on a given triplet, and for some time (e.g. 5 weeks) it will pass all subsequent messages without a delay while each subsequent attempt shifts this time stamp. As a result, if two partners exchange at least one message a month, their communication is carried out without delay, except for the 15-minute delay of the first message. If, on the other hand, there is no repeated attempt after the first one (e.g. within 1 hour), the triplet will be blacklisted for a period of time. It was the combination of black and white list ideas that gave the method its name.

Another way to solve the problem was the so-called Sender Policy Framework. The basic idea is simple: a domain defines a list of servers it uses to send mails, and no server in the world should accept mails whose sender is someone from that domain coming from a client not listed here. Relatively complex syntax rules were developed

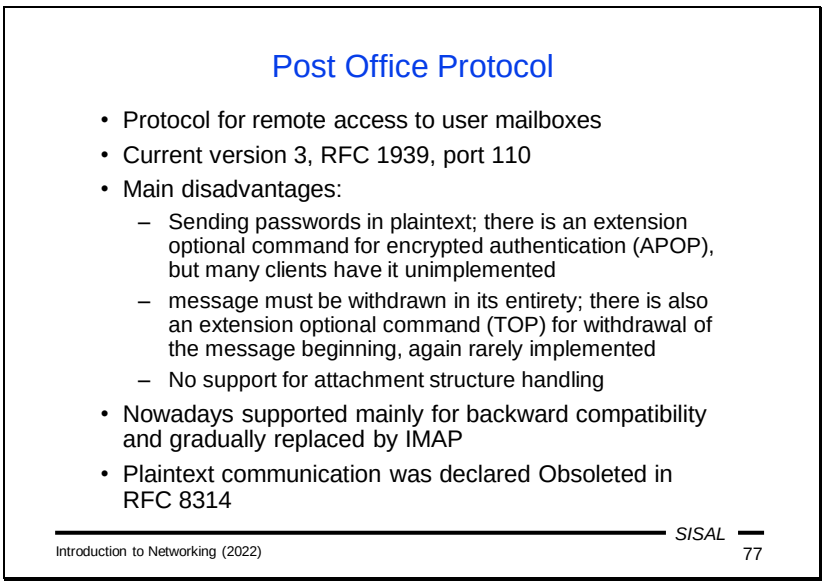
on how to define the correct sending MTAs, and these were formerly published using the DNS TXT record, later even with the newly created SPF record. But the catch to the principle is that once a message arrives somewhere where a user has set forwarding of incoming mails to a different location, the verification fails because the message with the original sender is suddenly being sent by a different machine! While correction mechanisms were established, the entire system proved to be inflexible, so that the DNS SPF type has been even withdrawn.

However, a more successful replacement was created, the so-called DomainKeys Identified Mail. The basic idea is the same; the difference is that a sending machine **digitally signs** all mails. More specifically, it signs the mail body and selected headers. The receiving server checks the signature and, as a result, either delivers the message to the destination mailbox or rejects it. Compared to SPF, this solution has the advantage that forwarding does not affect the signature in any way. All sending MTAs for a given domain can be equipped with a separate key, so that sending can be distributed to multiple machines and the list of them changed flexibly.

Various **spams blockers** are also a common means of protection. These are algorithms that attempt to estimate the probability that a message is spam. They use a number of attributes that a local administrator can give its own weights to. Attributes relate to both form (e.g. Subject in capital messages) and content (e.g. the frequency of some words). Based on the resulting score, the message can either be completely discarded, quarantined, or just tagged and delivered. Unfortunately, these algorithms are debatable in their effectiveness, because spam machines are increasingly adept at mimicking real messages, while strict rules increase the percentage of wrongly misclassified messages (so-called *false positive* outcomes).

Summary 4

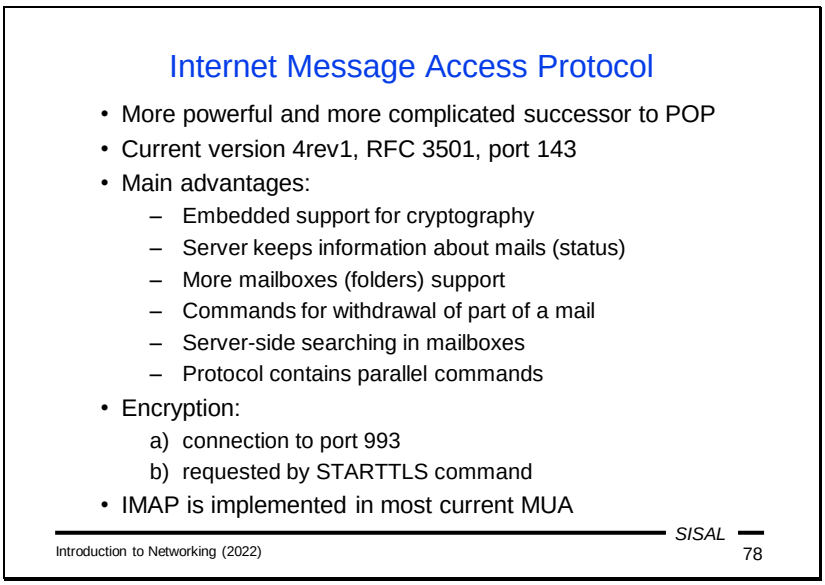
- What are the most important FTP security risks?
- What is the nature of the problems in opening additional data channels?
- Explain MTA and MX and their role in mail transmission.
- What is an “envelope” and how does it relate to the contents of mail headers?
- How can non-ASCII characters be used in mail headers and bodies?
- How do UUENCODE, Base64 and Quoted-Printable differ?
- What is the problem with open-relay servers?
- Explain the nature of grey-listing and DKIM.



Users can use either the POP or IMAP protocol to remotely access messages in their mailbox.

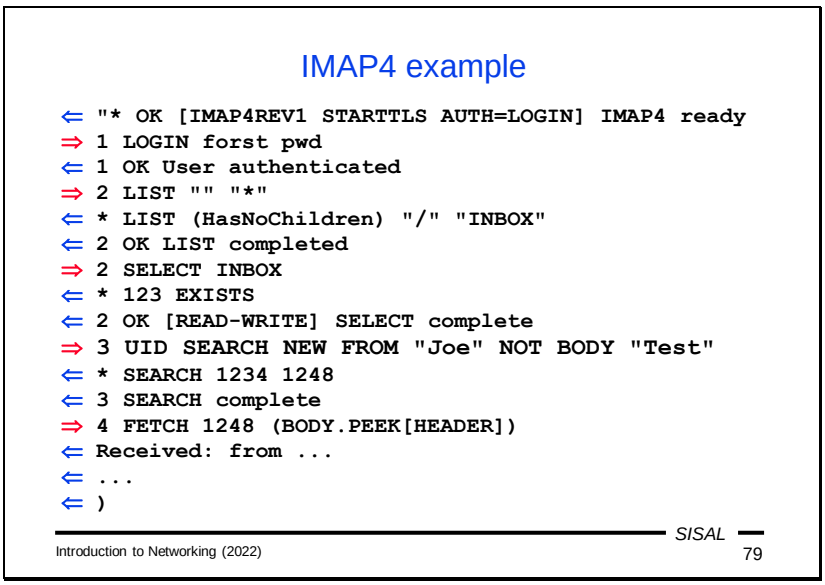
POP is the older of the two protocols and contains many known security concerns. Over time, there have been extensions that have tried to eliminate them, but in the meantime IMAP has gained more popularity, so implementing these extensions isn't common. Today, therefore, TLS security is more likely to be used, and in 2018 plaintext communication in the protocol was even declared Obsoleted.

Interestingly, the first two versions of POP used the *push* principle, that is, the server initiated the transfer of messages to the client. The most recent version now uses the *pull* principle. However, the main drawback of the protocol remains its limited capabilities for working with a mailbox.



Although the first version of the IMAP protocol appeared only two years after POP1, its author put more emphasis on working with a remote mailbox from the beginning, and IMAP2 included e.g. commands to search for a message in a mailbox according to several criteria, or to download only certain information about a message. The development of version 4 has been taken over by an IETF working group, and this version includes a number of security and functional extensions.

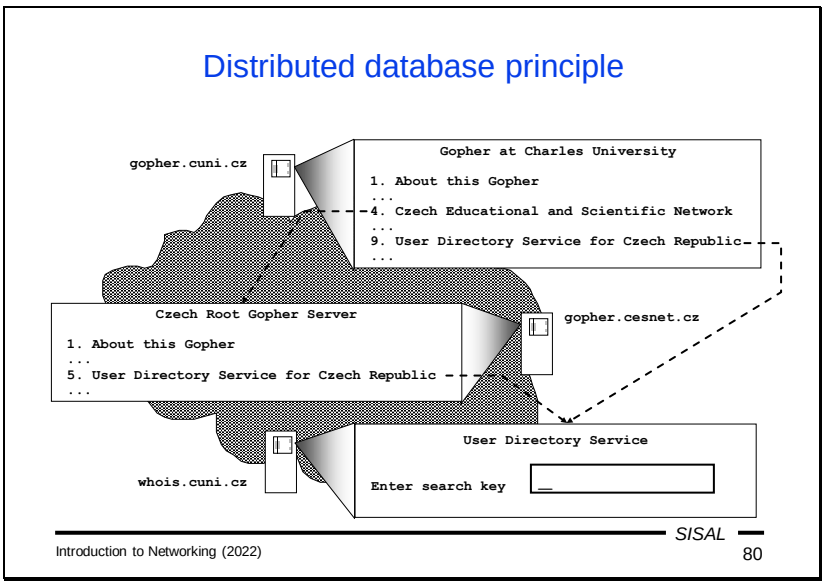
This version is now widely supported by mail applications and, in accordance with current standards, is used in combination with TLS, either on the dedicated port 993 instead of 143, or on demand by a STARTTLS command issued by the client.



The protocol is also text-based, but compared to POP, it is much more user-friendly. The client can use commands to work with various folders and subfolders, can search for letters according to complex conditions, download only parts of letters etc.

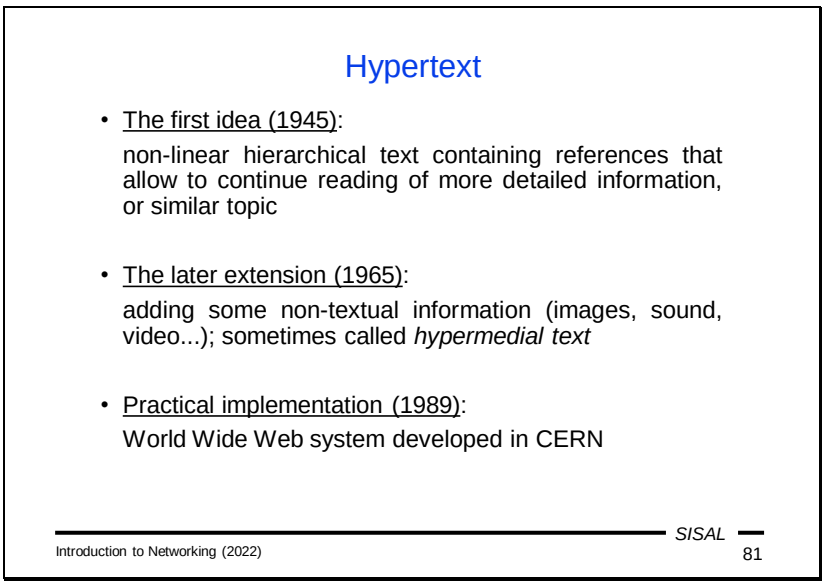
An interesting feature is also the tagging of commands and responses, which makes it easier to assign responses to sent commands. Thus, the client can send multiple commands to the server in parallel. The absence of this mechanism complicates e.g. in FTP the implementation of some operations, such as aborting a file transfer.

Slide 80



In 1991, the Gopher system appeared, the world's first distributed database service, a database of information that was stored on a huge number of servers and interconnected so that a user could navigate from one server to another without necessarily realizing it. At the time, Gopher was an information retrieval service with some of the same functionality as the web today. Even then, for example, all universities offered access through Gopher to study information, telephone directories, etc. When user connected to a Gopher server, a menu was displayed. After the user selected a menu item, another page appeared containing another menu, text with information the user requested, or a form which the user could fill out and submit to see some dynamically generated information. Links between individual pages could lead to a completely different server. As you can see, working with Gopher was similar to how users work with the web today. The only difference was that Gopher provided only textual information. If a link led to an image, Gopher downloaded it, but the user had to view it in another program.

Slide 81



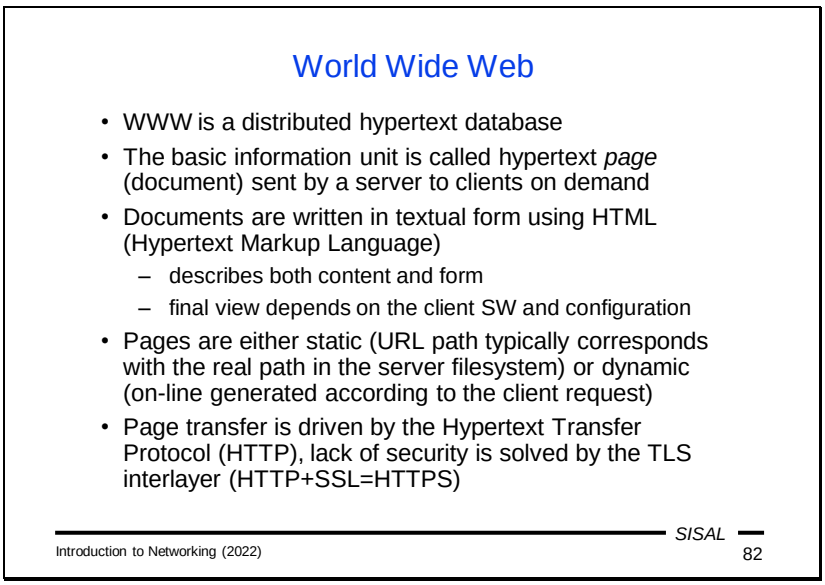
What Gopher lacked was hypertext.

The idea of hypertext is surprisingly very old. As early as the middle of the last century, at a time when printed materials were the only source of information, there were ideas that a text might contain links to explanatory or related texts. This would be somehow similar to an index, but available directly from the text, not as a summary at the end of the book. However, implementation at that time was of course not possible.

Twenty years later, the basic idea expanded to include the possibility of directly including non-text elements in the text, and an alternative name, *hypermedia* text, appeared. However, the idea still had to wait for the technology that could implement it.

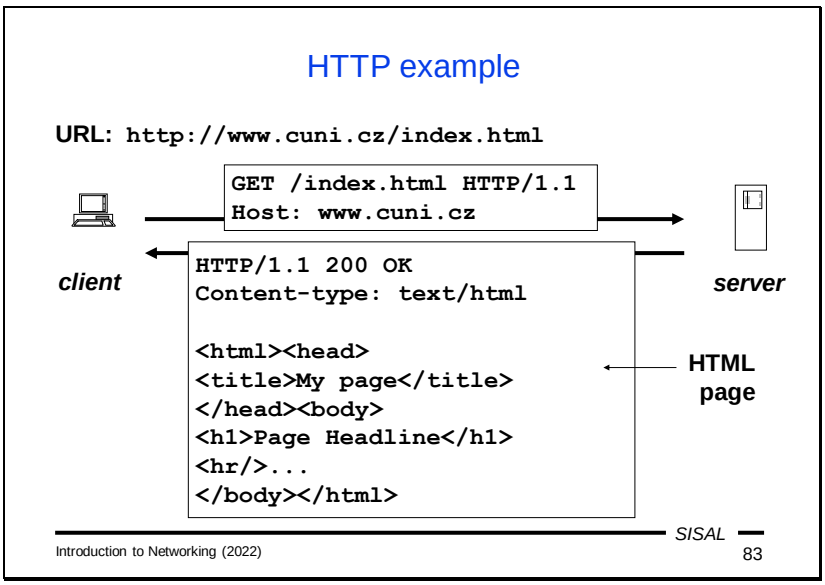
After another twenty years, more precisely in 1989, the idea of a system for disseminating information of various kinds was born at the Swiss center CERN using a service later called the World Wide Web.

Slide 82



Simply said, the web is a distributed hypertext database. As with Gopher, global information is distributed on a huge number of servers around the world and is interconnected so that you can browse between them without users having to worry about which server they are connected to.

The basic unit of information is a *page*. This term corresponds to what existed in Gopher before the web, but its meaning is quite different. A web page is sometimes referred to as a *document*, which better corresponds to the fact that a web page has nothing to do with a page in some printed material. A web page can be stored as a file on a server (then we call it *static*), or it can be generated *dynamically* by a server based on a parameterized user request. In both cases, the client sends a request to the server and the server responds with the text of the requested page (or an error message). Pages are usually written in HTML (Hypertext Markup Language), which contains means for expressing both content (i.e. text, embedded images, links, etc.) and form (font, color, tables, alignment, etc.). HTML describes an author's idea of how a page should be displayed on the client's screen, but the exact appearance of the displayed page is determined by many other factors (implementation of the language's properties in the browser, user settings, etc.).



The Hypertext Transport Protocol (HTTP) is used to transfer web pages. In its basic version (1.1) it is a text protocol in which

- the client establishes a connection to a server and sends a text request containing a URL (written partly on the first line which contains a command and partly in the headers that follow it) and some other parameters
- the server responds with a line containing a response code (200 in our example), headers containing (among other things) the MIME type of the requested document, and the document itself (in our case an HTML page). In case of an error, the response will contain a different completion code and the HTML text of an error page.

Hypertext Transfer Protocol v.1

- Currently still mostly version 1.1, RFC 7230, port 80
- General message form:
  - the first line (request/response)
  - additional header lines
    - request: language, charset, page age, authentication,...
    - response: document type, encoding, expiration,...
  - (optional) document body
- Status (response) codes:
  1xx **informational** (provisional response, processing continues)
  2xx **success** (final response)
  3xx **redirection** (some additional client request expected)
  4xx **client error** (incorrect request)
  5xx **server error** (transient or permanent error on server)

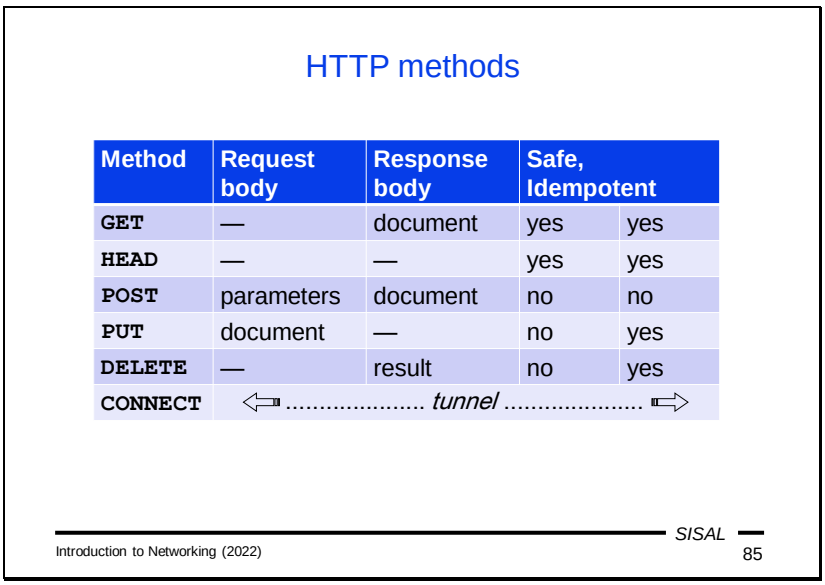
Currently, version 1.1 of the protocol is most widely used. This version is text-based; the request and response contain an initial line, usually followed by a series of header lines (with basically the same format as mail headers), a blank line, and in some situations can be followed by the body of a document.

In the request, the initial line consists of a method name (e.g. GET for requesting a page download), a path (i.e. the part of the URL after the server name) and the protocol version. Of the headers, in version 1.1 only the Host header is mandatory, which specifies which server the client is connecting to. This is because multiple servers can run at the same IP address, and it is this header that distinguishes which server a request is addressing. Other useful headers include the language and encodings that the client accepts, the maximum allowed page age, authentication data, etc. The body will not be present in a normal request, with the exception of special types of requests where either additional parameters are sent to the server or an entire document is uploaded.

In the server response, the first line is a status line, again containing the protocol number, a three-digit response code, and a verbal description of the response. Headers relate to formalities (such as the time of the last change or the expiration time of a page), details of the protocol (such as the method of transmission), and the properties of the document being sent (such as the MIME header). The body of the response contains the required document or the text of an error message.

The response codes basically correspond to what we know from previous protocols. 2xx is a final positive reply, 1xx and 3xx mean that the request is more or less OK; 1xx means that the ball is in the server's court (another answer is coming in a while), while 3xx plays the ball onto the client side (the client must enter another URL, if it

wants to get the page content). Only error response codes have semantics that slightly changed from other protocols; 4xx means an error on the client side (bad URL, authentication, etc.), while 5xx means an error on the server side.



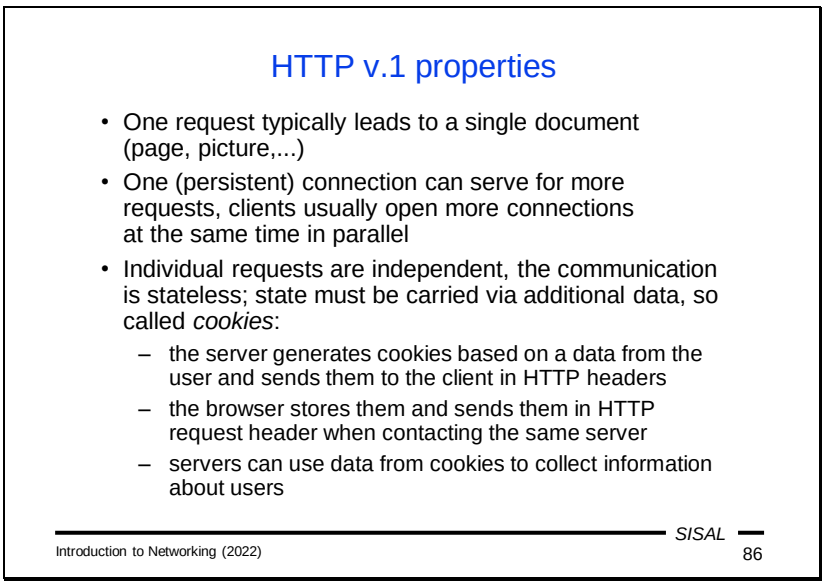
The most common HTTP methods are GET, HEAD, POST, PUT, DELETE, and CONNECT.

- GET is the basic method by which a client requests a page or document. The request body is empty; the response body contains the requested page. In the case of an error, the body of the response (as with other methods) usually contains the text of an HTML page describing the error. This method is declared **safe** in the RFC specification, which means that it **does not change** the content of the server. Even if a request was created by filling out and submitting an HTML form and it contains the form data in the query section of the URL, it should not cause the data stored on the server to change. This method is also defined as *idempotent*, meaning that **repeated** use has the **same** effect.
- HEAD is a simplified form of the GET method, where the server replies only with headers and not a document. Its purpose is to find out whether the requested page is available, how large it is, whether it is available in a certain language, etc.
- POST is a method whose purpose is to send certain information (usually data from a form) to a server and get the content of a certain document in response. It can be used as a way to get a dynamic page, but it is also valid to **change** the content of the data on the server by using this method. Therefore, the method is not idempotent, because each call can have a different effect (for example, the content of a file on the server could expand with each call).
- PUT is a method that overwrites the content of a document on the server with the document sent in the request. So it is definitely not safe, but it is idempotent – repeating the request should still have the same effect.
- DELETE is a method for deleting a document on the server. This method is also idempotent – when repeated we get another message (saying that the document

has already been deleted), but the result is in any case the non-existence of the document.

- CONNECT is a specific method which opens a connection according to the specified parameters and thus actually builds a tunnel through which it is possible to forward another connection in a completely different protocol via an HTTP connection. While this is a useful feature that web servers often use, it also poses a security risk because it allows you to bypass network security policy restrictions.

Slide 86

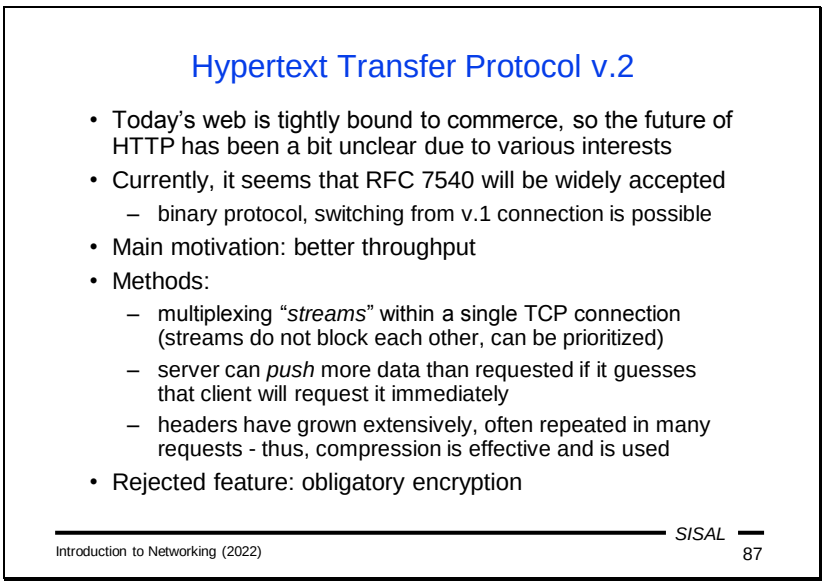


The way a web browser is used leads some users to think that when they visit a server, a communication channel is established between the client and the server, which transmits data back and forth for the entire duration if their visit to the server. But this is completely at odds with reality. In fact, each user request is sent by the client as completely **independent**. If a web page consists of text and three images, then there will even be four independent requests. Clicking on a link on the page will generate several more independent HTTP requests. Version 1.1 introduced a new feature, a **persistent** connection, which means that after one request, the client does not have to close the TCP connection and establish a new one, but can use the same TCP connection for multiple serial requests. In addition, browsers often manage this by opening multiple parallel TCP connections directly when connecting to a server, as they expect most pages to contain multiple components from the same server, so it makes sense to have appropriate TCP connections ready for subsequent requests.

The fact that the individual requests are independent means that the entire communication is **stateless**, or that the server generally has no idea of which requests from the same client "belong together" or about the current state of the interaction with a given user. If a user sends a server some information that will be needed for its further work with that user (e.g. some settings), the server has to receive it again from the client on each request. The problem of stateless communication is solved using so-called *cookies*. A cookie is a piece of data that the server generates based on information from the user and sends to the client when responding, in the form of Set-Cookie (or Set-Cookie2) headers. The browser saves this data and then sends it in the form of a Cookie header each time another request is submitted to the same server. A cookie lets a server identify the connection and/or user and ensure that the user's request is processed in a server environment that matches their existing (or previous) interaction.

This nature of cookies is important because it has several security implications:

- Cookies themselves do not pose any direct risk; they cannot, for example, contain viruses, etc.
- However, they pose a secondary risk, because a web server can store any information it finds about the user in them and then use it at will. For example, with the help of advertising banners placed on various pages, a server can collect comprehensive information about a user and carry out personalized advertising.
- There is also a risk if cookies stored on your computer fall into foreign hands.



The newest version of the HTTP protocol tries to capture current trends in web usage in order to increase "speed" from the user's perspective if possible. A fundamental difference is the change from a textual form to a binary one. This is, of course, annoying for many reasons: with version 1, a more experienced user appreciates that he can try to contact a server directly, enter his request and see exactly how the server responds, and administrators appreciate the ability to monitor any malicious activity of users or servers. However, this idyllic age of HTTP transparency did not actually end with HTTPv2, but rather with the massive adoption of HTTPS. At the transport layer, we now rarely see HTTP in text form, so this change to a binary protocol is not really that crucial from an operational point of view.

The authors of version 2 were not satisfied with the services of TCP and created the concept of multiple HTTP *streams* running over a single TCP connection. These custom streams are more efficient because they do not block each other and can be prioritized according to the needs of the application layer.

In version 2, the server can also send a client data that it has not yet requested, but that the server expects that the client will need in a while. A classic example is a page with images – in version 1 we learned that the client must create a new request for each image, but in version 2 it is not necessary.

Another feature of the new version of the protocol is the ability to compress headers. Especially with the increased use of authentication, the headers have also swelled a lot, moreover, they also often have the same content (authentication, offered languages, coding; all these will not change practically throughout the entire dialogue and it is unnecessary to send everything again with each request).

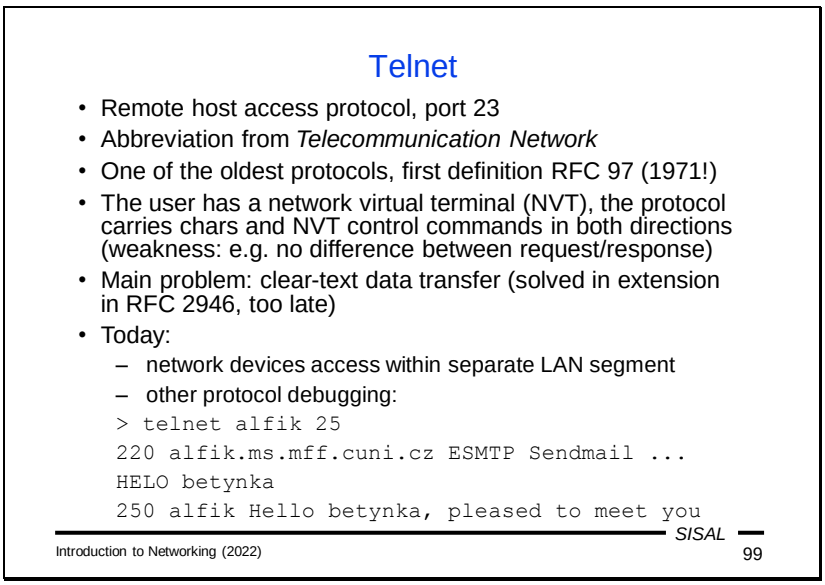
Hypertext Markup Language

- Progress in past years has been a little bit dramatic, 2014 released a compromise version 5
- Textual page content is supplemented by meta-tags: structural (e.g. paragraph), semantic (e.g. post address), formatting (e.g. bold)
- Application of older SGML (Standard Generalized ML) and predecessor to XML (Extensible ML)

- Tag form: **<***tag* [*attributes*]**>**
- Whitespaces not significant
- Special chars - entities (**`<`**, **`>`**, **`&`**, **` `**...)
- Comments (**`<!-- ... -->`**)

The curriculum on HTML and related topics will be taught in the second part of the course.

Slide 99



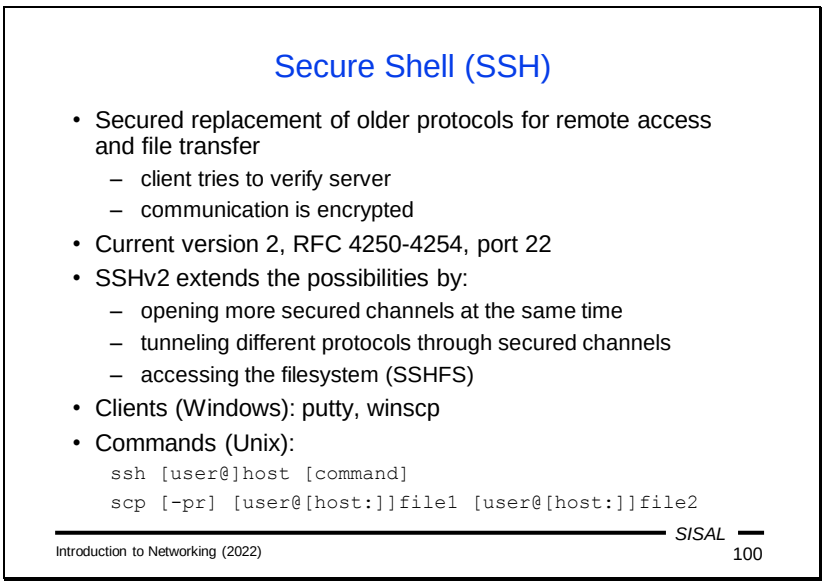
Telnet is another very old protocol and represents the first solution for remote login to another machine, in the form of terminal emulation. The client and server transmit user commands and host reactions so that the user has an environment similar to sitting at a real terminal.

An educational element of this communication is the way in which control commands are sent. The principle of terminal emulation is based on the fact that the client and the server must agree on which part of the work each one will do. When the user presses a key, someone must cause it to be displayed on the screen (a so-called echo). The client can either do it itself or wait until the server sends back an instruction to display the character after it as processed the keystroke. Conversely, if the user is typing a password, neither the client nor the server will echo. But they have to agree on that. There are four messages DO ECHO, DON'T ECHO, WILL ECHO and WON'T ECHO. The problem is that the message **does not indicate** whether it is a command or a response! So if the meaning of messages is accidentally out of sync, it has fatal consequences. Let's say a client sends a command DO ECHO, and because no response is received, it repeats it, but does not note that it was sent twice. The server correctly responds twice WILL ECHO, but the client will consider the second response as a new message and respond to it DO ECHO. This starts an endless loop of extremely fast DO ECHO / WILL ECHO message exchanges. The lesson is not to save money at all cost in the protocol planning phase at all costs and definitely sacrifice one bit per request/response flag.

From a security point of view, Telnet has all the weaknesses of the old Internet protocols. Open password transmission was solved by RFC 2946, but it came at a time when there was already a better replacement, SSH. However, even today, due to its relative simplicity, the protocol is used in places where there is no risk of

password interception – for example, on a separate segment of a local network intended for infrastructure management. However, even these devices are switching to SSH over time.

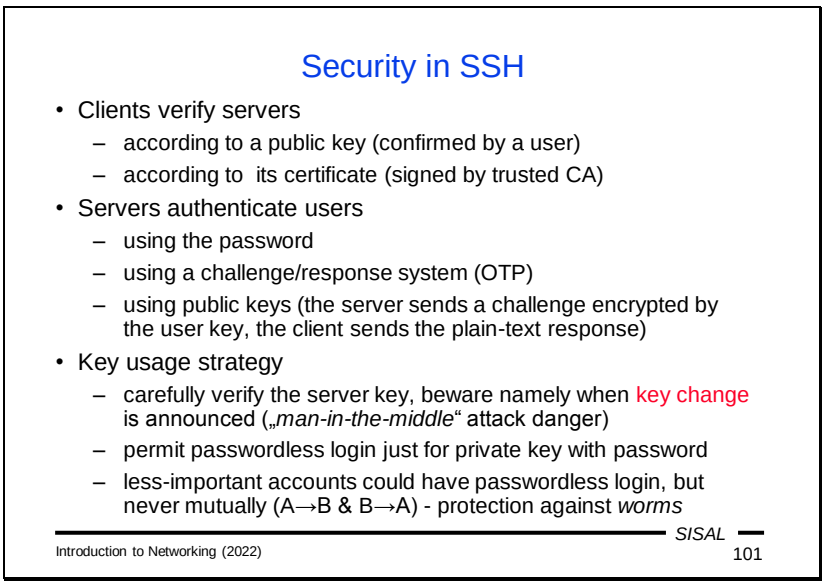
However, the **telnet** application can still be used for another purpose, namely to establish a connection to a server working in another (ideally text-based) protocol. For example, we can test how an SMTP or HTTP server responds to individual commands.



SSH (Secure shell) is a protocol used for remote login and file transfers. Unlike older protocols, it is based on consistent authentication of the server to which we connect and encryption of the entire communication.

Currently, version 2 is used, which expands the capabilities of SSH with the following:
- opening parallel channels; it is possible, for example, to be logged in using a virtual terminal and to transfer files at the same time,
- tunneling; communication on one side of the SSH channel is transmitted through the channel and on the other side is directed to a local server (a way how to bypass a firewall),
- SSHFS; the server makes part of its file system available through the channel so that it appears to the client as a local file system.

On the MS Windows platform, freely available programs for remote login (e.g. **putty**) and file transfer (e.g. **WinSCP**) are available; on UNIX there are line commands **ssh** and **scp** for this purpose.



Before a client starts communicating with the server about user authentication, it verifies that it is actually connected to the correct server. Either a server certificate is used, or the client only displays the fingerprint of the server key to the user and the user decides whether it is the correct key and therefore the correct server. If the user confirms the identity of the server, the application saves this decision and does not ask again at the next login.

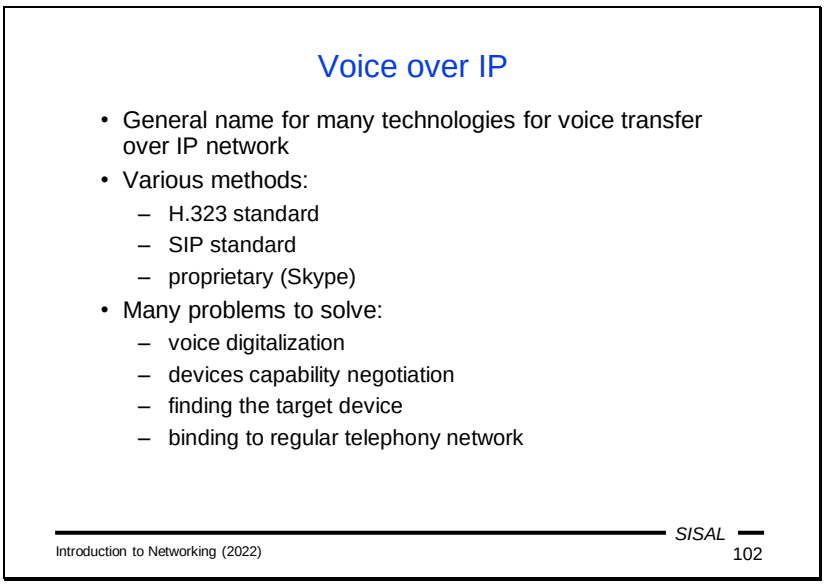
If we are logging in to a server for the first time using SSH, theoretically we should always thoroughly verify the correctness of the key that the server sends. In fact, the risk is normally very low. We must evaluate how important the server is, how likely it is to be attacked, how secret the password is that we have on that server (on how many other servers do we have the same password), and if we evaluate these risks as small, verification can be skipped. However, this is definitely **not the case** if the message about the key change on the server arrives **unexpectedly**. If the server we have been logging in to for a year suddenly uses a different key, it is necessary to pay attention and thoroughly verify everything! This could be a symptom of an attack. For example, we can verify with the server administrators by phone that they have really reinstalled the system and have a new key. If such verification is not possible, we should **terminate** logging in.

Once the server is verified, the client begins exchanging information with it to authenticate the user. The most basic option is to use a name and password. At this point, the communication is already encrypted, so there is no risk of disclosure. The second common way is to use keys. The user creates a public/private key pair on the client (or can share the same key pair on multiple clients), and stores the public key in a specified location on the server. If such a key exists and the client can prove that

he holds the corresponding private key, the user does not have to enter the password for the remote account.

Setting up logging with a key is very convenient, but it carries a risk. If our account is attacked, we open the door to other servers. Many people use a two-tier key system: they use one pair of keys on machines of low importance and another pair on important machines, while the secret key from the latter pair is protected by a local password. If the key is not password protected, an attacker can use it to log onto other machines. Theoretically, the attacker has no information about what other machine he should try to log into. However, he has information about other computers **from which** you are allowed to access this account (just going through the list of stored public keys). So he can try to log into each account from this list, in case you also happen to have reciprocally set up login without a password there. And if so, he'll get to another computer. This is the working principle of Internet **worms**. Protection against this kind of attack is simple – avoid having logins enabled between your two accounts back and forth, and if you really need to have it set up like this, password protect the secret keys.

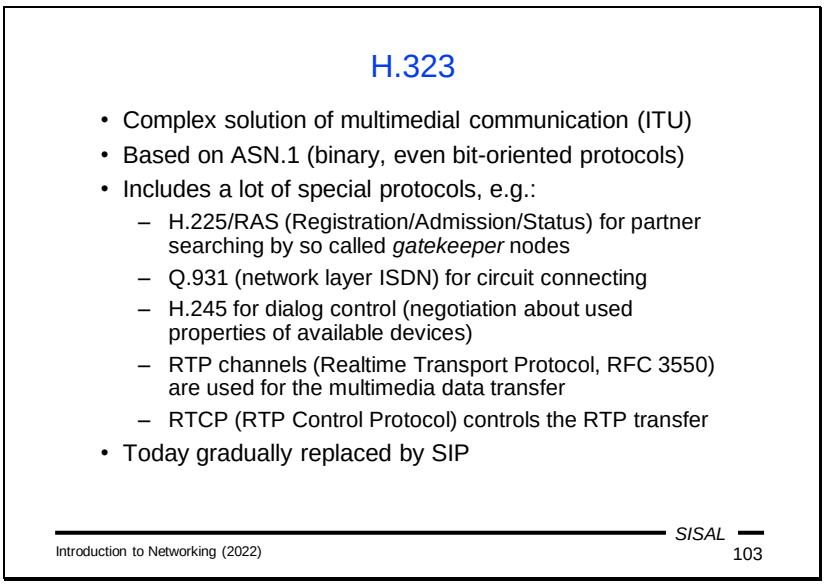
Slide 102



The term **Voice over IP** does not refer to one specific protocol, but generally to any tool for transmitting (not only) voice over a TCP/IP network. There are a number of options for such a transmission, some of which are implemented through more general protocols (such as Skype over HTTP), but we will be mainly interested in those that use their own protocols.

This whole issue is of course very complex. It includes the area of voice digitization with all its methods, variants, parameters, etc., it includes how the counterparties agree on the communication parameters for both sides and how to adapt the behavior of one counterparty to another, there must be some way to find a partner for communication and ideally to interconnect the entire system to a regular telephone network so that it is possible to communicate from a regular telephone to a number represented by a VoIP device and vice versa.

Slide 103



Telecommunication companies (or ITU, International Telegraph Union) long ago embarked on a path to digitize individual processes. There are a number of separate protocols for each area, collectively referred to as **H.323**, which address the issues of searching of a partner, connection establishment, agreement on device properties, and finally the actual transmission of audio or video data. Unfortunately, these protocols are not all freely available and come from a time when data flows did not have the capacity we have today, and therefore they were designed to desperately save every bit. Yes, these protocols are not only non-textual, but are **binary** in the true sense of the word. The boundaries of the values here are not bytes, but **bits**, and in addition, each bit can affect the boundaries of other values. The protocols are not only unreadable by a text editor, but even very difficult to implement – imagine how you would program the reading of a ten-bit number that spans three bytes, but only sometimes…

The protocols in charge of the initial phase of call preparation are now gradually being replaced by the SIP protocol. The audio or video data itself is sent using the Realtime Transport Protocol (RTP) and the RTCP protocol is used to control its operation. Although both protocols are also binary this is adequate for their purpose, since their task is to transmit binary multimedia data; in addition, their description is freely available in RFC 3550.

Abstract Syntax Notation 1

- Formal definition of data structures, e.g.:

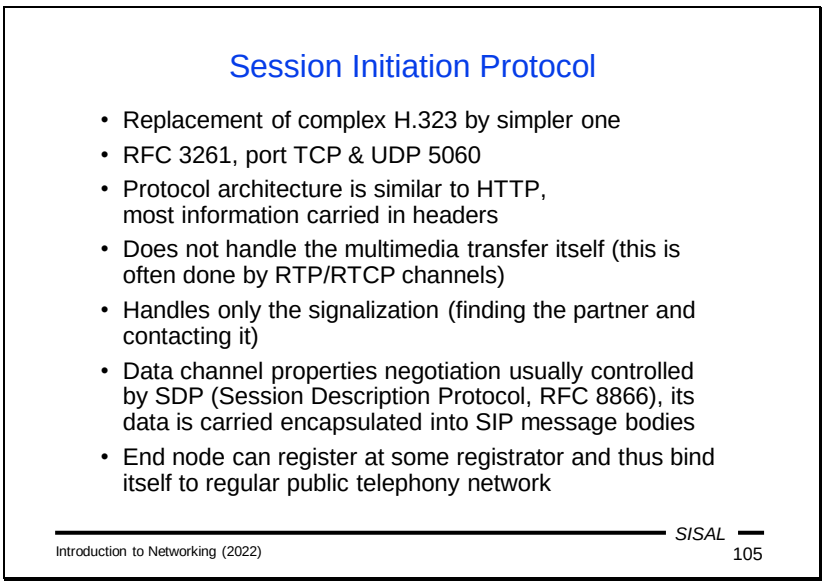
```
Answer ::= CHOICE {
   word PrintableString,
   flag BOOLEAN }

AlgorithmIdentifiers ::= SET OF AlgorithmIdentifier

SignedData ::= SEQUENCE {
   version CMSVersion,
   digestAlgorithms AlgorithmIdentifiers,
```

- Comes from the 80's (and implementation looks like this)
  - e.g.: enumeration value is by default stored into as many **bits** as needed, preceded by a bit signaling an extension, meaning a **different** number of bits used for this value
- Automatic generation of protocol parser possible
- Usage examples: H.323, X.509

The definition of H.323 protocols is based on a method called Abstract Syntax Notation. It is a method of defining a data structure or content of protocol data units using a formal definition with mnemonic identifiers (including enumerations, sets of named values) and special constructs to express a sequence (record), an array, selection from several alternatives (something like case or select constructs from programming languages), etc. As a means of formal description, it is a very useful tool.

The problem is its default implementation. As I have already indicated, each value is written only as many bits as needed for the item. However, the authors anticipated that protocols were evolving and built into the design a general mechanism for indicating whether each value uses the original design or its extension. Thus, each actual value is preceded by one bit, which tells whether the value is extended or not, and thus actually determines how many bits it actually occupies… The implementation of such an encoding is extremely complex and is solved by purchasing a library that converts a text notation in ASN.1 to source code that implements writing and reading it.

In addition to this extremely economical variant of value encoding, there are also variants that at least respect the byte boundaries, so that their decoding is much easier.

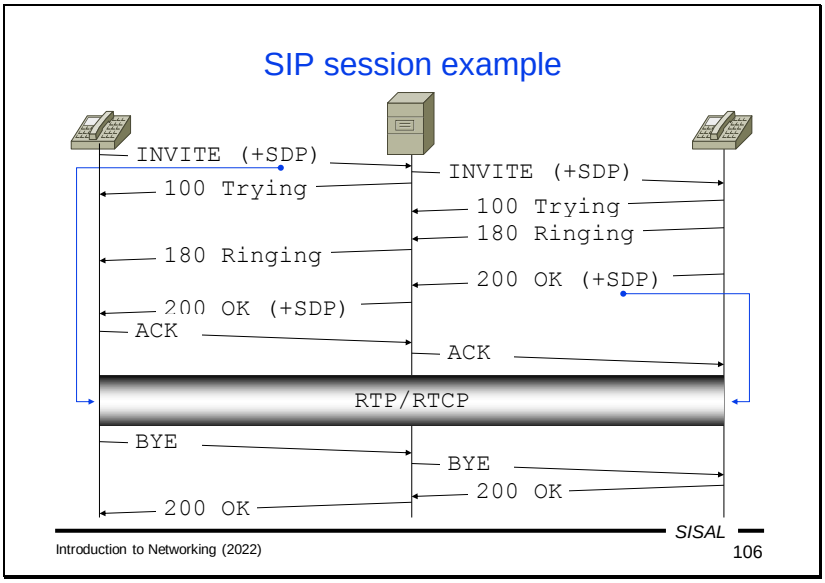


Session Initiation Protocol (SIP) is a more modern solution for tasks related to establishing connections and negotiating device properties. Unlike the H.323 family of protocols, it is a text protocol with a message structure that resembles HTTP, but can run on both TCP and UDP. The advantage is that the protocol messages are simply readable. Today, this protocol is gradually replacing H.323 and is becoming the standard for building a telephone network in newly designed buildings.

It is interesting that this protocol is one of the first that already in the initial proposal introduces the term **proxy**, i.e. a node in the communication chain that facilitates communication across the boundaries of various networks, including private ones. As with SMTP, special headers are inserted into the message during transmission that record the path (here they are called **Via** and **Record-Route**) and according to which the other party and all nodes along the path can correctly route the response. Likewise, the headers include data such as a call identifier, the caller and the called SIP URL (usually a number plus a domain), etc.

SIP itself only solves the problem of finding a destination, finding a route and establishing a connection. The next stage, i.e. the agreement on device properties and data channel parameters, is solved by the Session Description Protocol (SDP). It is again a text protocol consisting of lines of the format *keyword*=*value* and is transmitted using SIP protocol messages. A SIP message carrying SDP information has the MIME type **application/sdp** and there is an SDP block in the body of the message.

As soon as the counterparties agree on the form of the data channels and open them, they will start sending audio (and possibly also video) data via them, using the RTP/RTCP protocols.



An SIP call example:

- The calling device sends an INVITE command, which contains the URL being called and an offer of data channels as an SDP message.
- The command arrives at the nearest proxy, which begins to resolve the target. Depending on its configuration and the destination URL, it will find another node in the path to the device being called. For simplicity, we will imagine a situation where the target network is directly behind the proxy. In this case, the proxy sends the request to the target device, but checks the contents of the SDP message and modifies it so that it is usable for the partner – it de facto makes adjustments in it corresponding to the address translation (NAT) function.
- At the same time, the proxy sends a temporary 100 Trying response to the calling device.
- Once the called device processes the request, it also sends a temporary response 100 Trying. This response is a *node-to-node* message and therefore the proxy does not forward it to the calling device.
- The called device starts ringing and informs the calling device with another temporary response 180 Ringing. Unlike the previous response, it has an *end-to-end* character, so it is forwarded by the proxy to the calling device.
- If the called party picks up the phone, the called device will send a final answer 200 OK. This response also carries an SDP message with an offer of data channels on the called device.
- When this message arrives at the proxy, it checks the contents of the SDP again, adjusts it to address translation needs, and sends the message to the calling device.
- To complete the initial phase, it is still necessary for the calling device to acknowledge receipt of the SDP message, and therefore the entire phase ends by sending an acknowledgement message (command) ACK.

- From this point on, both devices can start sending multimedia data over RTP/RTCP data channels offered to them by the other party.
- The call is terminated by any end device by sending a BYE command and receiving the 200 OK answer. Of course, the proxy also responds to this pair of messages and cancels its data channels, which it has opened toward both parties.

Summary 5

- What is the purpose of the IMAP protocol?
- What is the purpose of HTML?
- Describe what happens in the HTTP/1.1 protocol if a user requests a page with two embedded images.
- Briefly describe the purpose and properties of the most common HTTP methods.
- What are cookies for and what are their risks?
- Compare the Telnet and SSH protocols.
- Compare the H.323 and SIP protocols.

Filesystem sharing

- Connecting a foreign filesystem transparently into local one
- Network File System (NFS)
  - originally developed at Sun Microsystems, today IETF
  - current version 4.1, RFC 8881, port 2049 (UDP, TCP)
  - source identification: server:path
  - authentication: Kerberos
  - note: relation (RPC) and presentation (XDR) layer
- Server Message Block (SMB)
  - originally developed at IBM, later adopted by Microsoft
  - open implementation Samba (UNIX)
  - source identification: UNC (\\server_name\source_name)
  - authentication: usually username and password

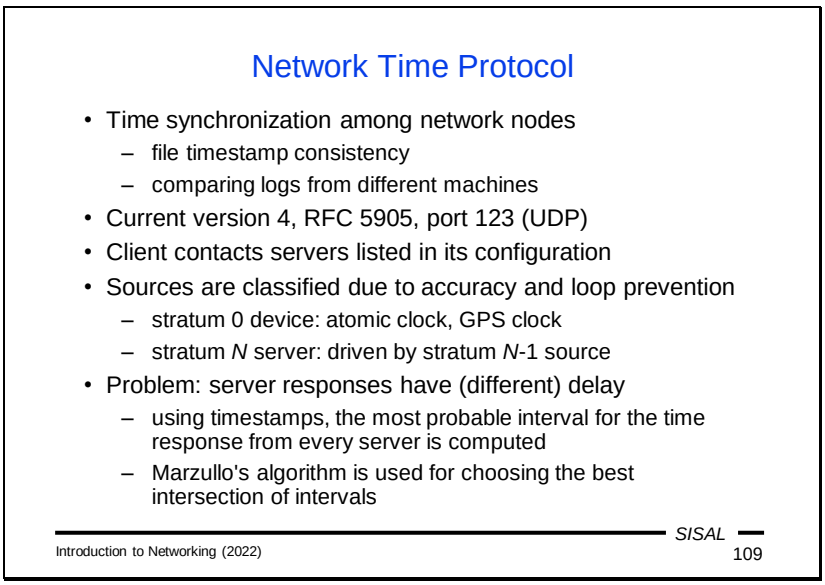
The last group of application protocols we will talk about are protocols that usually remain somewhat hidden from users.

A very important communication tool is the ability to connect a disk from another computer to your computer and work with it as if it were a local disk. The most used technologies for this purpose include NFS and SMB.

The Network File System protocol was born at Sun Microsystems, but over time it has become an open protocol described as RFC. It is still one of the most widely used systems of its kind in the UNIX world, unless the local environment requires a more sophisticated file permission system than the traditional UNIX system (in which case NFS is replaced e.g. by AFS, Andrew File System). NFS refers to a mounted disk as a *server*: *path* pair. For users this connection is transparent and they see a mounted disk as part of the local directory tree. Kerberos is usually used as the authentication mechanism. The NFS protocol works over UDP by default, although it can also be run over TCP. An interesting fact is that the internal structure of the protocol copies the OSI model in more detail and we can isolate the relational layer (Remote Procedure Call, RPC) and the presentation layer (Exchange Data Representation, XDR). RPC is a general mechanism used by other services – a client sends a request to a server to call a function, including a list of arguments, and the server performs the operation and sends a response to the client.

The Server Message Block protocol also comes from the commercial sphere, specifically from IBM. Its development was later taken over by Microsoft, so it was not published in the form of an RFC. However, an effort to connect the world of UNIX and Microsoft Windows led the Samba project to reverse engineer this protocol and their free implementation of SMB actually enabled such a connection: clients from both

families of systems can mount a disk from servers running on any system. A disk that is mounted via SMB is referred to using a notation *server**path*; authentication is performed by the system itself using a username and password.

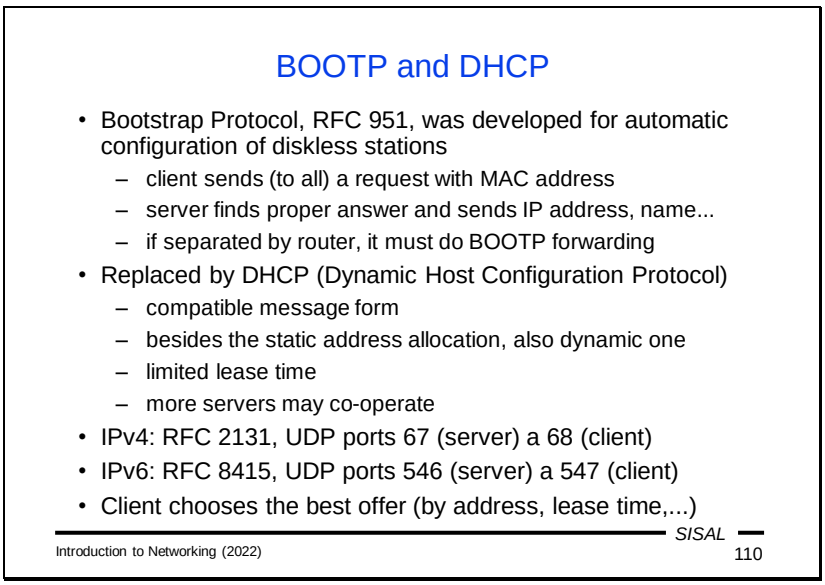


A very important feature of a local network is the synchronization of time on individual nodes. It is not necessary for the proper functioning of a network – if that were the case, then any desynchronization, which can occur very easily, would lead to a network breakdown. However, time synchronization is essential from the user's point of view, especially in two situations:

- If users transfer files between network nodes (and especially when sharing disks), it is essential that both computers have the same time. Otherwise, a file stored on one computer will be considered out of date on the other. Such a discrepancy could cause e.g. repeated update attempts or repeated unnecessary compilations of modules that have already been compiled modules, etc.
- Network administrators very often solve problems by comparing log records from different computers. If these records are written on machines with asynchronous clocks, it is a nightmare if for each line, the administrator must think about how many minutes he must add or subtract to which time to reconstruct the correct sequence.

One of the protocols that perform synchronization is the Network Time Protocol. The basis of the protocol is the fact that somewhere there are sources with absolutely accurate time (e.g. an atomic clock). These are called the source of *stratum 0*. From them, the time is taken by other servers (the source of *stratum 1*), which again serve as a source for the next level. The idea of marking a source according to its "logical distance" from an absolutely accurate source serves both to estimate the degree of accuracy of a particular source and as a protection against loops (the stratum *N* source will ignore the time from the stratum *N* + 1 source). A typical configuration for a LAN is based on running one or more NTP servers, they are synchronized with the NTP server(s) provided by the ISP, and all clients in the local network are synchronized using these servers.

Network latency must be taken into account when implementing the calculation algorithm. A client cannot simply take a time that it receives in a response verbatim because a certain amount of time elapses between sending a time and receiving it. The client calculation therefore uses timestamps from the response, which determine an interval in which the actual time probably lies. Using a technique called Marzullo's algorithm, these intervals are used for the calculation of the most accurate time with the highest possible probability. Since each client is not able to determine the time exactly, inaccuracy increases between the individual levels of NTP servers. From a practical point of view, however, these inaccuracies are fairly below the resolution of users.



The last technical application protocols we will deal with are BOOTP and DHCP. They are used by clients connecting to a network for obtaining an IP address that they can use and other information about the local network.

BOOTP, the Bootstrap Protocol was developed a long time ago, at a time when it was economically advantageous to acquire *diskless workstations*, i.e. computers that had no devices for permanent data storage (disks were relatively expensive at the time). It was not possible to save the configuration, including the IP address, anywhere on such a machine. The BOOTP protocol allowed each workstation to send an address assignment request, which the dedicated BOOTP server checked and possibly replied to. The server considered each request by verifying its MAC address against a whitelist – the MAC address was the only data that the client knew and that it could use for identification, because it was burned into its network card. If the client was on the whitelist, the server determined the IP address to be sent to it (the address assignment was fixed at that moment) and sent a response. Once the client received the response, it could start using the address.

From a technical point of view, it is important to mention that the client must send its request unaddressed, because it has no information about the addresses in the network where it is located. It will therefore use the so-called *limited broadcast* as the destination IP address, which means that the request will reach all nodes in the network (but they will ignore it). In order to prevent such requests from spreading meaninglessly throughout the Internet, routers do not pass them outside the network where they originated. This causes a small complication if we have a more complex subnet structure in the LAN, separated by routers. Then we must either have a BOOTP server in each subnet, or run so-called *BOOTP forwarding* on the routers,

where the BOOTP router forwards queries from the network where they originated to a certain BOOTP server and then forwards its response back to the client.

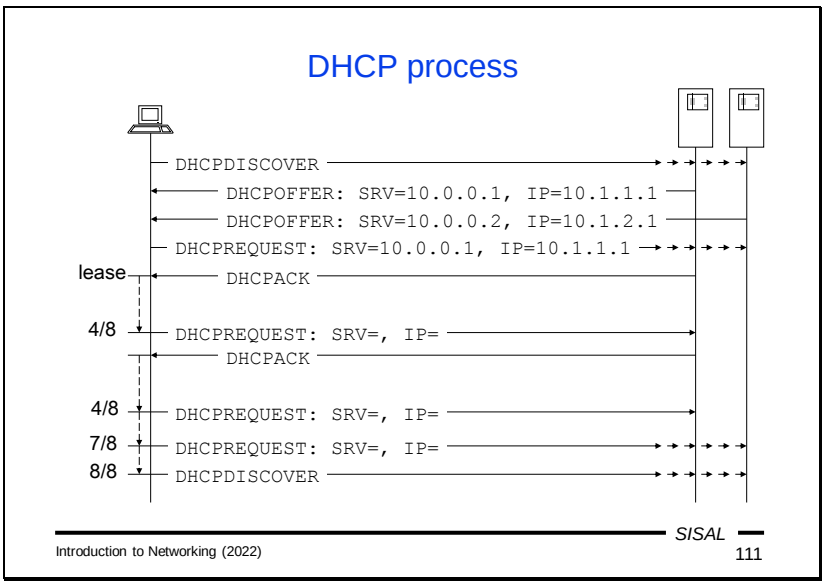
Over time, it turned out that sending just an IP address is not enough. We have already seen several examples of additional information useful for clients: addresses of routers, name servers, NTP servers, mail forwarders, etc. The protocol gradually expanded several times, until in a major change it evolved into DHCP, the Dynamic Host Configuration Protocol. The main DHCP extensions include:

- Dynamic address allocation. Fixed address assignment has lost its security significance today due to rewritable MAC addresses. In addition, network management often does not even know about all clients. And last but not least, with dynamic allocation a network can offer significantly fewer addresses than the number of potential clients.
- Limited rental time. Due to dynamic allocation, it is necessary to restrict clients' access to addresses for a certain period of time. The client therefore receives its address only for a certain period of time, the so-called *lease-time*, and after its expiration it must stop using this address.
- Cooperation of multiple servers. Multiple DHCP servers can operate in a network; they can differ in their purpose and the range of addresses provided, and the client may choose from their offers.

DHCP is backward compatible with BOOTP, which allows BOOTP clients to communicate with and receive the necessary information from DHCP servers.

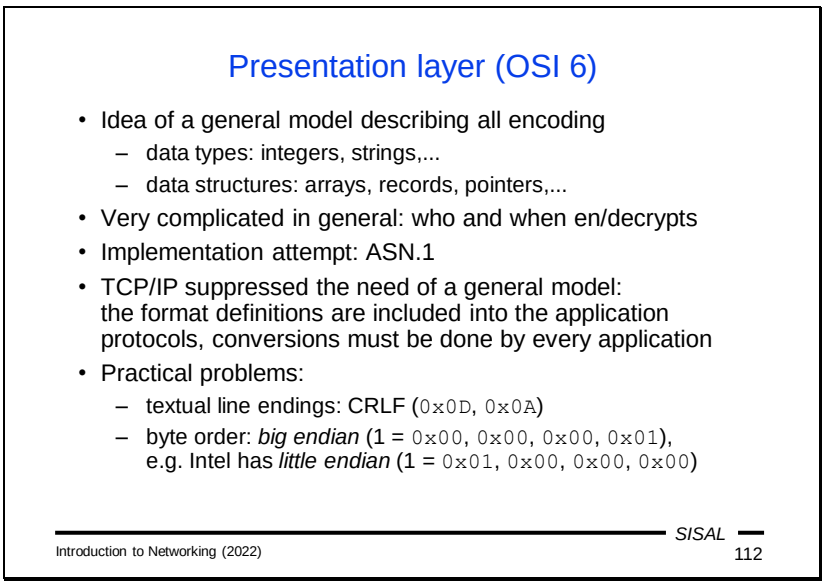
Today, DHCP is the most common way clients connect to a network. For instance, on Microsoft Windows systems, this option can be found under the somewhat obscure name "obtain IP address automatically".

Slide 111



Example of communication in DHCP:

- The client sends a DHCPDISCOVER request using the limited broadcast address.
- All DHCP servers in the network send their offers (DHCPOFFER).
- The client waits for a specific timeout, collects and checks responses.
- It chooses the best offer it has received. The specific algorithm may vary from client to client, but usually the client primarily prefers if a server offers an address that the client already uses (or has recently used). If it does not receive such an offer, it usually chooses according to the length of the lease-time.
- The client sends a DHCPREQUEST message that contains the address it has chosen. This message is still a broadcast because all servers must receive it. After sending their offer, they block the offered address for a while, so that they do not offer it to two clients. If a client has not chosen their offer, they must unblock the address again.
- The server, whose offer the client has chosen will then confirm with a DHCPACK message that the address is really still available.
- From this moment, the lease-time period begins.
- However, halfway through this time, the client should send a DHCPREQUEST message, this time only to the selected server, to make sure the address is still available.
- If it receives a response, a new lease-time period is started.
- If the client does not receive a response, it sends a new DHCPREQUEST within seven eighths of the lease-time period, but this time again by a broadcast.
- If it still does not receive an address, it must start the procedure again from scratch after the leas-time period has expired.



We move from the application layer to the OSI model layer number 6, the **presentation layer**.

The intent of the design was to create a general mechanism to shield the specific architecture of the network node from the format used by the lower layers. However, creating such a mechanism for completely general data communications, i.e. encoding various data types or structures, is relatively complicated. One of the attempts to solve it was ASN.1, which we talked about in connection with H.323. As we mentioned at the time, in terms of description it was a good attempt, but the implementation as a universal tool was far less successful due to its enormous complexity.

Therefore, the TCP/IP design did not go that way and embedded the necessary coding directly into the application protocols, so it moved the concern for encoding and decoding to the application implementation. Fortunately for the programmer, there are simple tools to do this. The most crucial differences between platforms lie in two aspects:

- Different operating systems use different characters or combinations of characters to indicate the **end of text lines**. Microsoft DOS followed the principle used by teletypewriters, mechanical typewriters and old printers: to establish a new line, it was necessary to move the so-called *carriage* to the beginning of the line and then turn the cylinder by one line. The result was the use of a pair of characters CR (carriage return, hex code 0x0D) and LF (line feed, hex code 0x0A) at the end of the each line. Microsoft Windows continues this tradition. The authors of UNIX evaluated the impracticality of this combination and introduced the use of only a LF character. For interest, we can mention old Apple systems, which, on the other hand, used only the CR character. Unfortunately, TCP/IP protocols began to use

the CR + LF model. This has some annoying consequences – e.g. how should a node behave when a counterparty sends the CR character? Should it wait to see if a LF will arrive as well? How long?

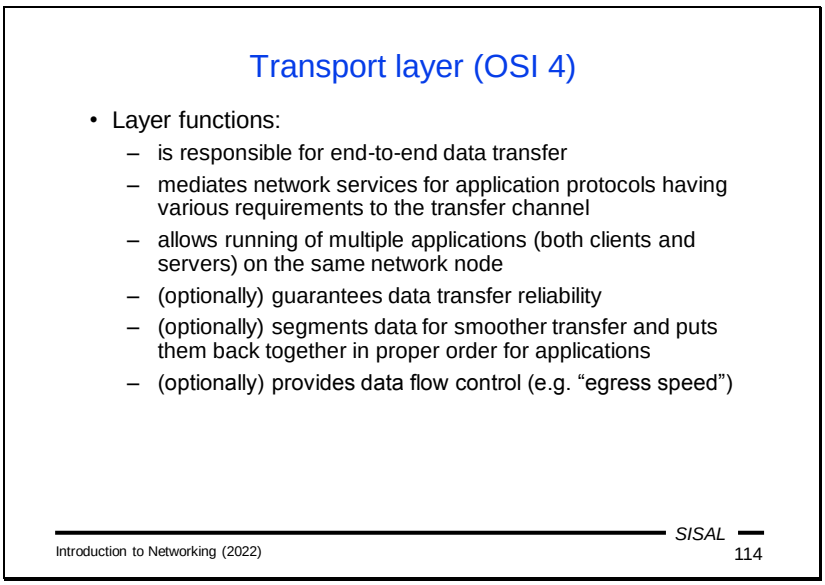
- Different hardware architectures use different byte orders for multi-byte values to store them in memory. With the exception of some obscure alternatives, there are two basic approaches: either the bytes are stored by placing the least significant byte first ("little end" first) and then the next, or vice versa ("big end" first). TCP/IP protocols use a **big-endian** system, i.e. the first byte of an IP address goes through the network first. Libraries may help a programmer to perform an appropriate conversion if the compilation is performed on a little-endian system.

Session layer (OSI 5)

- Idea of a general dialog model
  - one dialog can consist of more connections
  - one connection can carry more dialogs
- TCP/IP suppressed the need of a general model: the dialog principle has been included directly into the application protocols, e.g.:
  - within one SMTP connection a client can send several mails to the server
  - SIP initializes dialog using more partial media data channels

The **relational layer** met a similar fate as the presentation layer. The idea of a general tool that can control the dialogue of communicating parties for all types of application protocols has not proved to be practically realizable, although we can find this functionality in some protocols (like RPC in NFS).

In the TCP/IP model, therefore, the functions of OSI 5 as well as OSI 6 are integrated directly into the application protocol. Examples of situations where the logical dialog of the communicating parties does not correspond to one connection can be found in SMTP (the client transmits several separate messages to the server within the same TCP connection) or in SIP (one video call can be realized by one control connection and two data channels for audio and video). All application protocols then describe the actual control procedure of the dialog flow.



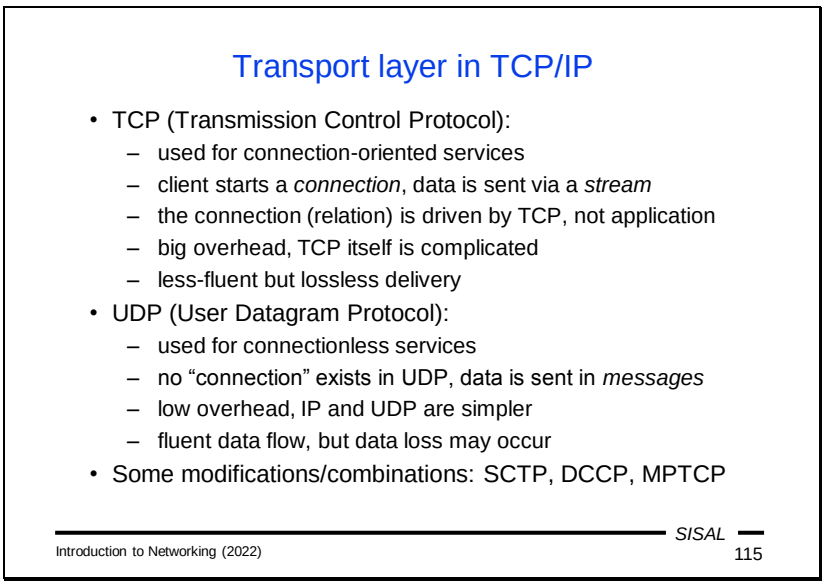
A task of the **transport layer** is to provide applications with a communication channel for the transfer of data units between applications running on end devices. It can be implemented using different protocols, which differ in the scope of services provided.

The interface between the transport and application layers generally gives an application the ability to send a block of data to the counterparty; the specific parameters and conditions depends on the needs of the application. The fundamental difference between various transport layer protocols lies in the delivery guarantee. Some protocols (such as TCP) provide a so-called *reliable* service, which means that the interface function returns a value that indicates whether the data was delivered successfully or not. Some protocols (such as UDP) provide an *unreliable* service, so the function returns only the result of **sending**, not **delivering** data. Delivery detection and possible response to a delivery failure must be handled by the application itself.

An obvious function of the layer is *multiplexing*, in this case access to the network for various connections between local and remote sockets (clients and servers). The individual communication channels are distinguished via **ports**.

Some protocols (e.g. TCP again) perform data *segmentation*. They can receive blocks exceeding the size of data units that can be sent over the connected network, divide the data into smaller blocks (segments) and send them separately. The receiving side reassembles them into their original form and informs the sending party whether the data has arrived, so that it is able to resolve a possible failure of a segment.

An additional protocol function may be *flow control*, a mechanism for changing the data transmission rate so that the communication channel is used efficiently without congestion.



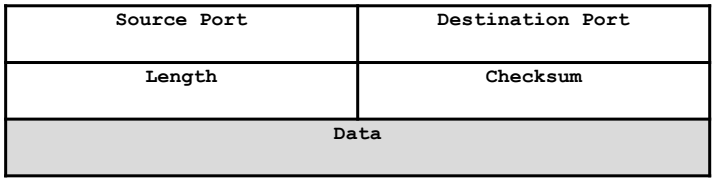
The major protocols used at the TCP/IP transport layer are TCP and UDP.

TCP is used for **connected applications**. To illustrate, a connected application can be compared to a phone call. The caller dials the number, his telephone operator establishes a connection to the target device, and as soon as the call is connected, the user starts an "application", i.e. starts talking and listening. The application has a very simple job: it does not have to worry about whether individual sentences arrived and whether they arrived in the correct order. Applications working over TCP behave similarly – the client establishes a connection and a so-called *stream* is created, a channel through which data flows in both directions. TCP is *reliable*, which means that if an application does not receive an error return code, it is sure that all data has arrived in order. The overhead for this guarantee is fully on the TCP side, and is therefore quite complicated and robust; applications, on the other hand, can be relatively simple. Of course, TCP overhead (acknowledging delivery, waiting for acknowledgements, forwarding lost data) reduces logical transmission capacity and can cause fluctuations in delivery frequency. A disadvantage of TCP is also that an application has a very little ability to react to the state of the network – once it calls a transfer function, it has no choice but to wait until the data is transferred or the appropriate timeouts expire and the function announces a failed delivery.

UDP is used for **connectionless** applications. An analogy is communication using traditional mail. If we drop a letter into a mailbox, it corresponds to calling the UDP send function and the result is only “succeeded to send" or "failed to send". If we send more letters, we have no guarantee that they will all arrive and that they will arrive in the correct order (and in the computer world it may even happen that some messages will be delivered twice). If we want to build a reliable channel in this way, we (the "application") must take control. In UDP, separate messages are sent; UDP

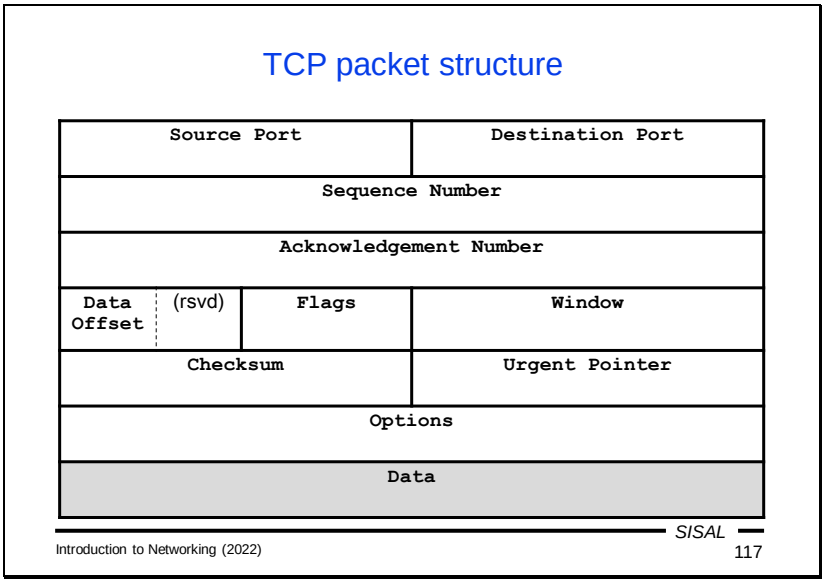
itself handles only transmission, so it has very little overhead. All overhead is transferred to the application, which also has the advantage that the application can respond more flexibly to the current state of the network. Unlike TCP, data in UDP can flow more regularly, and any loss must be resolved by the application – either it should arrange the retransmission itself, or it must be able to work without the missing part of the data.

UDP datagram structure



From the previous description it is clear that UDP does not need to add a lot of information to the data from the application layer for its work and the encapsulation of application data into the PDU of the transport layer is very simple.

In the UDP header, only multiplexing information, i.e. a source and destination port, and control information (length and checksum) are transmitted.



The TCP header contains a number of additional information fields compared to UDP.

In order for TCP to guarantee the completeness of the transmission, a unique identification number must be added to each segment. Roughly said, each segment contains a relative **offset** to the beginning of the stream (in the header we see the so-called *Sequence number*). In order to be able to confirm delivery as well, we need an analogous field for the opposite direction (the *Acknowledgement number*).

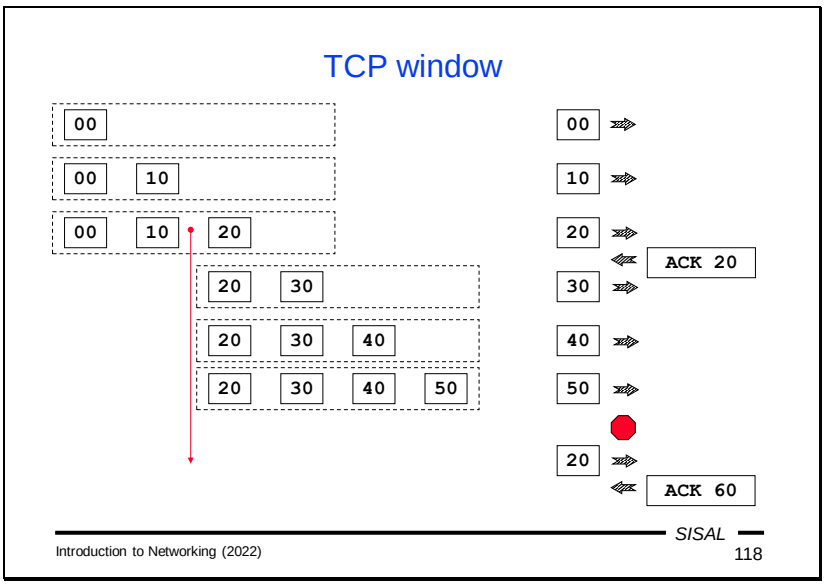
The *Flags* field contains flags, most of which we will discuss immediately.

The *Urgent pointer* field is used for an *out-of-band* transfer function. The application can mark certain data as **urgent** with the URG flag. For instance, the command by which a FTP client wants to interrupt a data transfer is sent as urgent. Such data is then inserted into the normal communication and its relative address within the data block is written in this field.

We will get to the other important fields of the header in more detail later.

Note: I present the structure of the TCP packet here to demonstrate its functions; I am definitely not going to examine the position of individual fields.

Slide 118



Let's take a closer look at how delivery acknowledgment and data flow control are handled in TCP.

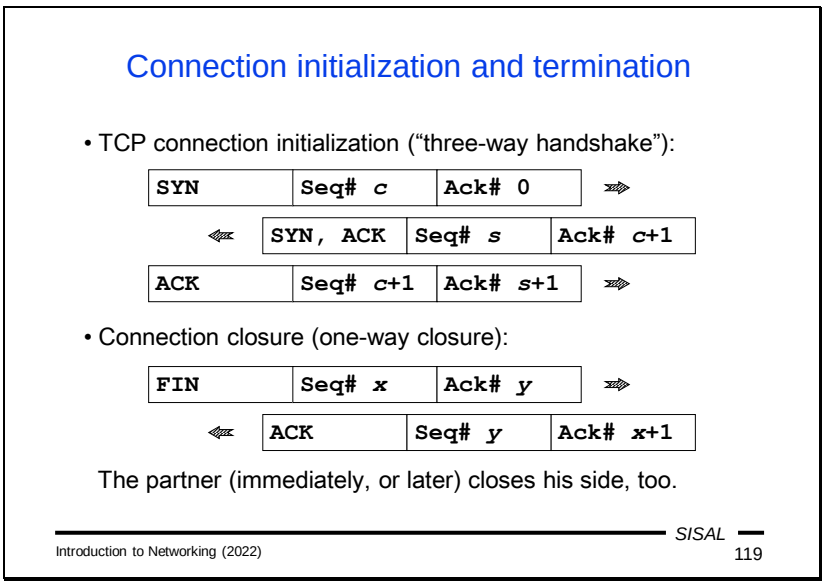
As already mentioned, TCP acknowledges the delivery of data. The recipient acknowledges the delivery of a block by sending a packet to the sender with the ACK flag set and the Acknowledgement number value set to the end of the data offset that was delivered. However, if these acknowledgements were sent for each packet in a separate packet, it would make unnecessary traffic. Therefore, it is possible to pack the confirmation (flag + offset) into some other data that the recipient needs to send back. The usual implementation is that a computer will wait a certain amount of time before sending an ACK as long as another data item that will need to be sent to the other side does not appear to connect the ACK to. If this does not happen within a given limit, it sends the ACK separately.

Similarly, the communication channel would be used inefficiently if the sender had to wait for confirmation of the previous packet before sending any new packets. Therefore, TCP allows both parties to agree on a range of data that the sender may send without waiting for confirmation. This limit is called a **window size**, and a node may announce a proposed size in the *Window* field in the TCP header.

Consider a situation where one side of the connection starts sending blocks with a data size of 10 bytes (in reality, of course, larger blocks are used) with a window size set to 40 bytes. The sender starts sending blocks without waiting for confirmation, but monitors the maximum window capacity. If a delivery receipt arrives in the meantime, it moves the "window" to the position it contains. For instance, if the acknowledged offset was 20, it can now send data up to offset 60. If no further acknowledgement arrives by then, it must stop sending and start waiting for a new acknowledgement. If

the acknowledgement does not arrive, it must resend the first unconfirmed block of data.

Slide 119



The term "TCP connection" refers to a situation where the client and server have confirmed their willingness to communicate with each other and have agreed on the sequence numbers to be used. In fact, sequential numbers do not start from zero, for security reasons, but from a **random number** chosen by the sending party. Therefore, it is necessary to send the initial value to the counterparty. This agreement takes place at the beginning of the connection using three special packets that have an empty data part and carry information only in the header, and is called a *three-way handshake*.

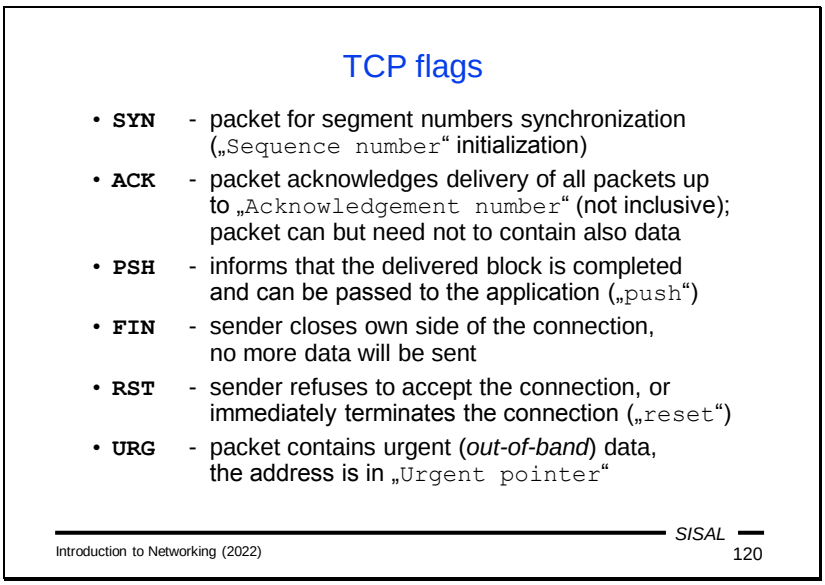
The first of these packets has the SYN (synchronization packet) flag set and has the initial value selected by the client as the Sequence number; let's call it *c*. The server acknowledges receipt by sending a packet with the ACK flag and the Acknowledgement number set to *c* + 1. At the same time it generates its initial value of the sequence number (*s*) and adds the SYN flag as well. The client then completes the procedure by sending a packet in which it acknowledges receipt by sending an ACK with a value of *s* + 1.

From now on, both parties can send data and gradually increase the values of the sequence numbers by the data length.

If either party wants to end the connection, it sends a packet containing the FIN flag. This tells the other party that the sender no longer intends to send **any data**. Usually, the other party immediately sends a FIN packet as well, but if it does not want to terminate the connection, it can continue to send its data. In that case, the party that FIN has already sent will have to forward its ACK **packets** so that the connection does not break. We can easily imagine this situation in HTTP, for example. After a client sends its request, it can close the connection and just wait for a server to

respond. This condition is called a *half-closed* connection and is valid, although not fully recommended, as various control mechanisms along the way can consider the connection as dead and terminate it. Therefore, for example, the mentioned HTTP client should leave the connection open and close it only when it has already received the entire response.

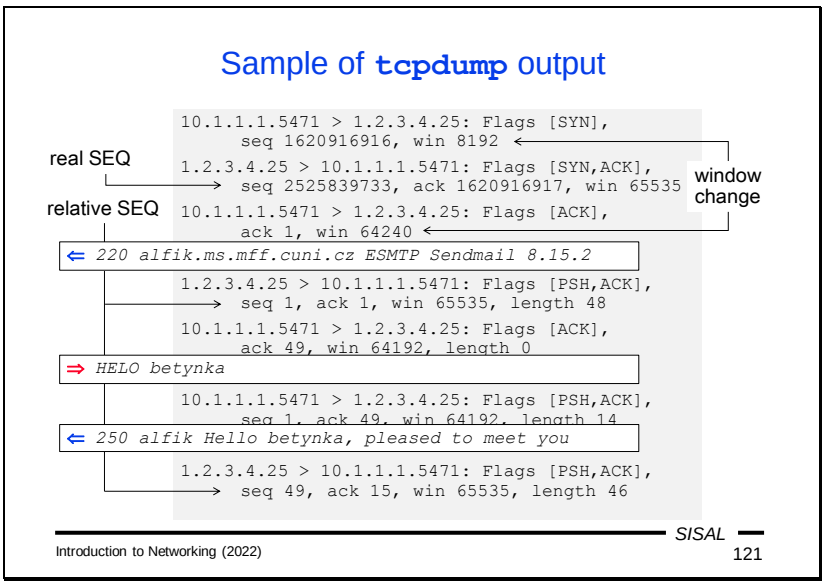
Slide 120



In addition to the already mentioned flags URG, SYN, ACK and FIN, the following are also important:

- PSH (push) - With this flag, the sending party notifies the other party of the last segment of a data block. For the recipient, this means that if it has already received all the other segments of the block, it can pass the block on to the application for further processing.
- RST (reset) - With this flag, the sender notifies the other party that it does not intend to continue communication. The flag can be used by a server during the three-way handshake as a response in case it does not want to accept the connection, or by either party at any time during the communication. Unlike termination with FIN, RST means an immediate termination without waiting for confirmation or for FIN from the counterparty.

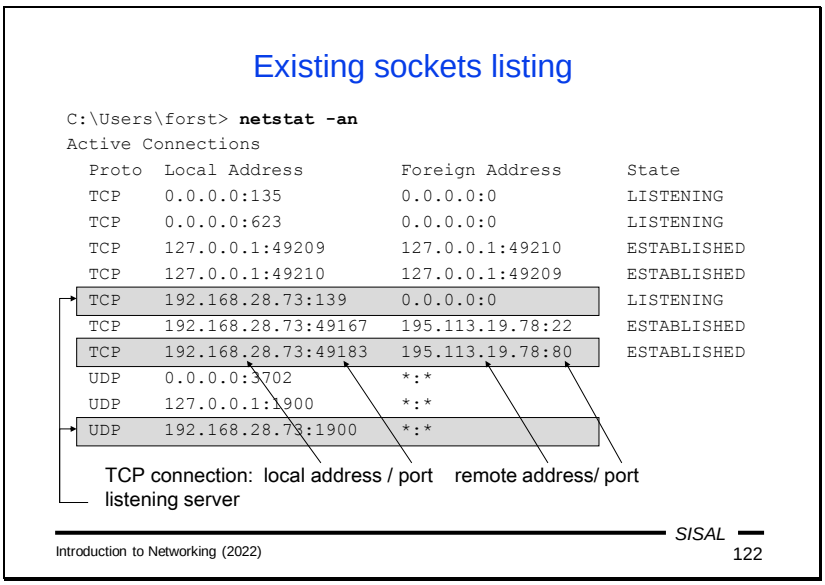
Slide 121



If we want to look at a TCP/IP communication in more detail, we can use a program that can capture and display it in a readable form (e.g. tcpdump, Wireshark, WinPcap). Let's look at an example of the output of the first of these programs…

- The first packet is the first packet of a three-way handshake, which we can recognize by the fact that it carries only the SYN flag. At the beginning of the line, we see the source IP address and the dot-separated port number, and after the arrow, the destination IP address and port. This is port 25, so this will obviously be the first packet of an SMTP connection. We also see the initial sequence number (**seq**) and the window size (**win**).
- The second packet is the server's response (it carries the SYN and ACK flags); we see the confirmed client sequence number (**ack**) and also a proposal of a larger window.
- In the last three-way handshake packet we already see the value of the Acknowledgement number **relatively**. Tools such as tcpdump display sequence and acknowledgement numbers as offsets relative to the beginning of the communication (subtracting the initial value) for better readability.
- The next packet is the server's introductory message. For the first time, we see a data length printed (**length**, 48 characters) and the PSH flag (push), because the line is complete.
- The client apparently lingered a bit with the sending of its first command, so the TCP software sent a separate delivery acknowledgement for the start line from the server, in the fifth packet. It used the initial value plus the length of the previous data (in relative numbers 1 + 48 = 49) as the Acknowledgement number value.
- In the sixth packet, we see the first client command.

- The seventh packet contains the server's response. In addition to the data itself, it also carries the ACK flag and a value that confirms to the client’s TCP layer the receipt of the previous 14 bytes (in relative numbers 1 + 14 = 15). The server therefore combined the sending of the acknowledgement with its own data.



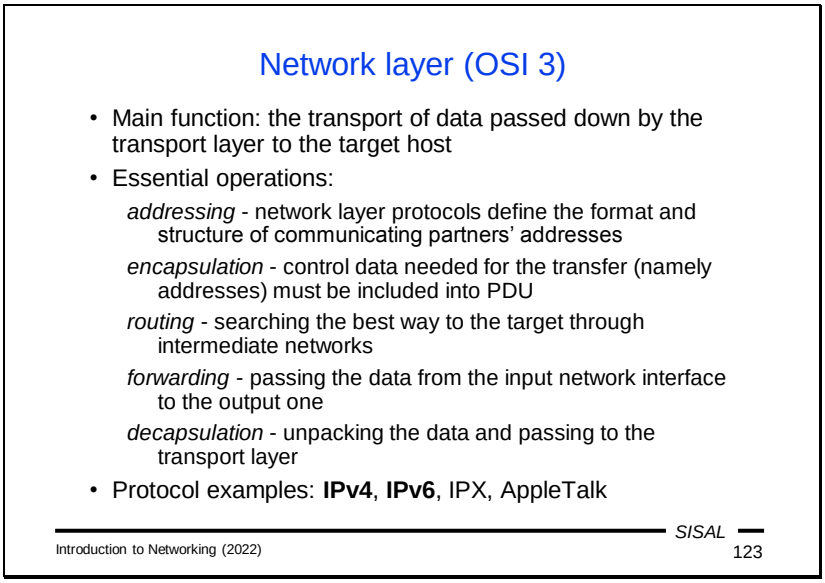
If we want to get an idea of what connections are currently open on our computer, we can use the **netstat** command. With the **-a** parameter, it lists all TCP and UDP servers and open TCP connections (there are no connections in UDP).

For TCP as a state protocol, the listing also contains the state of the connection in the last column; in the example we see LISTENING (a listening server) and ESTABLISHED (an open connection), but a number of other states exist.

In the second column we have the address of the local socket; there we see either the actual address of a network interface or the address 0.0.0.0, which means that the server is listening on all interfaces that are on our computer.

The third column is the socket address of the counterparty, here is either the actual address and port of the remote socket, or in the case of the server the value 0.0.0.0 or *, which means that any client can connect.

In the case of UDP, we only have running **servers** in the list, because UDP has **no connections**, so netstat has no information about the exchange of messages between clients and servers.



The task of the **network layer** is to find a path to a destination computer so that it is possible to deliver data passed by the transport layer regardless of technologies used in the lower layers.

The layer stands on two basic pillars:
- Address scheme – a way to identify individual nodes in the network so that it is possible to distinguish which network they belong to.
- Routing – a way to find a valid path from a source to a destination network based on the address.

Other features of the layer include encapsulation, which we have mentioned before, and **forwarding**; that is the principle that a router can receive a packet even though it is not the packet's final destination and tries to deliver it one step further to the destination as if the packet originated directly on the router.

Internet Protocol (IP)

- Properties:
  - connectionless (datagrams are delivered independently)
  - best effort (unreliable, logic delegated to higher layers)
  - media independent (higher layers need not to bother with it)
- Addresses:
  - contain network address part and node address part
  - IPv4: 4 bytes, IPv6: 16 bytes
- Assignment:
  - central: IANA (Internet Assigned Numbers Authority), department of ICANN
  - regions: RIR (5x, Europe: RIPE NCC)
  - further: ISP
  - local network: network management (manually/automatically)

In the TCP/IP protocol suite, the Internet Protocol (IP) is used at the network layer. This service is

- connectionless – individual datagrams are delivered independently, no formal connections are created,
- unreliable – the network layer does not guarantee delivery (as we have seen with UDP, which takes this property from IP),
- independent of the medium – the transport layer generally does not have to deal with the details of the technology used (except for the MTU, which we will talk about later).

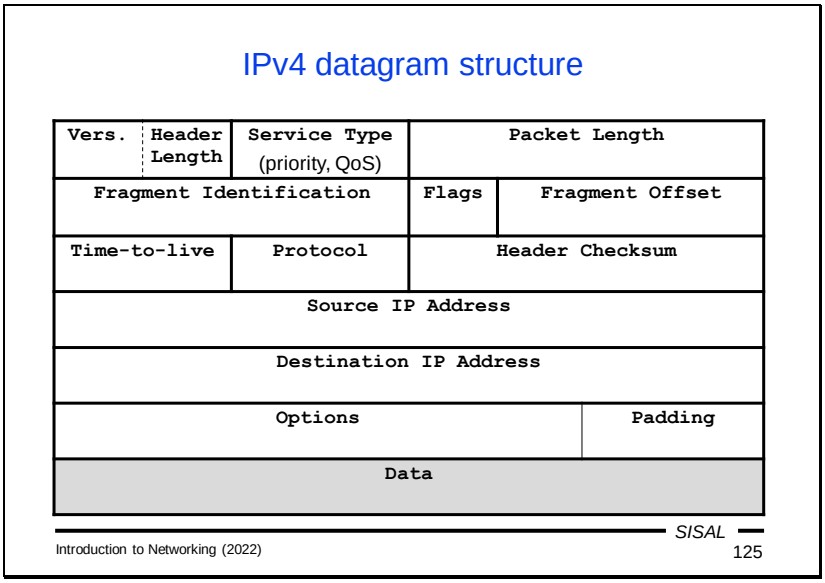
There are currently two versions of the protocol: version 4 has an address length of 4 bytes, and in version 6 is it 16 bytes. For both versions, there is a method to determine the **network address part** (which is responsible for routing) and the **host address part** (which is responsible for delivery to the destination node) of any address.

Each node that wants to communicate in a TCP/IP network must have an IP address. The method of assigning addresses to connected end stations is decided by the network administration. Each network uses certain ranges of addresses for its nodes – either private ones (these are chosen by the network administration itself) or public ones (these are assigned by the ISP that connects the network to the Internet). ISPs receive blocks of addresses from superior ISPs; at the top of the hierarchy is IANA (the Internet Assigned Numbers Authority, part of ICANN), which has five regional registries under it:

- RIPE NCC manages Europe, Russia and West Asia
- APNIC manages the rest of Asia, Australia and Oceania
- ARIN administers the USA, Canada, Antarctica and part of the Caribbean

- LACNIC manages Latin America and the rest of the Caribbean
- AFRINIC manages Africa.

Slide 125



Encapsulation at the network layer adds an IP header that contains information important for delivery. In the case of the version 4 protocol, the header contains the following information:

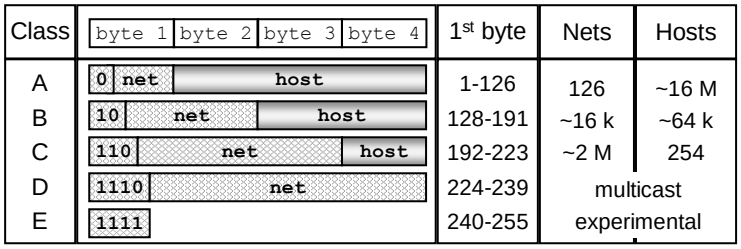
- Version. Half a byte is reserved for the version, so a different format will be needed for IP version 16.
- Header length. It takes up the second half of the first byte. Looking at the diagram, it is probably clear that a maximum value of 15 would not be enough for the entire length. This is really the case, which is why the length is given in **32-bit words** and not in bytes. This means that the maximum length of the header is 60 bytes, and if its content is not a multiple of 4 bytes, it must be padded.
- The second byte contains QoS information and the rest of the first word is filled with the length of the entire datagram.
- The second word is dedicated to **fragmentation**. This is a procedure that applies when the network layer receives a packet so long that when it is encapsulated, we would get a frame longer than the maximum allowed for a given link layer (the so-called Maximum Transmission Unit, MTU). In this case, the network layer must fragment the packet into multiple datagrams and send them sequentially. Fragmentation is an unnecessary complication if it can be avoided. This is the case with TCP, which segments the data itself, so it is possible to instruct it to use a correct segment size. Therefore, a **Path MTU** search method is used – packets are sent with the *Do not fragment* flag, thanks to which the sender learns about possible problems with the size of the MTU along the path and can correctly determine the segment size to be passed to the transport layer.
- The third word contains the Time-to-live value, to which we will return, the protocol number that is encapsulated in the datagram, and the checksum of the header.
- This is followed by the IP addresses of the sender and recipient.
- Additionally, the header may contain options.

Note: I present the structure of the TCP packet here to demonstrate its functions; I am definitely not going to examine the position of individual fields.

Slide 126

IPv4 addresses

- Originally: one byte
- 1975 (RFC 687): three bytes („*This expansion is adequate for any foreseeable ARPA Network growth.*“)
- 1976 (RFC 717): one byte (network) + three bytes (host)
- 1981 (RFC 791): classes A, B and C



The situation in which computer networks began to be built is nicely illustrated by the fact that originally **a single byte** was reserved for the computer address! Even the authors of the original proposal clearly could not imagine how far our daily lives will be affected by what they were just beginning to build.

In 1975, it was decided that the address should be extended. The new length was set to three bytes, and RFC 687 contains a note that this extension was considered **adequate for any foreseeable growth** of the network, which is amusing from today's perspective!

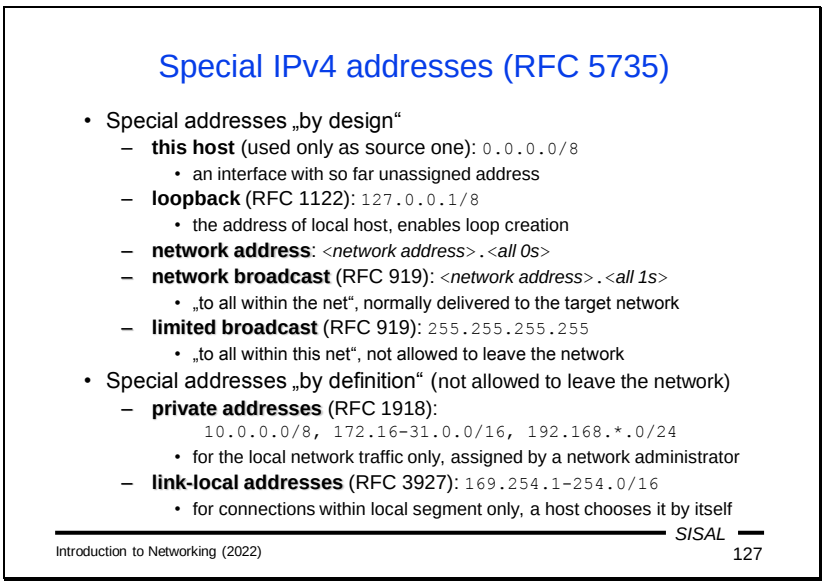
A year later, the address size has expanded again. Not that the addresses were running out so quickly, but addresses were supplemented with a section for a network address (1 byte) and the original 3 bytes remained for a host address.

Another five years later, the number of networks turned out to be growing much faster than expected, the original model was refined and three classes of addresses were created for networks of various sizes. Class A covers the lower half of the address space and uses the original 1:3 division. Class B occupies the third quarter of the space and each class B address is divided into 2-byte portions. Class C occupies the penultimate eighth of the address space and divides addresses by a ratio of 3:1, which means that it offers about 2 million networks each containing a maximum of 254 computers.

Note: You might expect 256 instead of 254 for the number of addresses on a C network, but the first and last addresses on each network are reserved.

The last eighth of the address space was reserved for further development, and in 1986 class D for so-called *multicast* addresses was separated from it. Class D addresses completely lack a host address part, because they are used to address a group of recipients – for special services (e.g. NTP uses the address 224.0.1.1), video conferencing, etc.

Slide 127



Some IP addresses have a special meaning according to the protocol definition:

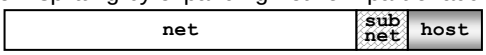
- An address with all zeros is used as the source in a situation where we need to communicate but we do not yet know our address.
- The address 127.0.0.1/8 is reserved for a special interface existing on each node of the network and representing "**this computer**", the so-called *loopback address*. When we need to establish communication between a client and a server both running on our computer, they can both use this address.
- An address with an all-zeros host part is the *network address*.
- The last IP address in the address block of a (sub)network represents a *network broadcast*, an address that we can use if we want to address all computers in the network.
- In addition to the network broadcast address, which is normally delivered across networks, there is also a ***limited broadcast*** (255.255.255.255) that must not leave the network where it originated.

In addition to these special addresses, there are two categories of addresses whose meaning is assigned by an organizational decision, not by the IP architecture:

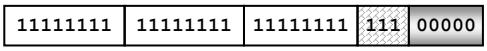
- The first group are *private* addresses, which include one class A network, 16 class B networks and 256 class C networks. As we already know, these are used in local networks and a packet with a private address can only leave the network in special cases. Otherwise, address translation (NAT) must be used
- The second group are *link-local* addresses. These addresses are freely usable; each computer can take any of these addresses and start communication on the network segment where it is connected. Unlike private addresses, however, they cannot be used to communicate outside the network at all.

Subnetting

- Network splitting by expanding network part of address:



using so called network mask (*netmask*), in this case 255.255.255.224:



- Subnets "all-zeros" and "all-ones" are not recommended, so we have here just 6 x 30 addresses (70%)
- Non contiguous mask is possible, but usually not used
- Nowadays, the classes are often ignored (*classless* mode), using only the prefix length in bytes (e.g. 193.84.56.71/27)
- The term *variable length subnet mask* (VLSM) describes situation when various masks are used in a network
- Moving the border in opposite direction: *supernetting*

Over time, it turned out that even a finer division into classes was not fine enough, and especially in local networks, it is necessary to move the boundary between the two parts of the address even further "rightward", to create several **subnets** from one network. In this case, however, it is not enough to set an address when configuring a network interface; you must also specify the network size. One way to do this is to add a so-called **netmask**, a number that contains ones just where the network address is. For example, if we want to shift the network boundary of a class C address by three bits, we use the value 255.255.255.224. The choice of input method is quite logical – if we have a host address and want to know if it belongs to our network, we just take the netmask and the host address, perform a *bit AND* operation, and if the result is our network address, the host address is "ours". However, calculating a netmask is a bit complicated, so today it is often replaced by the so-called *classless* format, where we write only the number of bits that make up the network address part after a slash.

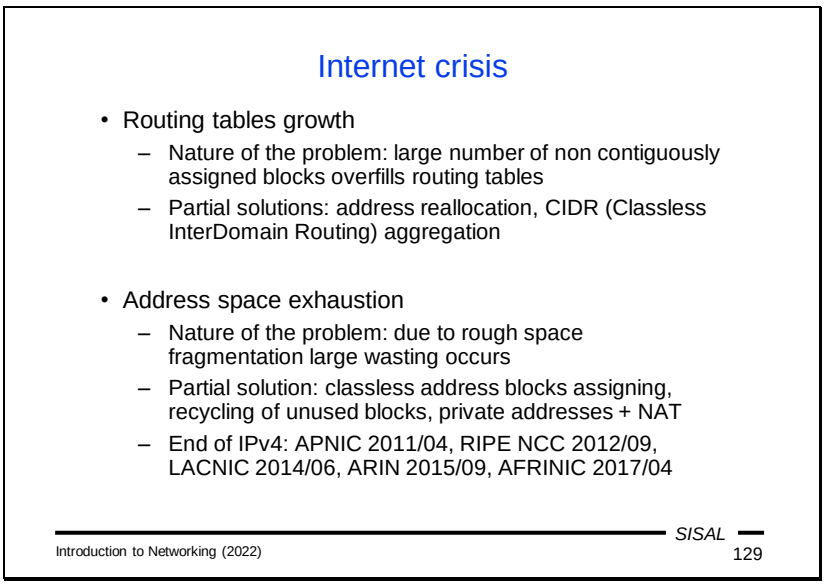
Using subnetting allows you to divide a network into smaller parts, but reduces the number of addresses available. Since the use of a subnet numbers with only zeros and only ones is not fully recommended, a subnet mode in our example will reduce the network to 6 subnets of 30 computers.

Moreover, this division is again very rough, and if we need a more flexible division, we will not be able to do with one mask. For instance, if one of our 6 subnets has more than 30 computers, but two have at most 14 computers, one class C address will not be enough for us with subnetting using a fixed mask. We will have to use the so-called *Variable Length Subnet Mask* (VLSM) mode and use the /28 mask in two small subnets, and /26 in the large network.

The natural question is whether the border can also be moved in the opposite direction, i.e. to join networks instead of dividing them. This is useful e.g. in a situation where I have more than 254 computers in a LAN and I have two class C networks for them. If I did not use *supernetting* and connect them to one network with the /23 mask, I would have to deal with unnecessary complications when communicating between two computers which, although in one LAN, happen to have addresses from different class C networks.

Fortunately, private addresses are used extensively in LANs today, where there are a sufficient number of networks of all classes, so subnetting does not need to be addressed as often.

Note: If we are handling an IP address that does not belong to our network, of course, we cannot know what subnetting its local network uses. But that doesn't matter, because we don't care about it. From the point of view of routing, we communicate with a foreign address as if it does not use any subnetting – our task is to deliver the packet to the target network and the final delivery is already fully under control of that network.



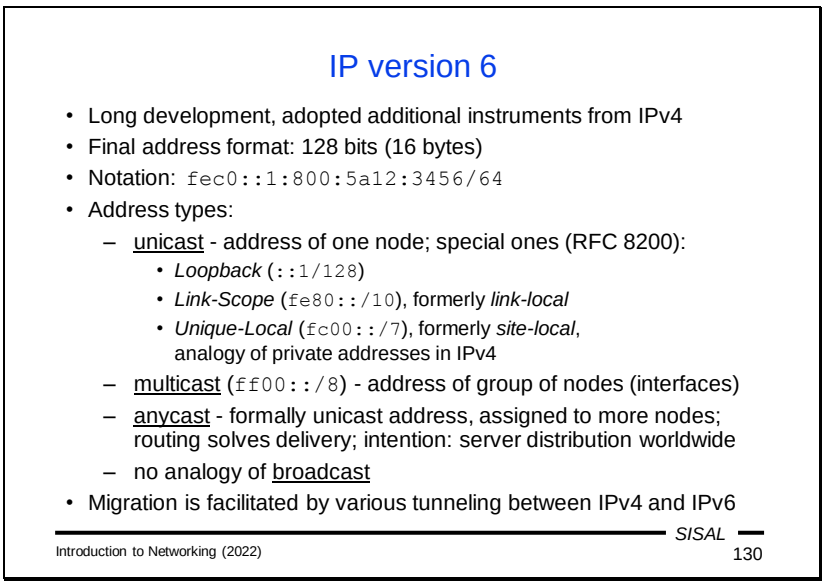
The general feeling of IP address sufficiency in the spirit of the daring sentence from RFC 687 meant that IP address allocation was not governed by any strict rules at that time. At the turn of the 80s and 90s, however, it began to appear that this was beginning to bring problems in two ways.

Tables of central routers began to overflow. This could be prevented by the approach that if in reality some networks were adjacent to each other and they also had adjacent numerical values, it was possible to **aggregate** their records on most routers, i.e. actually do supernetting over them and combine both records into one with a shorter network mask. This principle is the so-called Classless InterDomain Routing (CIDR), but its effective implementation had to be preceded by massive renumbering of many networks on the Internet, so that they could be effectively aggregated at all.

At the same time, it became clear that allocating arbitrary ranges to any network would sooner or later lead to exhaustion of the IPv4 address space. ISPs started to save addresses, renumber networks, allocate blocks that did not correspond to IP classes, etc. Fortunately, the use of private addresses and NAT began to be massively introduced, which dramatically reduced the need for public IP addresses (a network with one thousand computers needs with NAT only one address instead of one thousand).

In spite of that, however, it was clear that these methods are able only to postpone the problem, not to solve it, and intensive work began on the preparation of the so-called IP next generation (IPng).

Slide 130



The creation of a new version of IP took quite a long time, as well as its subsequent introduction into various operating systems and debugging problems. Fortunately, CIDR and NAT postponed the problem, so even before IPv6 could actually be deployed, both the protocol and network software was well prepared. In addition, the long co-existence of both versions was expected and the transition is facilitated by various variants of IPv6 tunneling to IPv4 and vice versa. In addition, the authors of IPv6 took a number of additional tools and features from the previous version and, after a thorough analysis, modified them and incorporated them directly into the IPv6 design.

In its final form, an IPv6 address has 16 bytes and is normally written in 16-bit words in hexadecimal with CIDR mask notation. In order to be able to shorten the notation of current addresses containing long sequences of zeros, a single series of null words can be omitted (so that there are two colons in a row in the notation).

In terms of address types, we find many identical and different features in IPv6. Basic, *unicast* addresses include, in addition to public ranges, a loopback address, link-local addresses and unique-local addresses. The latter group is a certain analogy of private addresses in IPv4. Multicast addresses play a greater role than in IPv4. IPv6 lacks **broadcast** addresses and they are de facto replaced by the multicast address ff02::1 ("All IPv6 devices").

Anycast addresses are a complete novelty. They are used in various situations where a resource is available in geographically separated locations and we would like the network to choose the most suitable resource by itself. One anycast address can be set for all affected computers, and a packet sent to that address will be routed to the nearest destination without the client having to explicitly select it.

Summary 6

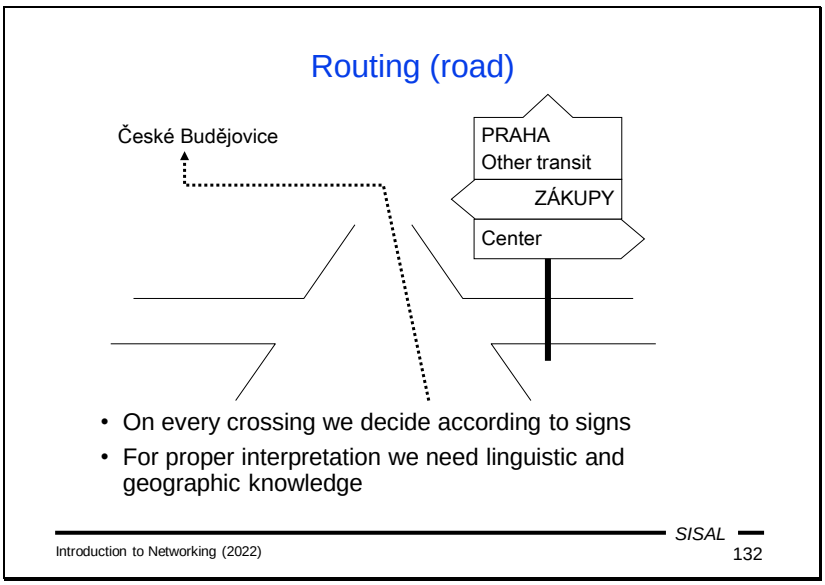
- Why and how is time synchronized on hosts on the network?
- What is DHCP for?

- How does TCP/IP deal with the missing presentation layer?

- Explain the difference between TCP and UDP.
- What is TCP window for?
- What is a three-way handshake?

- How are the IPv4 classes, subnetting, and CIDR related?
- Why are there only 254 host addresses in a class C network?
- What types of broadcast addresses do we know about?

Slide 132

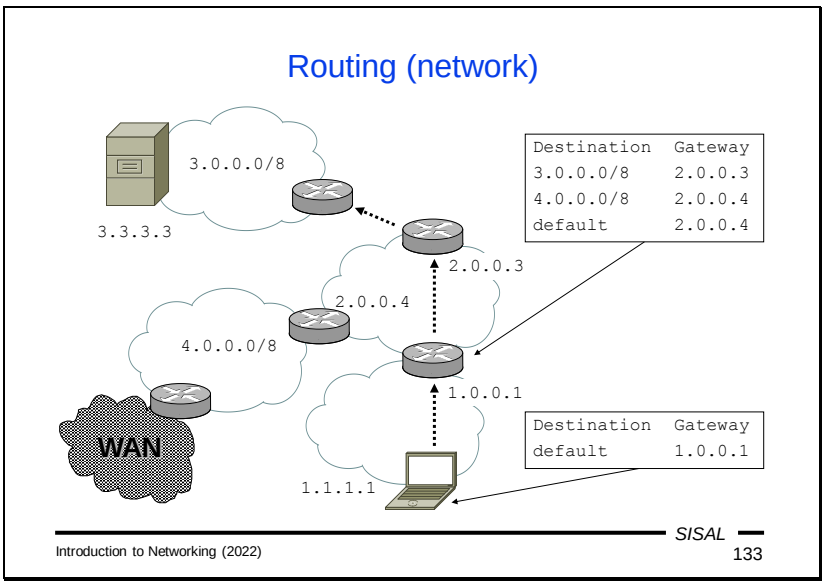


The issue of routing can be compared to a car driver's decision at a crossroads. The driver has a specific destination in mind and uses direction signs to choose an exit from the intersection. The situation in the picture above schematically illustrates what a crossroads in Česká Lípa (a city in North Bohemia) might look like when arriving from the north.

Let's say that the driver's destination is České Budějovice (a city in South Bohemia) which isn't on the signs. The driver must therefore use geographical knowledge and conclude that if he is almost exactly north of Prague and his destination is almost exactly south of Prague, then it can be said that České Budějovice is "in the vicinity of Prague" and the road that leads toward Prague is a good choice for him. If he had not made this decision in Česká Lípa, but somewhere between Prague and České Budějovice, e.g. in Tábor, then choosing the road toward Prague would probably not have been correct. If the driver cannot choose based on the signs, he will have to use the direction "Other transit."

If the driver's destination is Zákupy, it is certainly true that Zákupy is somewhere around Prague, but since he has a far more accurate direction sign pointing toward Zákupy, he will follow this one and not the one towards Prague or for Other transit.

Slide 133



Network software uses a similar process when it searches for the *next-hop* router on the path of a packet. It uses a *routing table* whose contents are controlled by the operating system. The records in this table are something like the direction signs at a crossroads. They also contain a “direction”, which in the computer world means a next-hop router (or *gateway*), and a *destination*. The advantage of a routing table is that each destination always contains a network address, including an address range specified by a network mask (*netmask*), so that here we are not dependent on any geographical knowledge and we know exactly which "surroundings" of the destination are relevant to us.

In the picture above we see a part of the network that our laptop with address 1.1.1.1 is connected to. We want to send a packet to address 3.3.3.3.

The typical configuration of an end station corresponds to the situation of a driver in a supermarket car park, where there's likely to be only one "Exit" sign. Similarly, on our laptop, there will be only one record in the routing table, a so-called *default record*, which will point to the router (1.0.0.1) through which our network is connected.

Once our packet arrives at the router 1.0.0.1, it will search for a next-hop in its routing table. The table will be a little more complicated here. We see three entries:

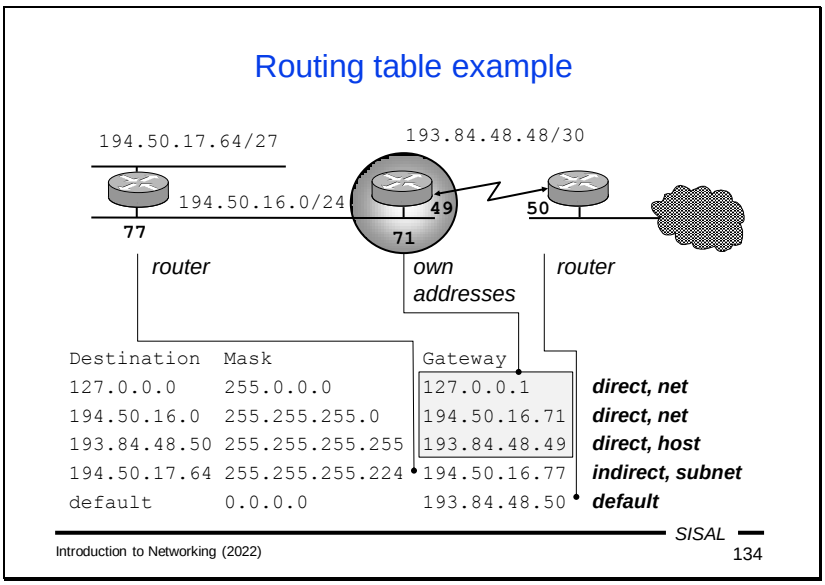
- The network with address 3.0.0.0 and /8 mask (i.e. a class A network) is reachable via router 2.0.0.3.
- The network 4.0.0.0 with mask /8 is reachable through router 2.0.0.4.
- To reach the other networks we can use the router 2.0.0.4 as well.

Which of these records fit our purpose? Just look at the mask for each record and compare the corresponding number of the first bits (the *prefix*) of the record address in the table to our destination address:

- The first record has a mask of 8 bits, so we compare the first byte of both addresses; both of them have value 3, so the record matches.
- For the second record, we also compare the first byte, but it does not match (4 is not equal to 3).
- The third record can be considered as having a mask of **0 bits**, so we compare 0 bits of both addresses and come back with a match.

We now have two suitable records and we have to choose from them. And this is the same situation as when a driver going to Zákupy chooses between the Zákupy and Prague signs – he chooses the direction that indicates a **smaller area** including his destination (an area around Prague that included Zákupy would have to be at least 200 km in diameter). A computer similarly chooses the record that has a smaller area i.e. a **wider mask**. So in our case, the correct option is the record 3.0.0.0/8 and not the default (which is actually 0.0.0.0/0).

Slide 134



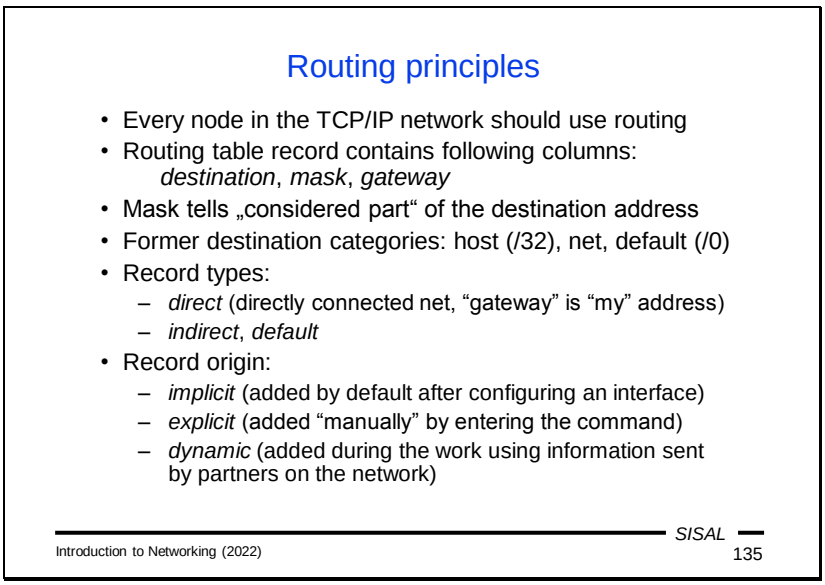
Now let's look at a routing table example in more detail. Let's imagine the situation from the picture above where we have a router connecting the LAN on the left via a point-point connection to the ISP router on the right.

The routing table will contain three so-called **direct** records, or records that describe a directly connected network. These records are created automatically in the table when we configure the appropriate network interface.

- The first record describes a formal network interface with a loopback address. There is no next-hop router in the *gateway* column for direct records, but instead **our own address**, with which we are connected to the network (here 127.0.0.1). This makes good sense – when the software selects this record, it knows that it no longer has to search for a router, but can simply send the packet using a network interface with this particular address.
- The second entry describes the main network of our LAN with class C address 194.50.16.0. In the gateway column we have our own address 194.50.16.71 again.
- The third direct record is the record for the point-to-point network through which we are connected to the ISP. Here we can notice that the destination even has a /32 mask, i.e. one single machine, because at the end of the point-to-point connection there is only one machine, an ISP router with address 193.84.48.50.

Another entry is an **indirect** entry pointing to one of the subnets in our LAN, namely 194.50.17.64/27. An internal router in our network will be used as a gateway. We just need to be aware which of the addresses of this router will be in our table. The router, of course, has addresses on both networks 194.50.17.64/27 as well as 194.50.16.0/8. Of course, in our table, there must be an address from a network we know (ours), otherwise this record wouldn't be much help for us in routing.

The last record is a **default record** that directs all other traffic to the ISP router.



Now let's review our findings on routing.

Any station operating on a TCP/IP network should be able to route. It compares a destination IP address in a packet with records in the routing table. These records contain three basic values:

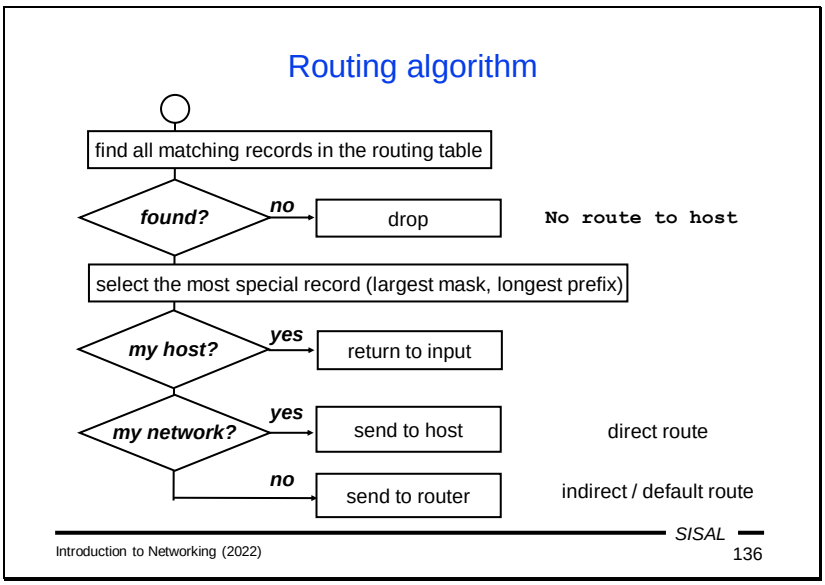
- the network address for which the record is valid (the *destination*),
- the range of this network (the *netmask*),
- a *gateway*, which is either
  - the next-hop router to which the packet needs to be forwarded if the record is indirect (leads to a foreign network),
  - the address of a custom network interface if the record is direct (leads to a directly connected network).

From the point of origin, we divide the records into these categories:

- implicit – a record is created automatically by configuring a network interface,
- explicit – a record is added to the table by a command that is either issued manually by the administrator or run by the operating system when the computer starts,
- dynamic – a record is created only during operation using information received from other nodes in the network.

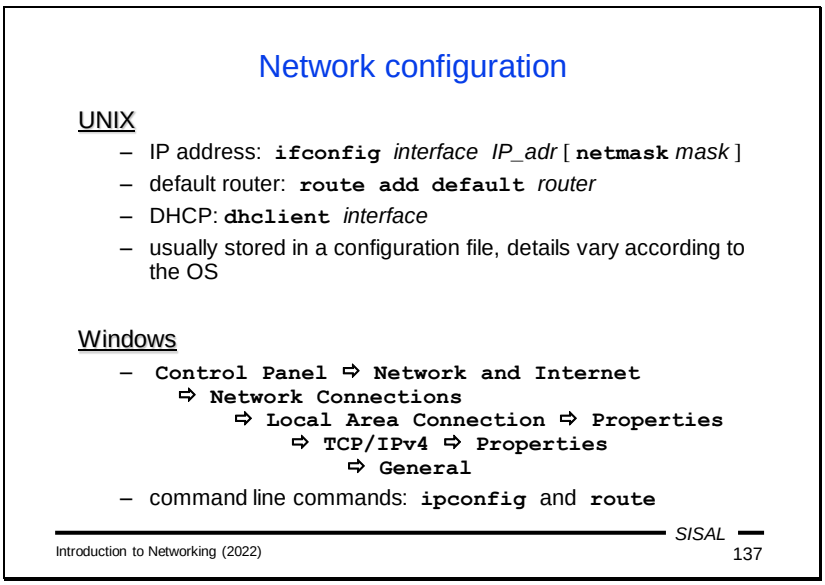
Formerly, entries were categorized by their masks; an entry with a /32 mask was called a *host route*, and an entry with a narrower mask either a *net* or *subnet route*.

Slide 136



We can describe the routing algorithm as follows:

- In the routing table, the algorithm searches for all records that match the packet's destination.
- If no such record is found, the packet cannot be delivered. However, this can only be the case if the routing table does not contain a **default record** (that is, "Other transit" from the analogy with a crossroads). If the table contains one, at least this record will always match. If this situation occurs, the packet is **dropped**.
- The most specific record (the one with the widest mask) is chosen from the records found. The default record, if any, will therefore only be used if the table does not contain another record that matches the target.
- If the record references our own computer (loopback), the packet is placed on the input as if it had just arrived from the network.
- If the record is a direct one, the packet is directly passed to the link layer to be sent to the recipient.
- If the record is an indirect one, the packet is passed to the link layer with instructions to send it to the next-hop router.

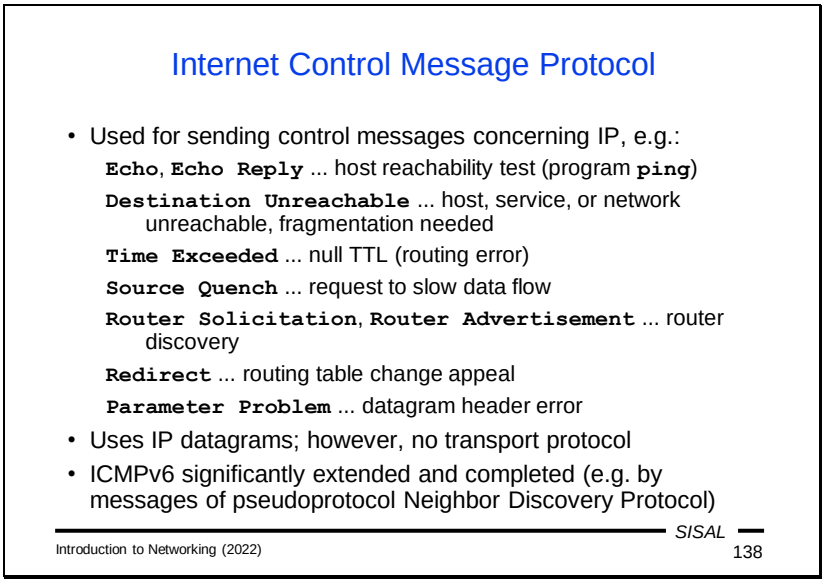


Now we know the most important parameters to specify when configuring a TCP/IP network on a computer. So let's see how it's done.

On UNIX systems, settings are controlled by the following commands:
- a network interface address is set with the **ifconfig** command,
- records in the routing table are changed by the **route** command,
- to perform the configuration via DHCP, you must run the **dhclient** command.

The commands are almost the same on all UNIX systems, but there are administrator tools for each of them that make configuration easier to manage, and these tools vary significantly from system to system.

On Microsoft Windows systems, network parameters are entered in a similar way to other settings, in older systems using the Control Panel, in newer ones using the Settings tool. However, there are also commands that can be entered from the command line (**ipconfig** and **route**).



Before we continue, we need to learn one auxiliary protocol that IP networks use to send out information about different situations, which can then be used to better control the network. This is the Internet Control Message Protocol (ICMP). Its messages are sent as IP datagrams, so in the hierarchy, it can be placed above the third layer of the OSI model, but it cannot be placed at the fourth layer as a transport protocol. IP version 6 has significantly expanded the functionality of ICMP, e.g. it distributes Neighbor Discovery Protocol (NDP) messages that a station can use for detecting the status of the surrounding network.

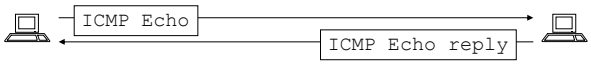
Further on, we will talk about the most important types of messages, so I will only mention the **Destination Unreachable** message at this point. With this message, a station or router announces that it has no way to deliver a received packet and that it discards it. In the case of a router, this is usually due to a failure to find a matching record in the routing table. In the case of a station, this message most often means that there was an attempt to deliver data to a UDP service that is not available (in the case of TCP, such an attempt is resolved using a response with the RST flag at the transport protocol level, but there is no such option in UDP, so this is handled by ICMP).

Slide 139

Ping

- Basic tool for network diagnostics

```
betynka:~> ping alfik
```




```
PING alfik.ms.mff.cuni.cz (195.113.19.71): 56 data bytes
64 bytes from 195.113.19.71: icmp_seq=0 ttl=64 time=0.214 ms
64 bytes from 195.113.19.71: icmp_seq=1 ttl=64 time=0.323 ms
64 bytes from 195.113.19.71: icmp_seq=2 ttl=64 time=0.334 ms
^C
--- alfik.ms.mff.cuni.cz ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.214/0.290/0.334/0.054 ms
```

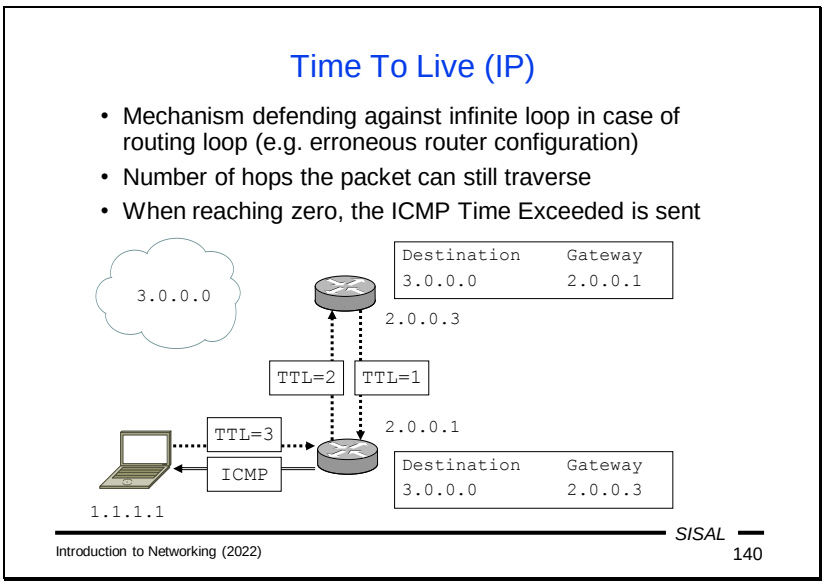
- no need of any special SW on the target machine
- pure network layer reachability, tells nothing about services

The **ICMP Echo** and **ICMP Echo reply** messages are used by the **ping** program, an essential tool for network diagnostics. Using it, we can test the accessibility of a remote network node.

When we run the ping program, it will begin to periodically broadcast ICMP Echo messages to a specified destination machine. When each of these messages reaches the target, its network software will respond with an ICMP Echo Reply. When the answer comes back, ping writes a line with information about how long it took to arrive. The program broadcasts messages with a period of 1 second, until we interrupt it or it has sent a given number of messages. It then prints statistics, i.e. the number and percentage of responses received and the minimum, maximum, average and standard deviation of a value called the *round-trip time* (bidirectional delay).

The important fact is that no special server has to be running on the target machine, ICMP handling is the responsibility of the network software itself. By using ping, however, we will only verify that our packets can travel to and from the target, not that the target will be willing to provide any application services. In reality, the opposite may even be the case: we may be able to make an HTTP connection with a computer even though a ping attempt failed – either the target machine may have ICMP Echo responses disabled in its configuration, or ICMP Echo/Echo reply packets may be withdrawn by some router along the way.

Slide 140



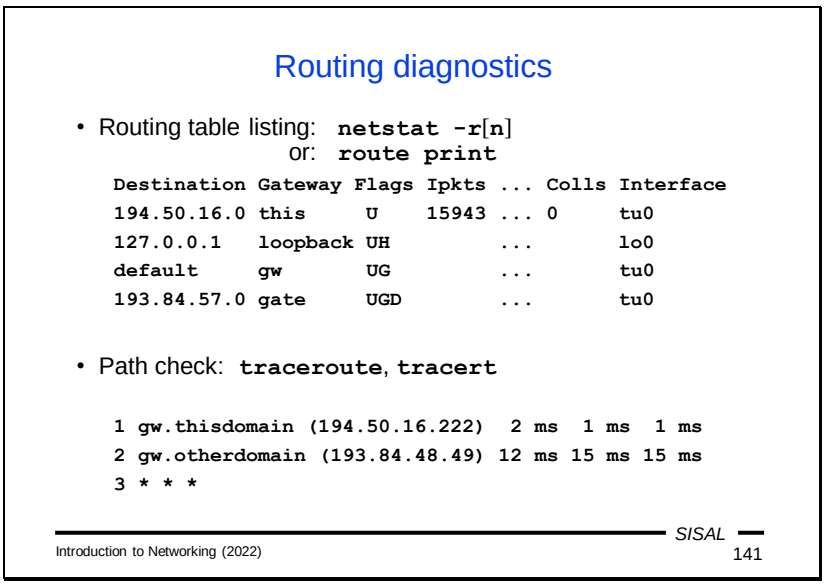
Another important ICMP message is **Time Exceeded**. It is related to the Time-to-live (**TTL**) field in the IP header, and together they are used to prevent packets from running in an infinite loop due to an error in the routing tables.

The situation in the image above resulted in a routing loop between routers 2.0.0.1 and 2.0.0.3 on the way to network 3.0.0.0. Both routers have a record for this network in their routing tables, pointing to each other. This can occur either through an administrator mistake or a software bug. The TTL will prevent this loop from having fatal consequences. It expresses the **number of routers** allowed to forward the packet (the number of "hops").

Imagine that our laptop is about to send a packet to address 3.3.3.3 and that its software sets the packet's TTL to 3 (this setting can be changed by a special function call; the usual default is 64, today). The packet arrives at router 2.0.0.1, which finds that the packet should be forwarded to router 2.0.0.3. And so it decrements the TTL to 2, and since this value is not zero, it actually forwards the packet. A similar procedure will take place on router 2.0.0.3 and the packet will return to router 2.0.0.1 with TTL 1. But now decrementing results in a zero value, so the packet will not be forwarded again; instead, the router **drops** the packet, and sends an ICMP Time Exceeded message to the original sender.

Note 1: Note that in this context, the terms "Time Exceeded" and "Time-to-live" are really **not about time**, but the number of hops! The name is historical. On the other hand, in DNS records, a TTL value really means a time in seconds.

Note 2: **Every** router must lower the TTL value and thereby **modify** the content of the IP header, forcing the router to recalculate the **checksum**. In IPv6, the header checksum was therefore dropped.



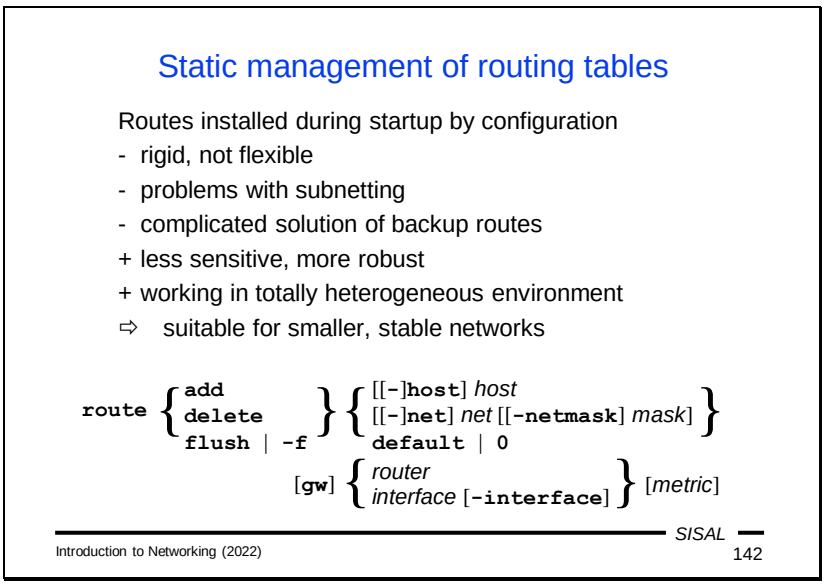
If we need to diagnose routing problems, we usually start by listing a routing table. This can be triggered by a **netstat –r** or **route** print command. In addition to the columns we've already seen on previous slides, the listing above also shows flags (H for *host route*, G for *gateway*, i.e. indirect record, D for *dynamic* record), the name of the network interface, and various statistics (which give the netstat command its name).

Note: The **-n** option prevents the netstat program from trying to translate IP addresses into names. There are two reasons for this: first, we tend to be more interested in numerical values, and second, with this option we do not run the risk that a broken network will fail to translate an address and we will learn nothing.

The next diagnostic step is likely to be **ping**, but it usually doesn't help. In fact, if an answer is not returned, there might be various reasons for this and we may not be able to distinguish them. First, our packets may not have reached their destination. Second, they may have arrived, but the target may not be functional. But it is also possible that the packets were lost on the way back. The routing tables work **in one direction**! It is again similar to our car example – the fact that we are able to travel from Česká Lípa to České Budějovice by following direction signs doesn't mean anything about whether we'll find the way back.

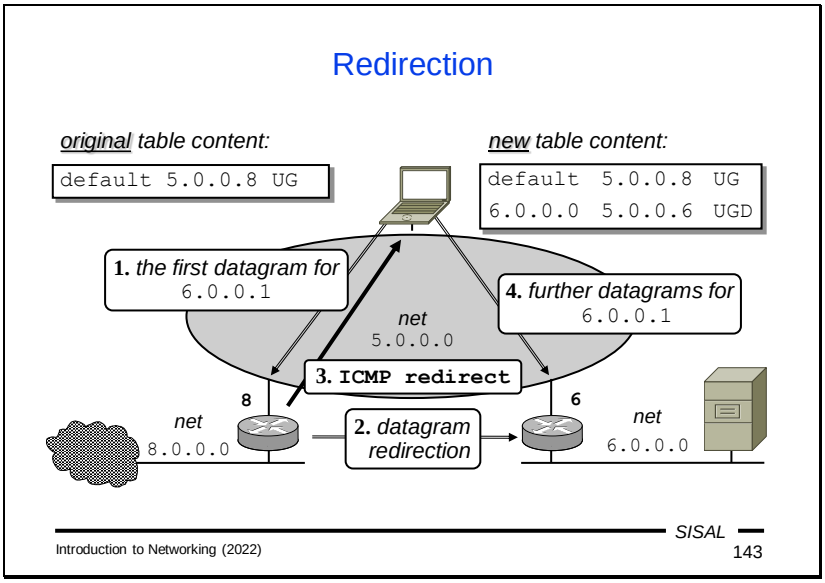
A better diagnostic tool is the **traceroute** command, which exploits the properties of the TTL field. The program first sends a packet with TTL 1 to the destination. As a result, if the target lies behind at least one router, the packet won't reach the destination because the router will refuse to forward it and will send back an ICMP Time Exceeded message. The traceroute program will capture it and display the address (and possibly the name) of the router that sent it, including the round-trip

time. The program performs this attempt three times and then starts sending exactly the same packets, but with the TTL increased to 2. These packets now pass through the first router, but are stopped by the following hop. In this way, the program will gradually disclose the structure of the path to the destination. The process will either end in success, or responses will stop coming back at a certain TTL value. The last router that responded to us will then be the starting point of our investigation – either the error is directly at it, or at the next router, or at the previous one (in case the last responding router should never have received our packets).



Now we know how a computer uses a routing table, but we don't yet know how the content of the table is managed.

The simplest approach is the **static method** of table management, in which a computer has data for all the needed records stored somewhere, and adds them to the table one by one after it boots. The advantage of this method is its stability – if the network does not change often and is not too complex, this method guarantees proper functioning in all circumstances and with different types of nodes. This method is used, by the way, when we plug our computer into a network and it gets the address of the **default router** of the local network through DHCP. That's all it needs for routing. But if the network is too large or changes dynamically, this approach is not appropriate.

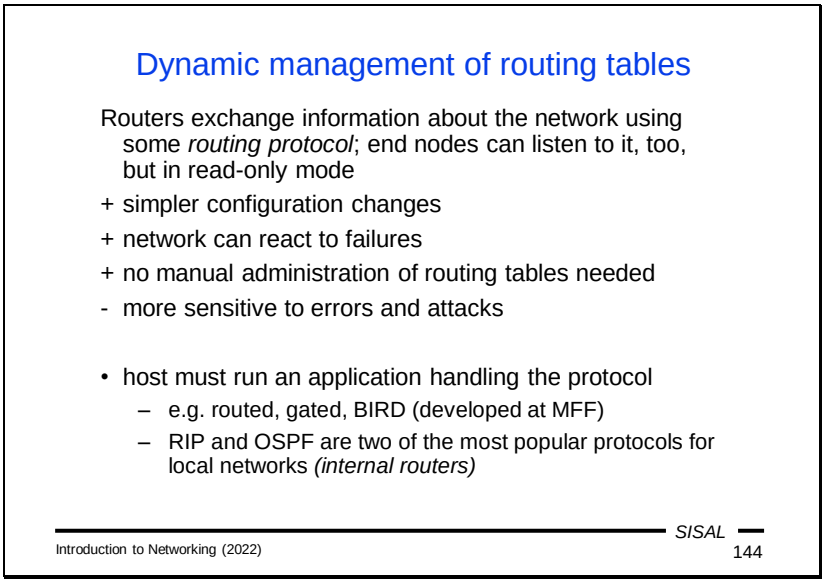


One way to cover even a more complex network, with static routing tables, is to use another type of ICMP message, namely **ICMP Redirect**.

In the LAN in the image above we see network 5.0.0.0 with the default router 5.0.0.8. This default route is distributed to every station, so we can see it in the routing table of our laptop. But there's another network 6.0.0.0 plugged into the LAN that our laptop doesn't know about yet. What happens if we try to send a packet to this network?

- Following its routing table, our laptop sends the packet to router 5.0.0.8.
- The router accepts the packet and is going to forward it. But it finds that it has to send the packet **back to the same** network it came from, only to another router 5.0.0.6. This indicates an error on the sender's side, as it could have sent the packet to router 5.0.0.6 directly. So router 5.0.0.8 redirects the packet to the correct router, but...
- …it also sends the original sender an ICMP Redirect message telling it to add a new record for network 6.0.0.0 to its routing table.
- If our laptop makes the change, subsequent packets for this network will now be sent to the correct router. In the routing table, we can then see a new record for this network, with a D flag (for a dynamically added record).

While ICMP Redirect solves other similar situations, it may also cause problems. If someone (intentionally or by software error) disseminates defective ICMP Redirect packets on the local network, they may cause the network to malfunction. It is therefore usually better to distribute the full network structure to clients and ban the reception of ICMP Redirect.



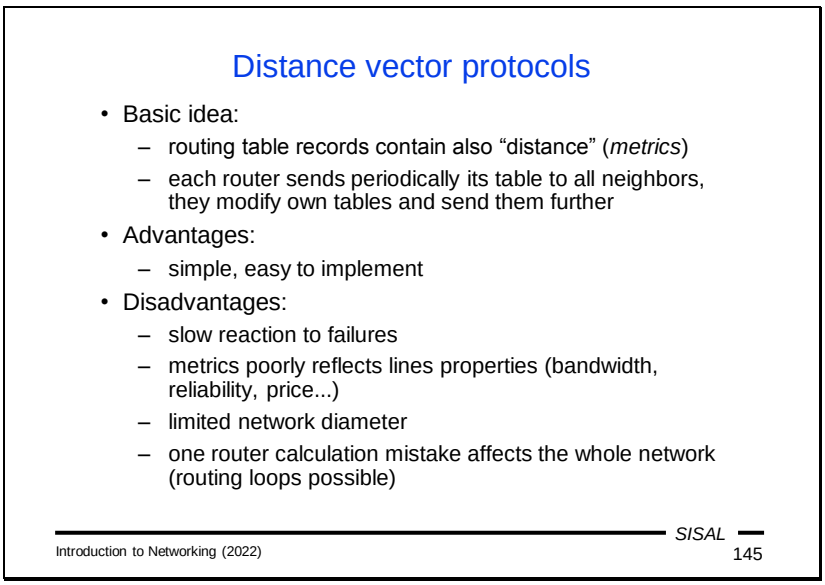
The solution for more complex networks is a **dynamic** method to manage tables. This is based on the information exchanged between neighboring routers used to adjust their routing tables. The routers communicate with each other using one of the **routing protocols**. Ordinary stations can also follow this information, but only in passive mode.

A network so controlled can adapt itself to current conditions, and configuration changes can be made centrally in one place. A small cost of this approach is a certain load on the network from routing protocol messages and a greater vulnerability of the network to errors caused by software attacks or errors.

If a node (router or station) wants to be involved in routing protocol communication, special software must be running on it. One highly successful routing program, **BIRD**, which is now used on a number of key routers in the world, was originally developed as a **student project at our faculty**.

Routing protocols for local networks are divided into distance-vector protocols (e.g., RIP) and link-state protocols (e.g., OSPF).

Slide 145

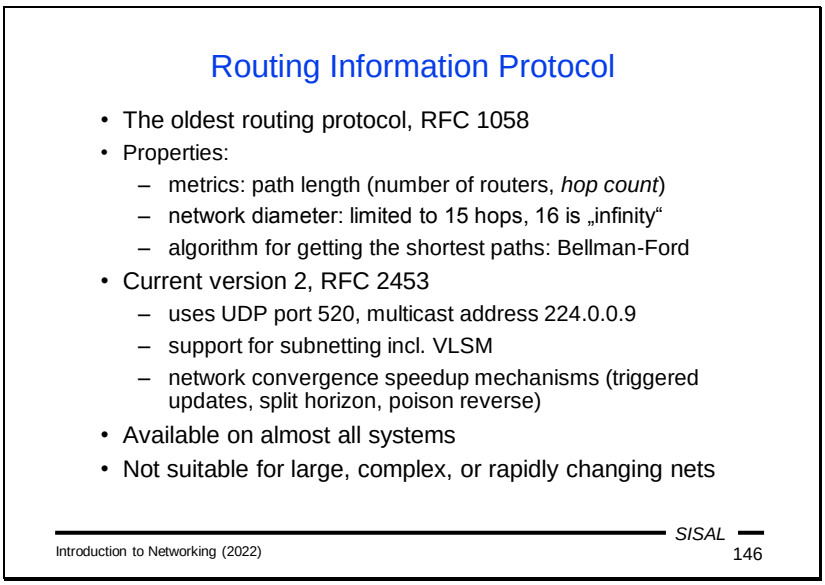


The basic idea of **distance-vector** protocols is something we can easily imagine in our crossroads example again. Direction signs also contain distances in kilometers, so if we regularly send photos of our crossroad signs to neighboring crossroads, administrators there can easily determine whether there is a path to a given destination via our crossroads, better than the one they have on their signs now. If so, they simply turn the sign in the right direction and write a new distance on it.

And it works similarly with routers – they periodically send their routing tables to their neighbors, who check to see if they can change any of the records in their routing tables.

The advantage of these protocols is relative simplicity and easy implementation.

However, there are some drawbacks such as a slow response to errors, an insufficiently fine evaluation of individual paths (we know only the number of kilometers to a given target, but not the quality of the path) and limited network range. But the absolutely fundamental problem is that if a router makes a **miscalculation**, it will **spread** it and may render the entire network inoperable.



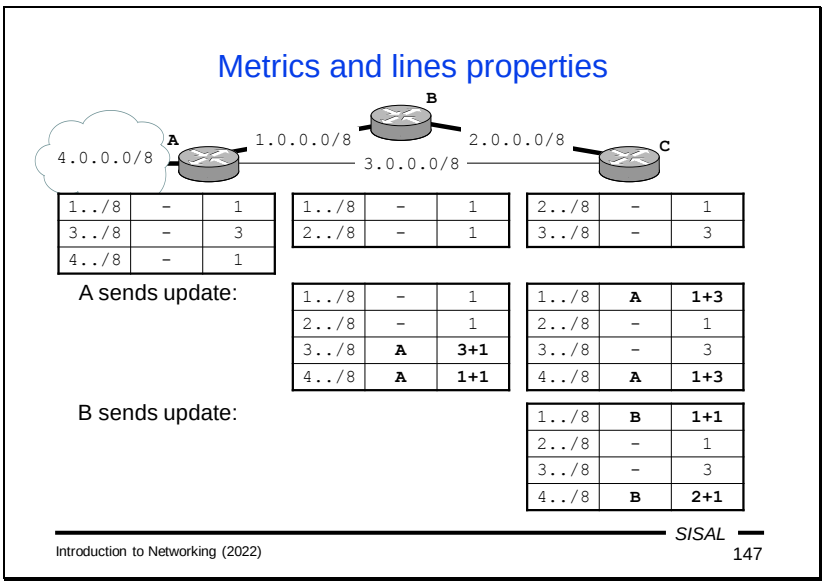
The best-known distance-vector protocol is RIP (Routing Information Protocol). It belongs among the oldest protocols, which makes its implementation very widespread, which is its main advantage.

At the same time, the era of its creation affected some of the decisions in the proposal. The role of **metrics** ("distance") is played here by the number of routers (hops) on the way to the destination. However, that number is limited to only 15 in the proposal, with unpleasant consequences that we will discuss later. These consequences are partially eliminated by the extensions added in version 2. Despite this, RIP is not suitable for large networks or networks, the structure of which often changes.

In every article about RIP, you'll find a note that it uses the **Bellman-Ford** algorithm to calculate the shortest paths, but there's almost no explanation for why that's the case and why Dijkstra's algorithm which is generally faster, isn't used instead. The reason is that by its very nature the Bellman-Ford algorithm actually "replicates" the incremental correction of calculated distances based on new information from neighboring nodes. Simply said, the arrival of a new message from our neighbor's routing table will trigger the calculation of a **single** step of the Bellman-Ford algorithm, whereas with Dijkstra's algorithm we would have to repeat the **whole** calculation.

Interestingly, version 1 of RIP did not support variable length masks (VLSM) networks because no mask was included in the packet format. Version 2 now includes a mask in the format, despite the fact that the packets have not lengthened in any way and are backward compatible – space that was unused in the version 1 format was simply used for it.

Slide 147



The number of hops along a path, unfortunately, does not reflect the properties of individual segments well. It is again similar to the situation on a road – if we can reach a city by travelling 80 km by road or 100 km by highway, the longer journey will be quicker, but if the driver only decides based on the number of kilometers, he will take the slower route. In the picture above, routers A and C are linked by a direct line (the number of hops between them is therefore 0) which is "slow“, and also by a "fast" line which runs through router B and therefore has a metric with one hop more. RIP will therefore prefer the slower line with a metric of 0. This can be prevented by artificially penalizing the slow line with higher metrics than the number of hops would normally indicate. In our example, we have "added two routers" to the line.

Now let's see how the network situation will change as the routers exchange information (send updates of their tables).

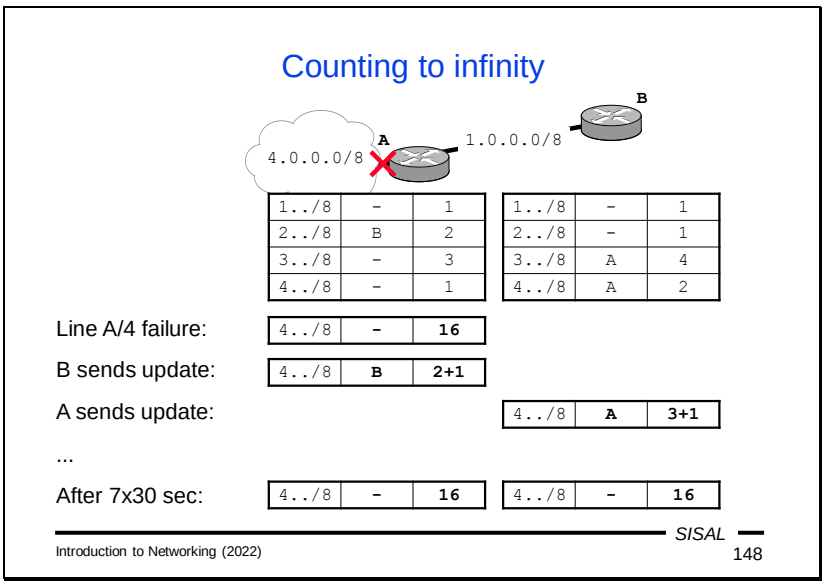
- In the beginning, all routers have initialized their tables with values for directly connected networks. These will all have a metric of 1, except for the line between A and C, which will have its metric artificially raised to a value of 3.
- Let's say that the first update is sent by router A. The table arrives at both routers B and C, which add records for paths to the new networks included in the RIP messages. When calculating a metric for the new networks, they take the metric in the message and add a metric to the network from which the packet arrived. Thus, router B will add 1 to all the values, so it will add network 3.0.0.0 with a metric of 3+1 and network 4.0.0.0 with a metric of 1+1 to its table. Router C has its metric to the network 3.0.0.0 (from which the message arrived) set to 3 and will therefore add both new records 1.0.0.0 and 4.0.0.0 with metrics of 1+3. The directly connected networks of both routers have a metric better than the one that was received, so those won't change.

- Next, router B sends its update. Router C will receive its packet over a network that has a metric of 1, so it will add one to the values in the message. Network 1.0.0.0 had a reported metric of 1, adding one gives us a value of 2, which is better than the current value of 4. So router C modifies the record in the table, changing the direction to router B and metric to 2. Check of network 4.0.0.0 has a similar result: the new metric 2+1 is again better than the existing 4, and the record is corrected. Directly connected networks remain unchanged. This form of the table on router C is now final, unless there is some change in the network.
- The process concludes with an update sent by router C, which allows the remaining routers to finish changes in their tables.

Since routers send initial messages with the contents of their tables right after they boot, this network converges very quickly.

We have solved the problem of varying line quality elegantly here, but only because this network is very small and the lines have only two different "speeds" (bandwidths). Once we have a more complex network, we will very quickly hit the ceiling of the maximum metric of 15. The logical solution would be to fundamentally raise that value.

Slide 148



Why is RIP's "infinity" so small? The reason is to try to minimize the impact of a problem called **counting to infinity**.

Let's imagine that there's a connection failure to network 4.0.0.0 on router A. The router responds by setting this network's metric to infinity (16). But before its period elapses to send out its routing table update to distribute information about the unavailability of the network 4.0.0.0, it receives an update packet from router B, which still has the network with metric 2 in its tables. Router A makes a calculation and finds that the "new" route to network 4.0.0.0 via router B has a metric of 2+1 < 16, and so “corrects” it in its table. This will remove information about the unavailability of the network.

Once the period for router A's update elapses and it sends its table to everyone, router B will be informed that network 4.0.0.0 is now available via router A, but with a metric of 3. Since the existing entry on router B also originates from router A, router B must respect the change in the metric on router A, and correct the network 4.0.0.0 record to the new metric of 3+1.

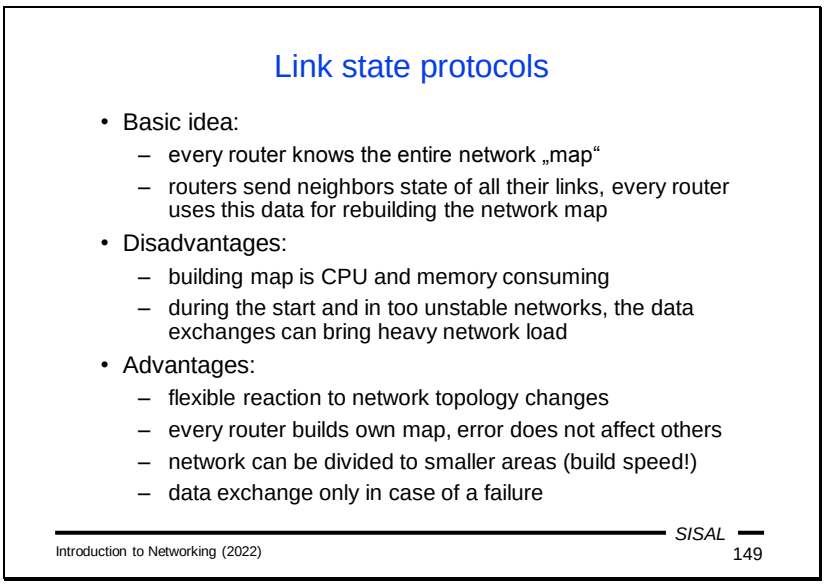
With subsequent exchanges of update packets, the metric on both routers gradually increases until it finally returns to the correct "infinity" value. Due to the length of the period (30 sec), for over 3 minutes the network 4.0.0.0 appears to be alive, although it is not really available.

If the "infinity" were significantly longer, the service disruption would be significantly longer as well.

At the same time, this problem also illustrates one important principle in designing algorithms: there is always a need to investigate and prevent possible **race conditions**, i.e. situations where two independent actions can occur in an inappropriate order.

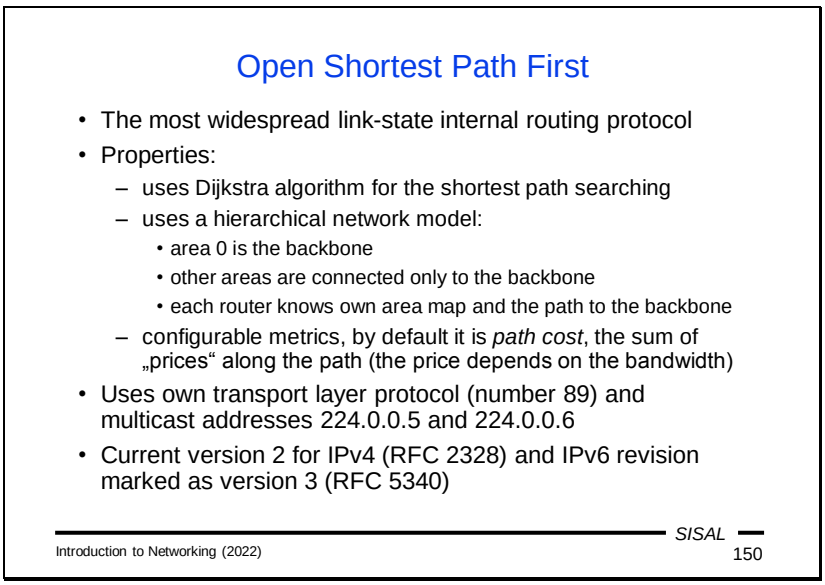
The negative effects of counting indefinitely are minimized in the RIPv2 extension:

- A *triggered update* is a mechanism by which a router sends an update **immediately upon** detection of a problem and does not wait for the update period to elapse. This will reduce the risk of a race condition that an update from a partner will arrive earlier. But beware, this risk will only be **reduced**, not **eliminated**, by this mechanism!
- In the *split horizon* mechanism, a router does not send a partner information about networks that it has learned about **from the same partner** (which would make no sense – the partner has better information about these networks than it does). This mechanism effectively **prevents** the race condition described above. A small tax on this is that the router cannot send the same update message to all its neighbors, but must prepare each one separately.
- A *poison reverse* is a slight improvement on the split horizon method – a router sends a neighbor data about the “neighbor’s" networks, but with a metric of 16, it "poisons" these records.



The second group of routing protocols are called **link-state** protocols. The name is based on the basic principle of these protocols – instead of a table with distances, only information about line states is sent. Each router holds the entire **network map** on its own and calculates optimal paths on its own, according to the line status messages it receives. If we use the car analogy again, in this case the driver has at his disposal not direction signs but a road map. If he learns about a problem on the radio, he can find the best alternative using the map on his own.

Even though this approach requires more computing power and (in the case of rapidly changing conditions) a certain network burden, benefits outweigh the disadvantages. Two of the benefits are substantial: the network can react far more **flexibly** to changes or outages, and each router makes its own calculations, so any mistakes **won't affect** any of its neighbors. On the contrary, the advantage for a stable network is that routers do not have to regularly exchange their entire databases.



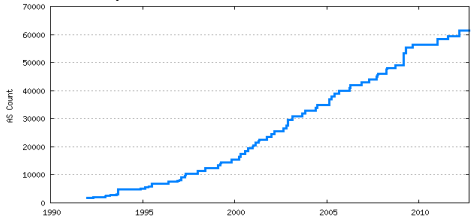
The most popular representative of link-state protocols is OSPF (Open Shortest Path First). Here, unlike RIP, **Dijkstra's** algorithm is actually used to calculate the network map. However, for large networks, even this algorithm would have problems with increasing network size and therefore an OSPF network can be divided into **areas**, so that each calculation is always done only within a single area and thus on a significantly smaller set of nodes. The network is arranged in a two-tiered schema: the area with number 0 is called the **backbone**, and all other areas are connected directly to it. Thus, any path in the network consists of no more than three parts: from the source area to the backbone, across the backbone to the target area's connection point, and across the target area.

The metric is called **path cost** in the OSPF, and is determined using a complex formula that is defined by the network administrator in a configuration. The path cost is able to include not only "distance" but also bandwidth, latency, throughput, and also the actual "price" of traffic along a line.

Slide 151

Autonomous systems

- Definition: network block with common routing policy
- Established in 1982: easier routing at the global level, using of *External Routing Protocols* (EGP)
- The most common EGP: Border Gateway Protocol (BGP)
- AS Identification: 16bit number, since 2009 moving to 32bit
- In the Czech republic: starts with 2, hundreds today

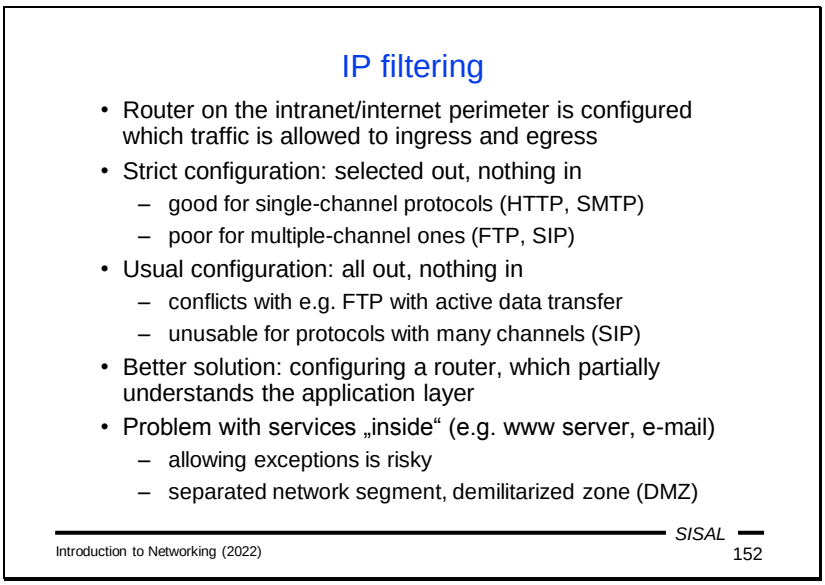


SISAL
Introduction to Networking (2022) 151

Until now, we've been talking about routing protocols in "relatively close" networks. But how does that change when we look at the Internet backbone?

Individual blocks of networks consist of so-called **autonomous systems** (AS). The exact definition is not very exact, it basically just means networks with a common routing policy, so it's not all that difficult to apply for an AS allocation, and so the quantity of AS numbers has increased rapidly. When the Czech Republic joined the Internet, it had two autonomous systems, and they now number in the hundreds. Typically, AS are owned by ISPs or large companies. In 2009, it was necessary to switch from the previous 16-bit AS numbers to 32-bit ones.

The purpose of autonomous systems is to unify routing for a whole group of networks at a global level. Routing between the different AS's then takes place on the basis of slightly different conditions than the one that takes place within each AS. It is not the network addresses that play a major role here, but the AS numbers. The protocols we have studied so far are called *internal routing protocols* (IGPs); so-called *external routing protocols* (EGPs) are used to manage routing between autonomous systems. The most common representative of an EGP is the **Border Gateway Protocol** (BGP). The critical features of an EGP are the inclusion of other factors in the evaluation of path metrics (similar to the calculation of path cost in the OSPF) and the avoidance of loops. Therefore these protocols use a **path-vector**, a sequence of AS numbers through which the path leads. Work with it prevents loops.



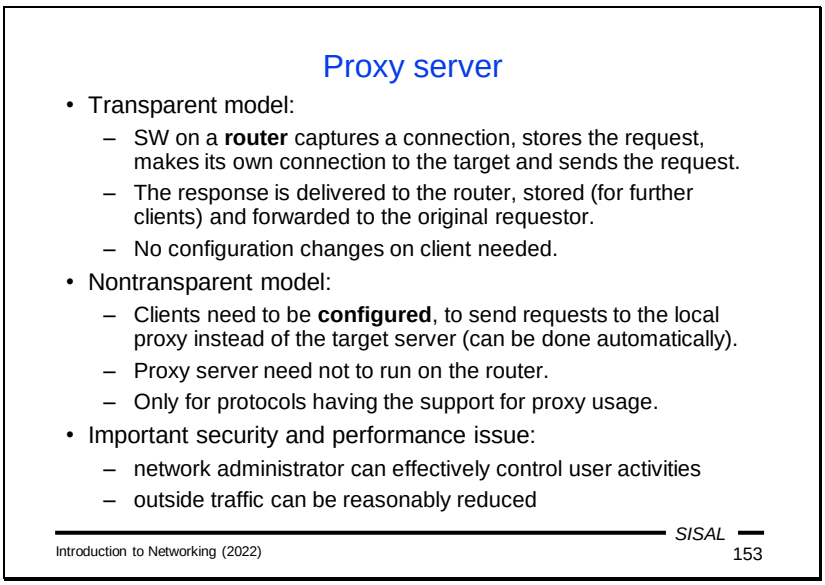
A router that connects a local network to the Internet is the point where the **security policy** of the entire network is applied. **IP filtering** is usually a primary part of such a policy. The name is somewhat misleading because it is not really filtering at the **IP layer**, but on the **transport layer**. It defines rules on what type of traffic (more specifically, traffic on "which ports") is allowed to and from the local network.

Note: The question is, what does “from" and “to" actually mean? In the case of TCP, of course, packets have to cross the border in both directions. What matters is who **opened** the connection (and therefore sent the SYN packet). In the case of UDP, we generally don't know the direction.

The most trivial configuration only allows traffic from the internal network to the Internet, and only on selected ports. This configuration is only acceptable for single-channel protocols such as HTTP or SMTP. When a protocol needs additional channels for its operation, the filter prevents opening them. The only option in this case is to use a filter that also understands that particular application protocol. For example, if filtering software intercepts an FTP communication in which a server and client are negotiating to open a data channel, it can “make a hole in the filter" for the two specific addresses, one inside and one outside the network, for a limited period of a few seconds so that the data link can be established.

A more typical configuration is not so restrictive regarding the list of ports that local clients are allowed to access. With such a configuration, for example, passively opening data channels in FTP will no longer be an issue. The issue will remain only with active channels or where more channels need to be opened (e.g. with SIP). Here, we can no longer get around without working with the application layer.

Another problem for filtering is providing services for the public Internet. In the past, this was a common problem, e.g. for a web server. Today, web servers are usually run by various **hosting** houses. However, if we want to run a server or other service on our own network, we will need to open a permanent hole in the filter allowing traffic from the external network to access a specific server and port. This obviously poses a risk, as once a potential attacker has an access into the internal network, he or she can exploit any software bug or server configuration error. Therefore, it is common to make a special segment of the network, a so-called *demilitarized zone* (DMZ), for such services. From the point of view of protecting the network, we consider it as a world that is neither completely safe nor completely unsafe. Therefore, filtering rules at the DMZ's borders with the internal network and the Internet are slightly more permissive than those at the border between the internal network and the Internet.



Software that controls the operation of a particular protocol (usually on a local network/Internet interface) is called a **proxy server**. It operates either transparently or non-transparently.

A *transparent* version of a proxy server works on a network's border router. The router intercepts a client's request and hands it over to the proxy server. The proxy server checks the request against the rules of the organization's security policy, and if the request is okay, it establishes a connection to the actual destination server as a client and sends the request to it. Once a response arrives, it is checked again (e.g. by an anti-virus program) and sent to the client. The advantage of a transparent proxy is that there is no need to change the configuration on the clients – each request reaches the proxy without the client having to know about it.

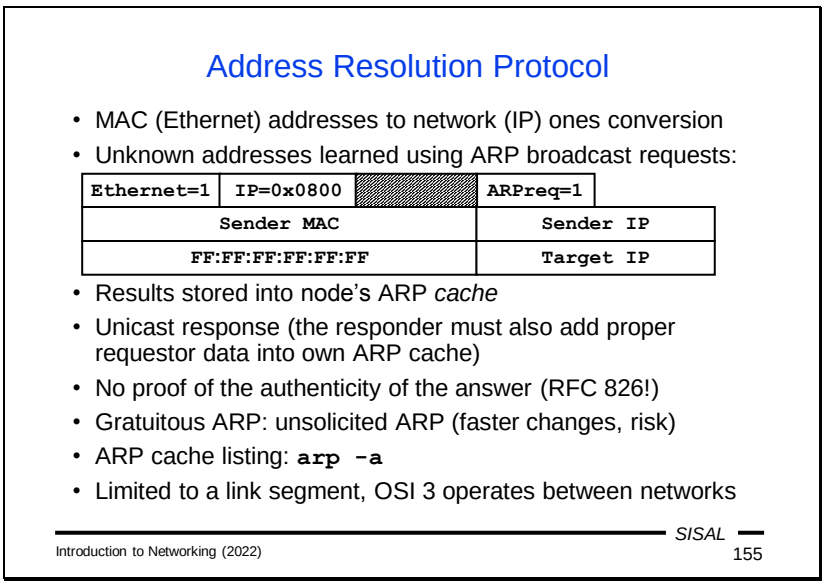
In the case of a *non-transparent* proxy, a client needs to know about the existence of the proxy because each request needs to be sent to the proxy server and not to the target server. A disadvantage of this solution is therefore that clients need to be properly configured (manually or automatically). An advantage is that the proxy server can run on a more suitable hardware or operating system than the router. But this solution is only available for those protocols that have support for it – the client must specify when sending the request that it is targeted to a proxy and tell the proxy the real target server address.

Using proxy servers has also security benefits (the proxy server can filter operations at the application protocol level) and performance benefits (the same request does not have to be sent repeatedly by the proxy server to the actual server; the response can be **cached** and sent to other clients by the proxy). However, the last point may

gradually become less relevant for HTTP, as caching cannot be used together with HTTPS.

Summary 7

- How does the routing algorithm work?
- Describe the types of columns and records in a routing table.
- What makes static and dynamic routing table managements different?
- What are routing protocols for?
- Compare distance-vector and link-state protocols.
- What is an autonomous system?
- Explain the meaning of the ICMP Echo, Time Exceeded and Destination Unreachable messages.
- What is the TTL field in the IP header for?
- What does IP filtering mean?
- What's a proxy server for?



The Address Resolution Protocol is an ancillary technical protocol that represents a connection between the network and link layers. It allows nodes on a network to discover data link (MAC) addresses corresponding to specific network addresses. It's a generic protocol, it can be used for any network and data link addresses, but we'll be demonstrating its use on Ethernet and IP addresses.

We already know that the link layer gets a request from the network layer to deliver data by a directly connected segment of the network, either to the target machine or to the nearest router, i.e. a target node at the data-link layer. So it needs to correctly fill in a destination MAC address corresponding to a specific IP address in its data-link header. To find this MAC address, it sends a frame to the target node with an ARP query. However, it does not yet know the destination MAC address, so it will use the **broadcast** MAC address (FF:FF:FF:FF:FF:FF). This causes the frame to be delivered to all nodes on a given data-link network segment, but the frame will be ignored by everyone except the queried IP address holder.

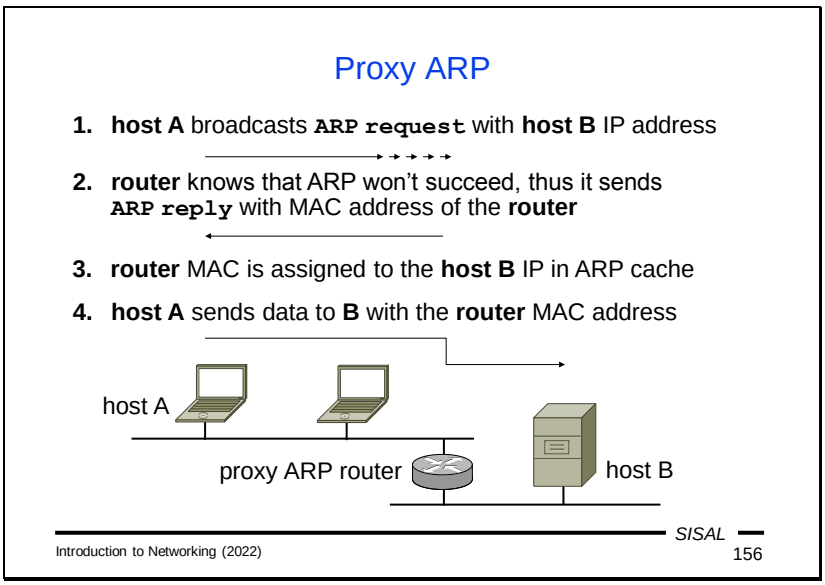
The node (i.e. an ARP server for this moment) responds to the query with a unicast ARP response containing the requested MAC address. The ARP client saves IP–to-MAC address assignments for further use in an **ARP cache**, in which a record lasts for a configurable time (of the order of minutes). The server will, however, perform the same caching operation. It is assumed that the communication that has just started will continue and the server would soon have to trace the client's address by itself. Therefore, the assignment of the client's IP and MAC address will be recorded in the server's cache, too. And that's why we can see machines in our ARP cache that we haven't communicated with – such hosts have simply sent us an ARP query. We can list the contents of the ARP cache with the **arp –a** command.

The ARP protocol (and thus the content of the ARP cache) is limited by the scope of the local data-link (OSI 2) network segment – we need to climb a layer up for communication outside this segment. Incidentally, this also means that network interface cards (NICs) with the same MAC address can be used on the same LAN if they cannot "see" each other, i.e. are separated by a router. Indeed, it is often the case that a router has only a single MAC address on all its network interfaces (although they have different IP addresses, of course).

A problem with ARP is its lack of security, i.e. that the broadcast query will reach everyone, so any node of the network can answer us. Worse, a potential attacker doesn't even have to wait for our request. An **unsolicited** ARP message (a so-called *Gratuitous ARP*) is possible, which is actually an answer without a prior query. Such messages are used e.g. for **cluster** solutions – important machines in the network can run redundantly, both sharing the same IP address. An active machine of the pair informs other machines on the network using gratuitous ARP about its MAC address as the one that they are currently expected to use for the shared IP address.

Some increase in security in certain types of networks can be achieved by **denying** certain important nodes (servers, routers) from using the ARP protocol and fixing their ARP cache content by a configuration.

Slide 156



In a complex LAN, network management can choose not to disclose details about subnets to all stations on the network, leaving the correct routing purely up to the routers. Let's imagine the simplified situation in the picture above: there are two subnets in the network, but the stations are not told this information and are sent a network mask that corresponds to the **entire** network. If host A wants to communicate with host B, according to the network mask, it believes they're on the same network, so it sends an ARP query. But as we know, a broadcast query only spreads across a data link segment, and therefore over the **subnet**, where computer A is connected, so it never reaches host B. For the entire network to work, we need to run an **ARP proxy** on the router that splits the network. The router with an ARP proxy intercepts the ARP query and, recognizing that the client would never receive a response, it sends a reply instead of the target host, returning the **router's MAC address** as the query response. The client will save this assignment to its ARP cache and use the router's MAC address for further communication with host B.

It may seem strange, but if we think about it more carefully, the client will actually behave in **exactly the same way** as if it had the right information about the netmask and knew about the router – if it did, it would also send packets for host B with the MAC address of the nearest router!

A user on the station can tell the difference by looking at the ARP cache. He will find **multiple IP** addresses with the **same MAC** address. If we have an ARP proxy in the network, this situation is expected. But if not, multiple occurrences of the same MAC address in the ARP cache may signal an attempt to attack our ARP cache using fake ARP messages.

Data link layer (OSI 2)

- Separated into two sublayers:
  - Logical Link Control (LLC) multiplexes various network layer protocols approaching a media
  - Media Access Control (MAC) controls addressing and media access: who, when and how may send and receive data
- TCP/IP does not deal with this layer („network interface“)
- Network segment (physical network):
  - set of nodes sharing the same media
- Data link layer PDU: frame
  - format depends on the media used
  - in general: synchronization field, header (addresses, type, control data), data field and trailer (Frame Check Sequence - error detection)

For us, the ARP protocol represented a move from the network layer to the **data-link** layer, and thus a move away from TCP/IP as well.

The link layer represents a very important milestone; you could say that this is actually a step from software to hardware. Unlike some other layers of the OSI model, it performs a lot of work, and we are actually dividing it into two sub-layers:

- The upper sub-layer is called *Logical Link Control* (LLC) and is in charge of **multiplexing**. It is thus responsible for correctly storing data of various network protocols and identifying it so that the receiving data-link layer software is able to forward it to the corresponding network protocol software.
- The bottom sub-layer is called *Media Access Control* (MAC) and handles both the **addressing** and **access control** of individual nodes to physical media they share within the same line (physical) segment of the network.

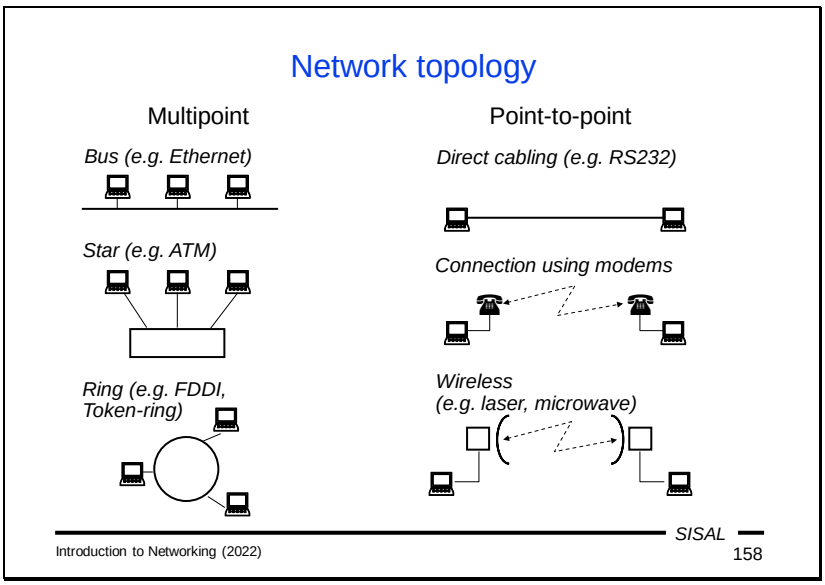
The MAC sub-layer is directly dependent on the underlying physical layer technology, and we will deal primarily with two of them, Ethernet (which now dominates cable networks) and wireless (WiFi) technology.

The data unit of a line protocol is called a **frame**, and its exact format may vary from one protocol to another, but it generally includes:

- A *Synchronization Field* – a special sequence of bits designed to "wake up" a target node, to distinguish subsequent data from “noise“. This field does not usually count as part of the actual contents of a frame.
- A *Header* – an initial part of the frame, which contains at least the MAC addresses of the recipient and the sender and LLC control information.
- *Data* (the payload) of a network protocol.

- A *footer* – a final part of the frame, which usually contains a so-called Frame Check Sequence (FCS), a value used to **check the accuracy** of delivery. In fact, the data-link layer is the last layer working with data above the physical layer, so it is desirable to check that the physical layer has transferred the data in order when it reaches the target node.

Slide 158



One of the criteria for classification of various technologies of the lower two OSI layers are their topology schemas (arrangement of nodes). A network topology can be studied from either a **physical** perspective, i.e. how the nodes are actually connected (e.g. by cable), or a **logical** one (how the nodes communicate with each other).

Technologies that can connect multiple nodes within a one network segment (**multipoint**) typically use one of the following topologies:

- A *bus* – all nodes are connected serially to the same media. They can all (more or less) simultaneously receive a signal that spreads across the medium (which is fine), but also they can all try to send a signal at the same time, which is more problematic. This situation is called a **collision**, and the protocol must address it. The physical bus topology of Ethernet networks built by coaxial cable was very popular in its day, with the great advantage that adding a new computer to the network could be done at virtually any time and anywhere, but its main disadvantage was that if the cable broke down anywhere, the **whole** network broke down.
- A star – one segment contains a central device and individual nodes are bound to that central element. This is the most common physical topology currently used by Ethernet networks. The term **structured cabling** is often used for it and stations are usually connected by a UTP cable. Logically, however, Ethernet's topology remains **bus-like**, but the bus is concentrated in that central device, where it is sufficiently physically protected, while damage-prone cabling links only the terminals, so that only the affected station is inoperable in the event of a problem.
- Circle - the individual nodes are connected to a circle. Token-ring and FDDI technologies, for example, work in this way.

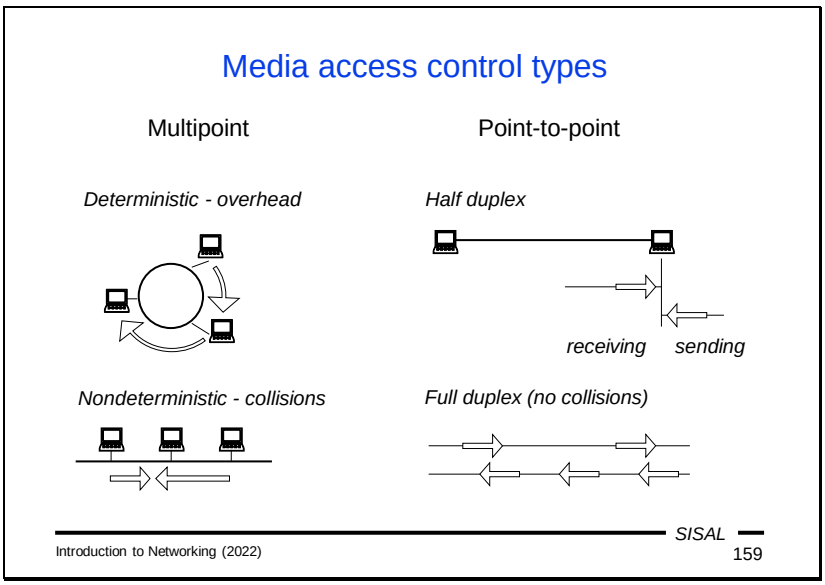
An alternative to multipoint technologies is technologies using point-to-point topology, such as RS 232. With them, only two nodes can connect to each other, so the address on the line protocol can be significantly simplified. But if we're talking about a physical point-to-point topology where we only connect two nodes together, for example, with an Ethernet cable, the same protocol as in multipoint wiring will still be used.

The point-to-point range of technology can be extended by replacing part of the line with another technology. A possible solution is to hook up a device (modem) to your computer that modulates data traffic so that it can be transmitted using a telephone connection.

The latest group of technologies using the point-to-point model are wireless, high-range links based on laser or radio waves. However, what one now typically envisions as a "wireless network" is another technology that actually works with a star topology.

Note: The central element of a star in a structured wiring tends to be a switch, and individual stations connect to it with a cable tucked into one socket. The drawer is commonly called a port. Be careful - don't confuse this term with the term port on the transport layer, it's a homonym! From now on, in my part of the lecture, we will use this word only in this new sense.

Slide 159



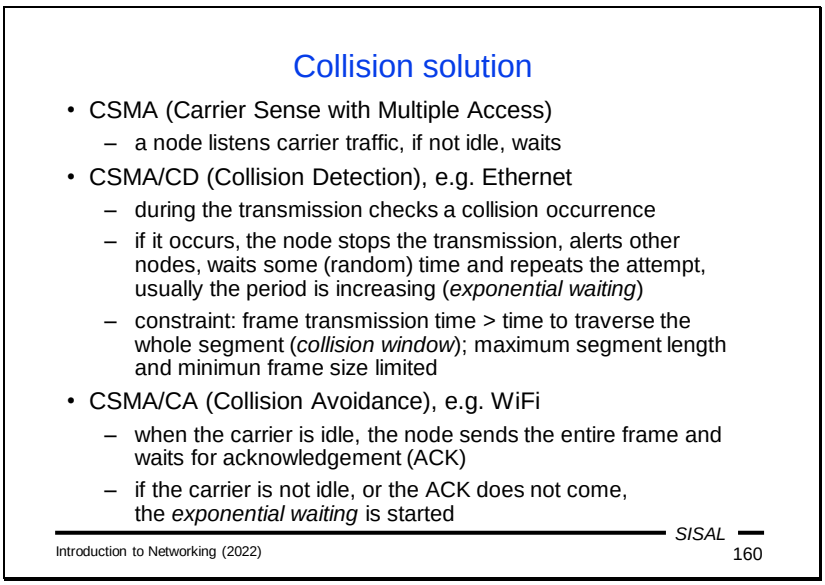
Network topology largely determines the capabilities of a given technology in terms of managing the access of nodes to a medium, so that the technology can cope with parallel transmission requirements.

Two basic approaches are used for multipoint topologies:

- In a *deterministic* means of control, there are no problems. Someone or something determines who is allowed to broadcast at any given moment. The downside is that if the "next" node has nothing to broadcast, its broadcast frequency will remain unused, increasing overhead and reducing network capacity.
    - In some networks (e.g. Token-ring), the controller role is played by a specific piece of data (a *token*) running from node to node. If a node wants to transmit, it waits for the token and sends its data over the network instead; the recipient deletes the data packet and sends a token back to the network.
    - In other technologies, there is a special control node in the network that sends a signal to other nodes when they have the right to transmit.
- In nondeterministic control (e.g. Ethernet), no one limits nodes in the broadcast, so **collisions** have to be dealt with subsequently (see the next slide).

With point-to-point topologies, we distinguish whether a node can simultaneously receive and transmit.

- If the node can't do this, it will work in the so-called *half-duplex* mode. If we plug in an Ethernet network this way, collisions will normally occur on it.
- If both nodes can handle both input and output simultaneously, we can set a *full-duplex* mode. For example, on Ethernet, this avoids collisions for a given segment, radically increasing throughput.



Networks that allow multiple access must somehow address when multiple nodes at once intend to transmit. The basic principle is that a node checks the "**carrier**," the transmission medium, for any current transmission. This step is called “carrier sense”. If the carrier is not free, the node is waiting. If the carrier is free, it can start transmitting. However, it is obvious that such a check does not guarantee that there will be no “multiple access” attempts. That's why there are different extensions that try to deal with **collisions**.

Ethernet uses the *Collision Detect* method. During broadcasting, a node checks the carrier at the same time, so it is able to **detect a collision**. If it occurs, the node will stop the broadcast, alert other stations on the network to the collision and start waiting for a new attempt. The wait time must be chosen at random from a certain interval to reduce the risk that the two nodes that detected the collision will wait the same amount of time and repeated transmissions will collide again. However, repeated attempts must not overwhelm the network, so the wait is **exponential**, i.e. the center of the interval from which the wait time is selected doubles with each subsequent attempt. However, a condition of the success of this method is that the frame broadcast time must be longer than the frame propagation time from one end of the segment to the other (the so-called *collision window*). If this condition is not met, it could be that a node completes broadcasting an entire frame before a conflicting frame arrives from the other end of a segment, the node does not detect a collision, does not do a repeated send attempt, and the frame is lost to most other nodes. The collision window determines both the maximum network segment length (segments that are too long must be split) and the minimum frame size (frames that are too short are padded).

WiFi uses the *Collision Avoidance* method. It uses a star topology – in the middle of a star, there is a so-called *access point* (AP), to which individual stations are connected. So any transmission at any given time is actually a point-to-point operation between the station and the AP, and confirmation of delivery can be implemented. A node detects a collision by not receiving a confirmation, in which case it begins exponential waiting.

Ethernet

- History:
  - the first LAN attempts in Xerox company
  - standardization taken over by IEEE (Feb 1980 → IEEE 802)
  - two most common formats Ethernet II, IEEE 802.3
- Currently the top technology for LANs
  - can flexibly react to progressive HW evolution
  - can adapt to a wide range of transmission media
- Media access controlled by CSMA/CD
  - a „jam signal“ is sent by sender when a collision is detected
  - exponential waiting terminates after 16 attempts by an error
- Addresses:
  - 3 bytes prefix (producer, multicast...), 3 bytes node number
  - formerly „burned-in“ in NIC, nowadays programmable

Ethernet originated at the company Xerox, and it's a little surprising that a copier manufacturer was behind the most successful network technology. In February 1980, the IEEE (Institute of Electrical and Electronics Engineers) took over the standardization of the protocol, and it is said that this date is actually hidden behind the number 802 in the official standard identification (IEEE 802). This step caused a split in development, resulting, among other things, in a different format of the two most common versions of the protocol. In its early development phase, Ethernet had a number of competitors; IBM’s Token-ring technology was more successful for a time, but over the decades it became clear that Ethernet was best placed to keep up with hardware development.

The CSMA/CD method is used to deal with collisions on multipoint segments and half-duplex point-to-point segments, as described in the previous slide.

Ethernet addresses are 6 bytes long, of which the first three are a card manufacturer's prefix and the second three are the card's own number. The address is stored by the manufacturer into the card; formerly this was done in a fixed way, today it can be changed by software. But if no one has changed it, you can identify a manufacturer from an Ethernet address. Once upon a time, the rule existed that manufacturers guaranteed uniqueness of addresses, but that is no longer the case. It may well be that if you buy two cards from the same manufacturer, they will have the same address. Such cards can only be used in the same LAN simultaneously if they are separated by a router. Otherwise, at least one of the addresses needs to be changed. In addition to manufacturers' prefixes, there are other special prefixes for broadcasts and multicasts.

IEEE 802.3 standards

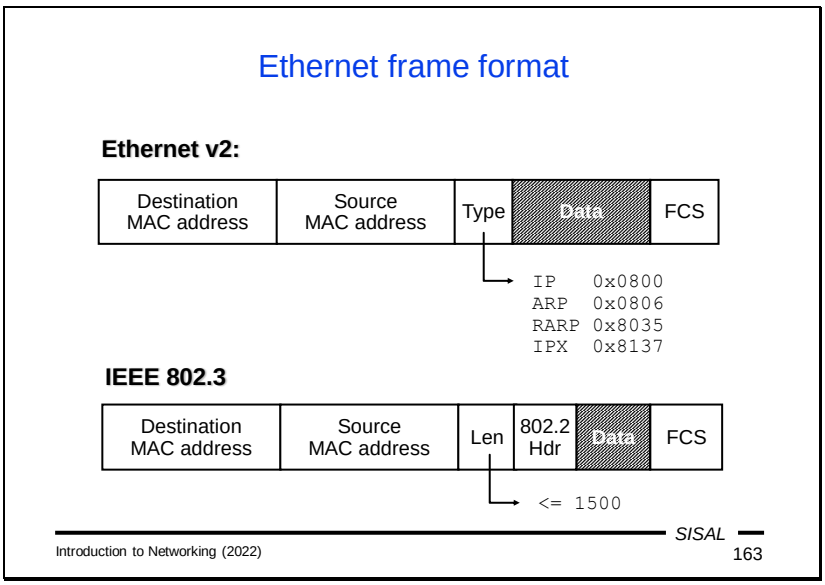
Standard	Year	Identification	Bandwidth	Media
802.3	1983	10BASE5	10 Mbit/s	thick coaxial cable
802.3a	1985	10BASE2	10 Mbit/s	thin coaxial cable
802.3i	1990	10BASE-T	10 Mbit/s	twisted pair (UTP)
802.3j	1993	10BASE-F	10 Mbit/s	fiber optic
802.3u	1995	100BASE-TX,FX	100 Mbit/s	UTP or fiber optic
802.3z	1998	1000BASE-X	1 Gbit/s	fiber optic
802.3ab	1999	1000BASE-T	1 Gbit/s	UTP
802.3ae	2003	10GBASE-SR,...	10 Gbit/s	fiber optic
802.3an	2006	10GBASE-T	10 Gbit/s	UTP
802.3ba	2010	100GBASE-SR	100 Gbit/s	fiber optic

Unlike RFC, IEEE standards are licensed.

This table shows how Ethernet gradually changed as hardware improved over time. This information will not be a subject of our examination.

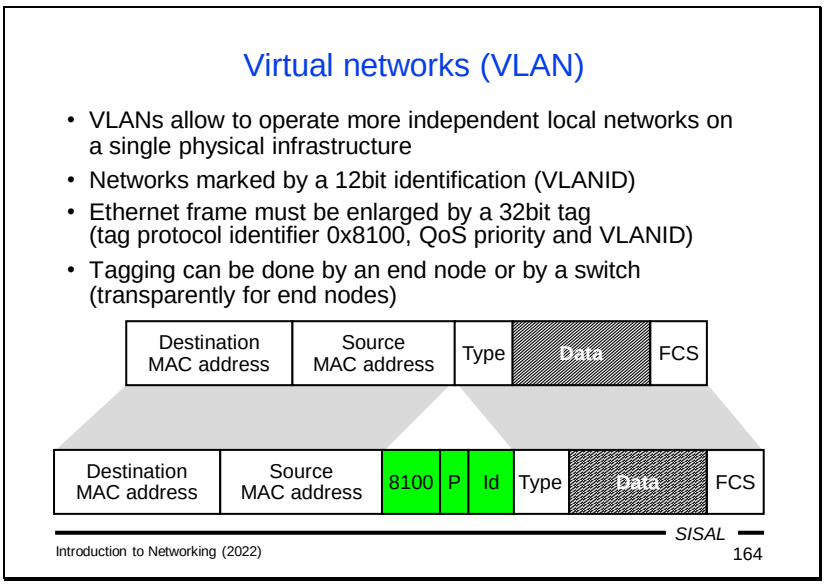
Important warning about protocol availability: IEEE standards, unlike RFCs, are **not public**. You have to buy them. Or borrow them from someone who has already bought them…

Slide 163



The long and turbulent evolution of Ethernet has also affected the structure of its frame format. While in the original concept the header contains a two-byte type of network protocol behind MAC addresses, in the IEEE version a frame length is in place and only behind it is the special added LLC header (IEEE 802.2).

Also note that, unlike in an IP header, the recipient's address precedes the sender's address here. This is so that hardware can detect as quickly as possible whether the incoming frame is intended for the node or not.

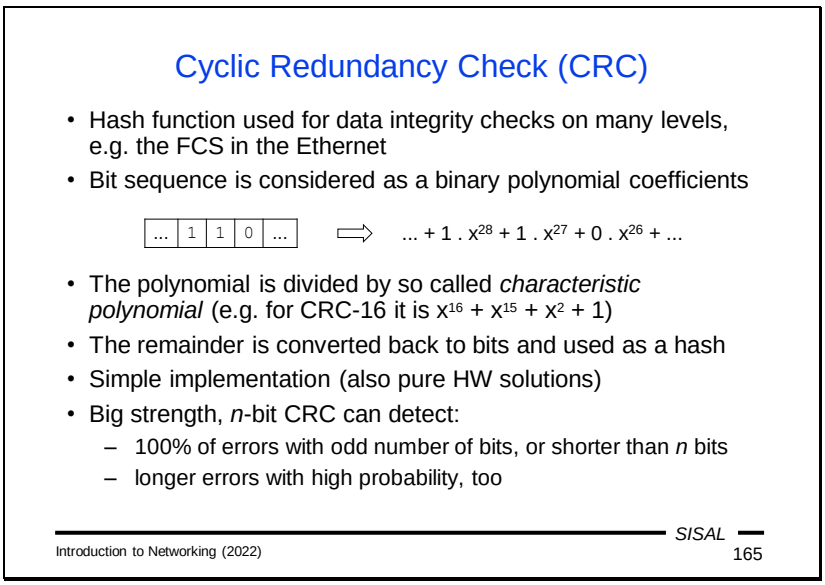


Ethernet version 2 frame format has enabled the deployment of an important network building tool, namely **virtual networks** (VLAN, Virtual LAN). It is used when one physical network needs to be split into multiple logical networks (e.g. when it is shared by multiple entities, or we want to separate segments with a different level of protection).

A principle of the method is to insert a four-byte portion (the so-called *VLAN tag*) into the frame, after MAC addresses, thus changing the frame **type** to a special “VLAN type”, and the inserted portion carries information about a virtual network number (VLANID). But an important feature is that this operation can happen **transparently**, without informing end stations about it. A station can be connected to a switch port that is configured as part of a network with a certain VLANID. In this case, from the station's point of view, the network will look like an ordinary network: the station sends normal frames, a switch inserts a VLAN tag with a correct VLANID into the frame and sends it on. Conversely, when the frame for that station arrives at the switch, the switch removes the VLAN tag and the station receives the frame as if it came from an ordinary network not using VLANs. The station does not have access to frames with a different VLANID that are transmitted over the network. This will logically separate traffic across different virtual networks. However, there are usually nodes in a network that need to have access to frames from all virtual networks (e.g. a central router). For them, the mount point is then configured as a so-called **trunk**, in which case the switch does not do any operations with the frame, and VLAN tag handling is left at the end node.

A slight complication is that each frame is lengthened by adding a tag, so that all the network devices that tagged frames pass through must be able to work with frames

longer than the maximum allowed. Another option is to inform all stations on the network that the maximum allowable frame is 4 bytes shorter than the standard.

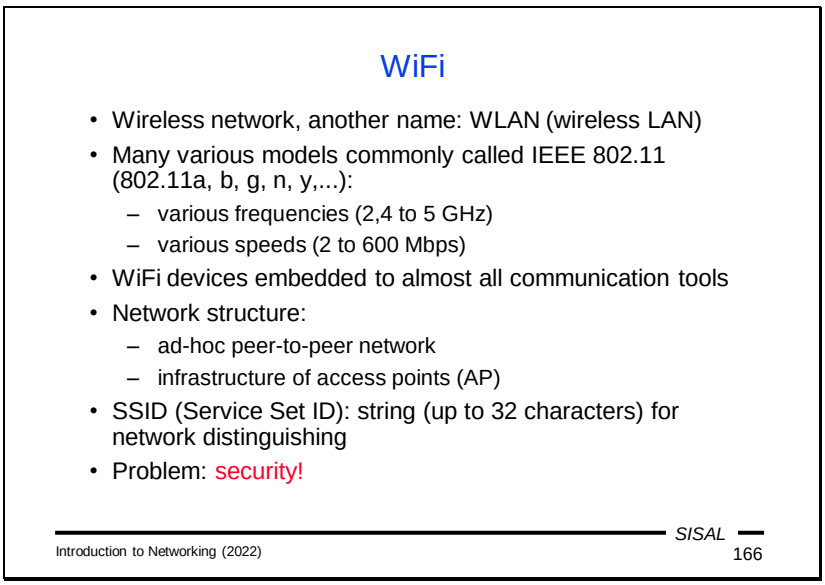


We have already encountered fields containing a content check several times when describing various PDUs. Examples include the IP header checksum, or the Frame Check Sequence in frame footer. Most of these control mechanisms use the **CRC** (Cyclic Redundancy Check) hashing function. This function is based on a division of polynomials, which is quite interesting, because it may seem like such a calculation would be quite complex, and yet it can be done very easily with hardware.

The basic idea is that we convert a sequence of bits into a polynomial with binary coefficients that correspond to individual bits and an order that corresponds to the number of bits. This polynomial is then divided by the so-called *characteristic polynomial*. This polynomial's degree must be equal to the number of bits in a control field. The polynomial that comes out as the remainder after the division is converted again into a sequence of bits, and this gives us the result of a fixed-size function corresponding to the order of the characteristic polynomial.

Despite its simple implementation, the method has surprisingly great power. It can detect all errors with an odd number of bits, all errors shorter than the number of bits of the control field, and even for errors of a different range it has a relatively high success rate.

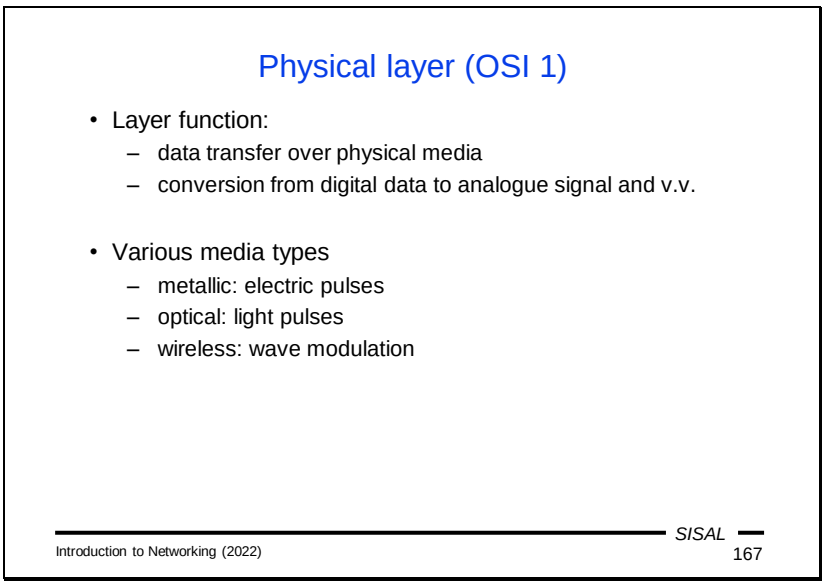
Slide 166



The term WiFi refers to a group of IEEE 802.11 protocols that are used for wireless communications in the unlicensed 2.4 and 5 GHz frequency bands. The name was originally meant to mean nothing, but over time it became a pun parodying Hi-Fi. Another name also used is Wireless LAN (WLAN). Unlike IEEE 802.3 protocols, however, the individual protocols of this group differ significantly from each other and do not form a coherent series.

Common features include the use of CSMA/CA as well as star network topology. Although a WiFi network can also be used in an ad-hoc peer-to-peer variant, it is usually used with a kind of infrastructure where there are *access points* (AP) at the center of each star, to which each individual terminal device connects within its reach. Planning the signal coverage of a campus or a building is complicated, because individual APs must either broadcast on non-overlapping frequency channels (of which there are only 13) or have a central control mechanism, which is relatively expensive.

A problem with WiFi networks is security. Since a potential attacker does not need physical access to the network (they can just stand outside the building), all possible security features must be taken into serious consideration on private networks. Each network is identified by a Service Set Identifier (SSID), but it is not used for security, only to distinguish different networks.

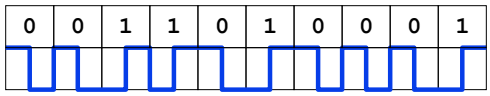


The last layer of the OSI model is the **physical** layer. Its task is to transmit a physical signal across a particular medium. For this, it is necessary to convert the digital information (bits) to analog (electrical pulses on metallic cables, light pulses on optical cables or various modulations of radio waves) first, and the opposite process is required when receiving.

Slide 168

Data transfer modes

- Analogue vs. digital
  - in fact, everything is analogue (e.g. electric current)
  - digital: thresholds exist that decide whether a signal level belongs to proper interval (less impact of noise)
  - convertors: A→D (and back) *codec* (coder/decoder) D→A *modem* (modulator/demodulator),
- Baseband vs. broadband
  - baseband carries directly encoded signal itself, the Ethernet uses so called Manchester:



  - broadband carries the basic signal and modulates it (phase, amplitude, frequency)

Let's first explain some of the concepts about transmission variants, which can be a little confusing.

Analog vs. digital transmission.

- In fact, every transmission is analog, because the world around us is analog. Whatever medium is used, its physical nature is analogous. So what does **digital transmission** mean? Basically, the difference is that the receiving device is not trying to interpret any possible incoming signal value, but only values that lie within a certain range. One signal range is interpreted as the value 1, another range as the value 0, and a signal outside these intervals is ignored. The transmitting device, on the other hand, tries to minimize the time when the signal is outside the proper intervals. The consequence of these features is that digital transmission is more resistant to interference.

Baseband vs. Broadband transmission.

- The name **baseband** suggests that this form of transmission uses a signal value to encode digital information directly. An example would be the propagation of an electrical signal over a metallic line. We might expect that the easiest way to do this would be to simply encode ‘one’ as a high signal value and ‘zero’ as a low value. With this approach, however, the sender's clock might start to diverge with the recipient's, and the bits sent and received would cease to match. So a "tick clock" must also be transmitted, which is why Ethernet uses so-called "Manchester" coding, where in each "tick" the signal value must be changed, with ‘zero’ being the change from high to low and ‘one’ reversed. Of course, even baseband transmission is in its physical nature analog, so even here in reality there are **not as sharp edges** as in the picture.

- The name **broadband** is not exactly semantically identifiable with the transport it denotes. The name is based on the fact that it is used for broadband transmissions. But its essence is in that data is coded using some **modulation** of the base signal. Either phase shift, amplitude change (known from radio as AM) or frequency change (FM) are used.

Slide 169

Unshielded Twisted Pair (UTP)

- The most common structured cabling media nowadays
- 4 pairs of copper conductors twisted around each other
  - twisting lowers both emission and reception of electromagnetic radiation (lower interference)
- 100Mb Ethernet uses only 2 pairs (cable can be “divided”)
- Connectors: RJ 45
- Cabling must reflect the nature of devices
  - nowadays usually the MDI/MDIX autodetection available

straight cable crossover

- Option: cable with metallic shielding (STP)

A commonly used connection method for stations on a local network is metallic cabling using the unshielded twisted pair (UTP) cable type. Again, the name is somewhat confusing, as there are **eight** wires in the cable. The essence of the name lies in the **way it is protected** from interference. It is based on the fact that if two conductors are twisted around each other at regular intervals, an electromagnetic field is created when an electric current passes through them, which forms a natural protection against a normal level of interference. In addition, the same signal is sent to both conductors, but with opposite polarity, so that any interference can be “subtracted” on the recipient's side. However, if we need to run the wiring in a highly “noisy” environment, an STP (Shielded Twisted Pair) cable can be used, which has additional metal **shielding** around the wiring.

A strange and interesting thing is why there are eight wires in the cable when Ethernet up to 100Mbps only uses four. However, this can be exploited to an advantage, using special forks to connect **two pairs** of computers with a single cable, each connection using a different four wires.

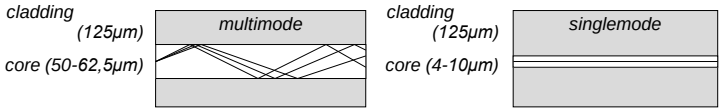
When connecting RJ 45 connectors to a UTP cable, it should be respected that the output pin on one side must be connected to the input pin on the other. So that we can use cables in which the same conductor is input on one side and output on the other (so-called **direct cables**), end stations (computers) use a different pin layout than switches. But this can also cause problems when we need to connect two identical devices (two computers or two switches). For such a connection, we should use a cable where the corresponding conductors are connected in reverse, i.e. a so-called **crossover cable**. Fortunately, when we connect a cable nowadays, network cards are able to negotiate with the counterparty when connecting a cable which wire will be used by which party for input and which for output, so in the normal case, if

both parties support this (so-called MDI/MDIX) auto-detection, there is no need to investigate the type of cable used. However, self-detection takes a certain amount of time, thus also increasing the time required to start communication, which may be unacceptable for some critical real-time applications.

Slide 170

Optical fiber

- Signal is carried as visible light through a fiber of SiO_2
  - high frequency, large bandwidth
  - low attenuation, no interference
- Disadvantages:
  - higher price, demanding installation, don't look into cable
- Fiber types:
  - singlemode fiber - light source: laser => single wave, higher radius, bandwidth („speed", not speed), price
  - multimode fiber - light source: LED



SISAL

Where there is a physical limitation on the spread of an electrical signal over a metallic cable, optical cabling is used. Information is propagated by light pulses that are transmitted via a silicon fiber. The light beam does not have an interference problem, has a very low attenuation and a high bandwidth ("speed"). The downside is a higher price and more demanding handling (the optical fiber has a significantly greater **minimum bending radius**).

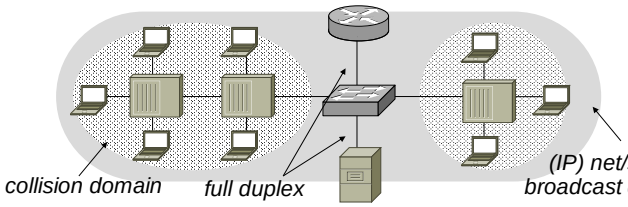
We recognize two kinds of optical fiber cables:

- *Single-mode* cables have a narrower silicon core and a laser is used as a light source. As a result, the refraction of light rays is significantly reduced, fundamentally increasing the range and transmission capacity. And the price. Therefore, these fibers are typically used for long-distance transmissions.
- In *multimode* cables, the silicon core is wider and LEDs can also be used as a light source. The beams are more refractory here, so the range and transmission capacity are smaller and are used most often for the LAN backhaul.

Slide 171

Network segmentation

- Repeaters connect segments on the physical layer
  - solves: larger radius (eliminates cable attenuation)
  - does not solve: throughput (collision probability increases)
  - terminology of structured cabling: *hub*
- Bridges connect segments on the data link layer
  - solves: larger throughput (by splitting the collision domain)
  - terminology of structured cabling: *switch*



Now let's look at an example of connecting a small local network to explain some other concepts. For reasons of both greater extent and greater throughput, we will need to include several network devices.

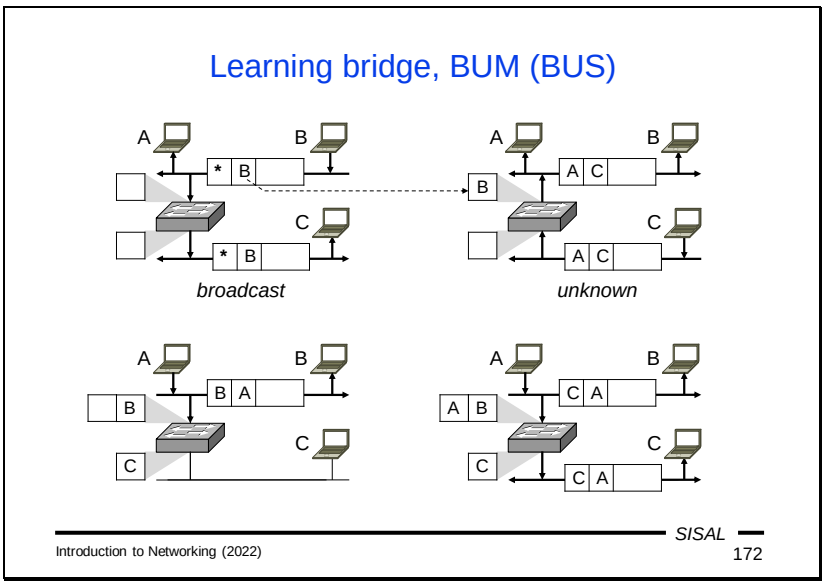
Some stations are far from the center and we can add *repeater* devices to the network to connect them. In structured wiring, the device is usually referred to as a “hub”. A repeater works at the **physical layer**, so its job is just to distribute the proper physical signal without understanding it in any way. This addresses the increase in signal range as attenuation is eliminated. But it does nothing to address the issue of network throughput or the number of collisions. The signal must be distributed from the source to all stations, so if there would be a collision without a repeater, it will occur with it as well. A repeater does not separate the so-called *collision domain*. Conversely, the likelihood of collisions with a repeater increases as a repeater takes time to transmit each frame from an input to an output interface, thereby extending the transmission time of a frame over a segment.

If we want to tackle the issue of **throughput**, we have to move up, to the **data-link** layer and use a *bridge*. In structured wiring, the term "switch" is usually used, which we've seen before. A bridge understands MAC addresses in transmitted frames (although its own MAC addresses don't feature here) and can send frames only where necessary. Each port of a bridge thus separates one collision domain, reducing the number of collisions rapidly. At the same time, it addresses throughput and, to some extent, safety, because each segment receives only traffic that belongs there according to its destination MAC address. Of course, such protection is not sufficient, because a potential attacker can fake an ARP transmission and convince our station to send its frames to him instead of the actual recipient.

To the central switch, two other nodes of exceptional status are connected in the image above – they are a central server and a central router. The two will most likely be connected separately, using a **full-duplex** line to give them maximum throughput.

The entire network, separated by a single router, represents one IP network (or subnet) as well as one *broadcast domain* (the area over which limited broadcasts are spread).

Slide 172



Switches typically operate in the so-called **learning bridge** mode. It means that for each port, they maintain a MAC address table of stations that are connected behind that port and maintain the table themselves by monitoring the traffic.

In the picture above, we see a switch that has just rebooted and whose tables are blank.

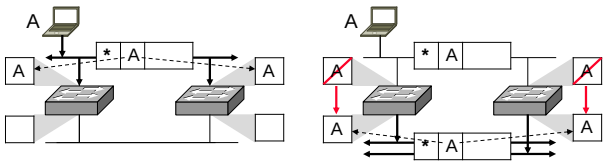
- Let's say the first frame to appear is a broadcast frame sent by station B from the upper segment. Since it is a **broadcast**, the switch must send it to all ports. At the same time, it looks at the sender's MAC address and adds the address of station B into the MAC address table of the upper segment.
- The second frame is sent by station C from the lower segment to station A on the upper segment. However, station A's location is still **unknown** at this time. Therefore, this frame must also be sent to all ports. At the same time, station C's MAC address will be added into the table for the lower port.
- If station A from the upper segment now sends a frame to station B, the switch can now detect that station B is on the same segment of the network (or switch port) and therefore there is **no need to send** the frame anywhere else! The station A entry will be added to the MAC address table, and from now on, the switch has the entire network map assigned to its ports.
- Therefore, when station A subsequently sends a frame to station C, the switch forwards it only to the port where it belongs. No other port or network segment will be burdened by this traffic anymore.

From step 3 onward, the switch will send all frames to the correct ports, with the exception of broadcasts, unknown unicasts and multicasts, which it will continue to send to all ports. That's why this behavior is sometimes called **BUS** (broadcast and unknown service) or **BUM** (broadcast, unknown and multicast).

Slide 173

Spanning Tree Algorithm

- If two segments are connected by two switches, the network is flooded by forwarded frames, the learning bridge doesn't work



- Reason: the graph contains a loop
- Solution: to find an acyclic subset, spanning tree
- Switches must agree, which acts as backup one (forwarding no data, only monitoring status)
- Protocol (STP) needs timeouts, switch port start is slow
  - usually, the STA can be suspended („faststart"), use carefully

Important points or connections in a network are sometimes backed up by redundant switches due to robustness. But this raises one fundamental problem. If both switches worked, the network would be flooded with frame forwarding, and the learning bridge method would fail.

Let's look again at the example in the picture above. Let's say that station A sends out a broadcast. When it arrives at the switches, they both assign station A's MAC address to their correct port address table, and **both** send the frame to the lower segment. Note that switches, unlike routers, do **not interfere** with the frame content. Therefore, both frames will arrive at the other switch over the lower segment. Both switches will respond by **changing the MAC address assignment**. This will "move" station A to the lower segment and make it unavailable for a time. But at the same time, they re-transmit the frame to the upper segment, forming an endless loop.

If we think of the network as a graph, where individual segments are vertices and switches are edges, the problem is **the circle** that we created by adding the second switch. The problem of reducing the graph and getting rid of the circle is known as the problem of **finding a spanning tree** of a graph. A distributed version of the algorithm is used in computer networks using the Spanning Tree Protocol (STP). Dropping an edge is in practice implemented as converting (some ports of the) switch to a **blocking** mode, in which frames are not forwarded and the switch merely monitors for a failure of the second switch, which is in a **forwarding** mode.

However, STP needs some time for its work. If we plug a station into a normal switch port, STP must first take place and the station will be unavailable for a few seconds. This may sometimes matter, and in that case, certain switch ports can be configured in a "fast-start" mode and the use of the protocol suppressed on them. By doing so,

the administrator **takes responsibility** for not creating a cycle with additional connections in the network.

Summary 8

- Describe the purpose and safety risks of ARP.

- Describe the purpose of the LLC and MAC layers.
- What is the difference between the physical and logical topology of a network?
- What is a collision, where can it occur and how is it dealt with?
- What is a VLAN for and how is it implemented?
- What is a CRC for and how does it work?

- Describe how data is transmitted depending on the underlying medium.
- Describe the various types of metallic and optical cables.
- What is a learning bridge and STP?

The End